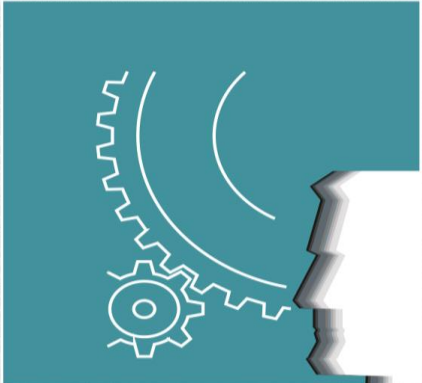


ΣΗΜΕΙΩΣΕΙΣ ΜΙΚΡΟΎΠΟΛΟΓΙΣΤΩΝ

Computer – Ανάλυση



ανάλυση
COMPUTER

ΕΡΓΑΣΤΗΡΙΟ ΕΛΕΥΘΕΡΩΝ ΣΠΟΥΔΩΝ
ΣΤΗΝ ΕΦΑΡΜΟΣΜΕΝΗ ΠΛΗΡΟΦΟΡΙΚΗ

**...λυση
σπουδων**

ΑΓ. ΑΝΔΡΕΟΥ 130-132 & ΓΟΥΝΑΡΗ, Τ.Κ. 26222, ΠΑΤΡΑ
ΤΗΛ: 310-097, 324-900 - ΤΗΛ/FAX: 313-547
E-MAIL: janelisj@otenet.gr, janelisk@otenet.gr

Περιεχόμενα

1. Εισαγωγή – Κύρια χαρακτηριστικά Z80.....	2
Διαφορά μικροεπεξεργαστή (Z80) – μικροελεγκτή (ATiny 2313) – Σήματα Z80	2
2. Κύρια μνήμη SRAM (Static RAM) και σύνδεση με Z80 - Ασκήσεις.....	3
3. Κύρια μνήμη DRAM (Dynamic RAM)	6
4. Αρχιτεκτονικές μνήμης Von Neumann και Harvard.....	7
5. Κύκλοι εντολής – μηχανής – ρολογιού – υπολογισμός bytes εντολής	8
Κανόνας υπολογισμού bytes μιας εντολής.....	8
Κανόνας υπολογισμού κύκλων μηχανής μιας εντολής.....	8
Κανόνας υπολογισμού κύκλων ρολογιού μιας εντολής.....	9
6. Σύγχρονη και ασύγχρονη επικοινωνία.....	10
7. Τρόποι διευθυνσιοδότησης Z80.....	11
8. Ακροδέκτες και εσωτερική δομή του Z80.....	13
Καταχωρητές Z80.....	14
9. Κώδικες Assembly Z80.....	15
10. Διακοπές Z80.....	20
11. Μετρητές και χρονιστές	22
12. Απευθείας (άμεση) προσπέλαση μνήμης (DMA) - ΑΠΜ	23
13. Κύκλοι εντολής Z80.....	24
Κύκλος προσκόμισης εντολής.....	24
Κύκλος ανάγνωσης/εγγραφής μνήμης.....	25
Κύκλος ανάγνωσης/εγγραφής περιφερειακής συσκευής.....	26
Κύκλος διακοπής INT.....	27
Κύκλος διακοπής NMI	28
Ομοιότητες – Διαφορές NMI και INT.....	28
Κύκλος διακοπής DMA.....	29
Κύκλος διακοπής HALT.....	30
14. PIO (parallel Input – Output)	31
Mode λειτουργίας PIO.....	31
Θέματα με το PIO.....	36
15. CTC (Clock Timer Circuit).....	41
16. Μικροελεγκτής ATiny2313	50
17. TIMER 0	60
Ασκήσεις με τον Timer	60
18. Port.....	62
Ασκήσεις με το Port.....	62
Ασκήσεις με το Port του Z80	65
19. Ασκήσεις με βήματα εντολών επεξεργαστή 8080A	66



1. Εισαγωγή – Κύρια χαρακτηριστικά Z80

Διαφορά μικροεπεξεργαστή (Z80) – μικροελεγκτή (ATiny 2313) – Σήματα Z80

Μια **πρώτη διαφορά** ενός μικροεπεξεργαστή και ενός μικροελεγκτή είναι ότι ο πρώτος περιέχει μόνο την ΚΜΕ και όχι μνήμες ή και περιφερειακές συσκευές που περιέχει ο δεύτερος. Επιπλέον, όλα τα κυκλώματα της Κ.Μ.Ε. μπόρεσαν να ενσωματωθούν σε ένα ολοκληρωμένο. Ο μικροελεγκτής αποτελεί ένα σύστημα που περιλαμβάνει στο ίδιο ολοκληρωμένο κύκλωμα: την ΚΜΕ, τις μνήμες των κατηγοριών RAM και ROM και τα απαραίτητα περιφερειακά κυκλώματα. Πρόκειται για **πλήρες υπολογιστικό σύστημα**. Μια **δεύτερη διαφορά** μεταξύ μικροεπεξεργαστή και μικροελεγκτή είναι η **αρχιτεκτονική μνήμης** που χρησιμοποιούν, δηλ. Von Neumann έναντι Harvard. Η πρώτη αρχιτεκτονική που χρησιμοποιείται από τον Z80, χρησιμοποιεί **ενιαίο χώρο διευθύνσεων για εντολές και δεδομένα**, ενώ η δεύτερη αρχιτεκτονική, που χρησιμοποιείται από τον ATiny 2313, χρησιμοποιεί **ξεχωριστούς χώρους μνήμης για εντολές και δεδομένα**. Λόγω της χρησιμοποιούμενης αρχιτεκτονικής Harvard, οι μικροελεγκτές αποθηκεύουν σε flash memory τα προγράμματα και σε μνήμη SRAM τα δεδομένα, και οι τιμές για τη διαμόρφωση του συστήματος υπάρχουν σε μνήμη EEPROM.

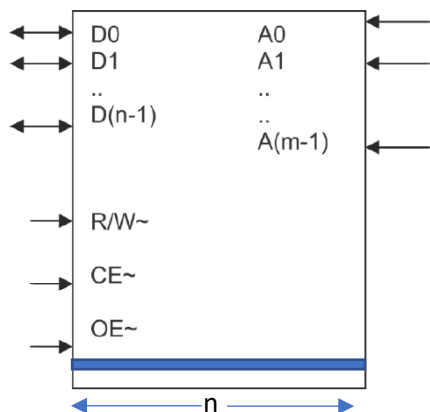
Επιπλέον ο μικροελεγκτής ATiny 2313 περιέχει ενσωματωμένα περιφερειακά κυκλώματα και οι ακροδέκτες του διαθέτουν **πολύπλεξη λειτουργιών**. Παράδειγμα μικροεπεξεργαστή είναι ο Z80 και παράδειγμα μικροελεγκτή είναι ο ATiny 2313. Ο μικροεπεξεργαστής Z80 έχει 8 – bit data bus και 16 – bit address bus, ενώ ο ATiny 2313 έχει data bus 8 – bit ή 16 – bit ανάλογα με το ποια μνήμη προσπελαύνει (εντολών ή δεδομένων).

Τα σήματα του Z80 διαιρούνται σε κατηγορίες, όπως:

- **Έλεγχος συστήματος:** επιτρέπουν την επικοινωνία της ΚΜΕ (Z80) με μνήμες/περιφερειακές συσκευές.
- **Έλεγχος ΚΜΕ:** επιτρέπουν μεταξύ άλλων το χειρισμό διακοπών (interrupts) από περιφερειακές συσκευές (I/O).
- **Έλεγχος διαύλων:** επιτρέπουν την επικοινωνία της ΚΜΕ (Z80) με τον ελεγκτή ΑΠΜ (Απευθείας Προσπέλαση Μνήμης – DMA).
- **Δίαυλος διευθύνσεων (Address bus):** τα σήματα αυτά επιτρέπουν εντοπισμό διευθύνσεων σε μνήμες και περιφερειακές συσκευές (ή αλλιώς συσκευές εισόδου – εξόδου).
- **Δίαυλος δεδομένων (Data bus):** τα σήματα αυτά επιτρέπουν τη μεταφορά εντολών/δεδομένων μεταξύ της ΚΜΕ (Z80) και των μνημών/περιφερειακών συσκευών ή αλλιώς συσκευών εισόδου – εξόδου).

Ο Z80 χρησιμοποιεί Assembly, βασισμένη σε Accumulator. Αυτό σημαίνει ότι οποιαδήποτε εντολή γίνεται μέσω του συσσωρευτή.

2. Κύρια μνήμη SRAM (Static RAM) και σύνδεση με Z80 - Ασκήσεις



- Η αρτηρία (δίαυλος) διευθύνσεων είναι από **A0** έως **A(m-1)**: συνολικά έχουμε m bits, άρα 2^m θέσεις μνήμης (θ.μ.).

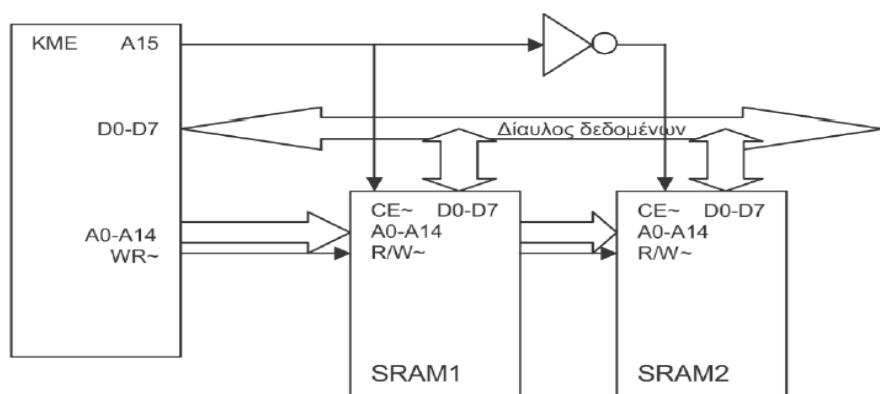
- Η αρτηρία (δίαυλος) δεδομένων είναι από **D0** έως **D(n-1)**: συνολικά έχουμε n bits. Αυτό σημαίνει ότι **κάθε μια θ.μ. έχει χωρητικότητα n bits**.

$$\text{Χωρητικότητα Κύριας Μνήμης} = 2^m \text{ θ.μ.} \times \frac{n \text{ bits}}{\text{θ.μ.}}$$

Το σήμα **R/W~** καθορίζει αν θα έχουμε ανάγνωση (1)/εγγραφή(0) μνήμης

Το σήμα **CE~** επιλέγει το ολοκληρωμένο κύκλωμα της μνήμης.

Το σήμα **OE~** (Output Enable) επιτρέπει την ανάγνωση του ολοκληρωμένου κυκλώματος της μνήμης.



Για να συνδέσουμε τις δύο μνήμες SRAM1 και SRAM2 με τον επεξεργαστή Z80, θα πρέπει να υπάρχει ένας κοινός δίαυλος δεδομένων (D0 – D7) για να μεταφέρει εντολές/δεδομένα από/προς τη μνήμη και επίσης ένας δίαυλος διευθύνσεων (A0 – A14) για τη διευθυνσιοδότηση των δύο ολοκληρωμένων κυκλωμάτων μνήμης.

Κάθε ΟΚΜ έχει συνολική χωρητικότητα $2^{15} \text{ θ.μ.} \times \frac{8 \text{ bits}}{\text{θ.μ.}} = 2^{15} \text{ bytes} = 2^{10} \times 2^5 \text{ bytes} = 32 \text{ KB}$. Τα δύο ΟΚΜ δεν μπορούν να επιλέγονται ταυτόχρονα, επομένως θα πρέπει να κατασκευάσουμε ένα χάρτη μνήμης, για να βρούμε το πως θα επιλέγονται. Κάθε ΟΚΜ έχει $2^{15} \text{ bytes} = 2^{15} \text{ θ.μ.} = 1000 \ 0000 \ 0000 \ 0000 = 0x8000 = 8000H = 8000h$.

Memory Map (Χάρτης Μνήμης)

SRAM1: $A_{15} \dots A_0$
0000 = 0000 0000 0000 0000
7FFF = 0111 1111 1111 1111

Επειδή το **A15 = 0**, σε όλο το εύρος των διευθύνσεων, χρησιμοποιείται για την επιλογή του SRAM1

SRAM2: 8000 = 1000 0000 0000 0000
FFFF = 1111 1111 1111 1111

Επειδή το **A15 = 1**, σε όλο το εύρος των διευθύνσεων, χρησιμοποιείται για την επιλογή του SRAM2.

8000 (Αρχική διεύθυνση του 2^{ου} ολοκληρωμένου μνήμης)

8000 (πλήθος θ.μ. που καταλαμβάνει το ολοκληρωμένο)

1)0000 (στην απευθείας άθροιση στο δεκαεξαδικό όταν βρούμε αποτέλεσμα από 16 και πάνω, αφαιρούμε το 16 και μένει ένα κρατούμενο). Από το τελικό αποτέλεσμα 10000h αφαιρούμε «1», οπότε απομένει FFFFh. ΣΕ



κάθε περιοχή διευθύνσεων κοιτάμε από αριστερά τα bits που μένουν **σταθερά** σε όλο το εύρος των διευθύνσεων και αυτά τα χρησιμοποιούμε για την επιλογή των ΟΚΜ. Στην πρώτη περιοχή διευθύνσεων, σταθερό είναι το bit A15 και μάλιστα ίσο με «0», ενώ στη δεύτερη περιοχή διευθύνσεων, σταθερό είναι και πάλι το bit A15 και μάλιστα ίσο με «1».

Σημείωση: οι **βασικές διαφορές** μεταξύ των μνημών **SRAM** και **DRAM** είναι ότι η πρώτη είναι πιο ακριβή, μικρότερης χωρητικότητας και μικρότερου χρόνου προσπέλασης και το βασικό στοιχείο μνήμης είναι το Flip-Flop. Αντίθετα η DRAM έχει μικρότερο κόστος, μεγαλύτερη χωρητικότητα, μικρότερη ταχύτητα και το βασικό στοιχείο μνήμης είναι ο πυκνωτής, ο οποίος επειδή εκφορτίζεται μέσα σε λίγο χρόνο, χρειάζεται ρεύμα αναζωογόνησης για να διατηρεί τα περιεχόμενά του.

Θέμα Φεβρουαρίου 2018

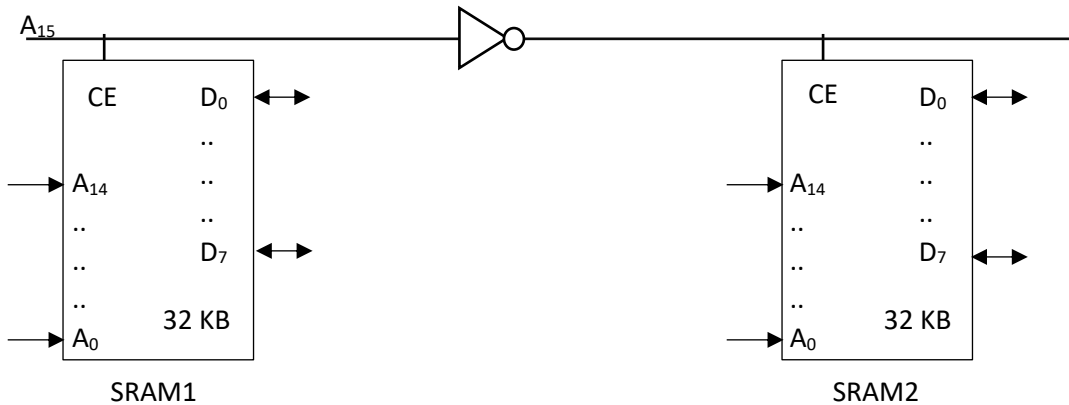
Δίνεται το παρακάτω σχήμα με δύο μνήμες SRAM. Έστω τώρα ότι αντί για δύο έχουμε τέσσερις μνήμες, όπου η καθεμία έχει χωρητικότητα 1 KB. Πως μπορούμε να τις συνδέσουμε, ώστε να έχουν εναλλακτικές περιοχές διευθύνσεων τις εξής:

0000 – 03FF

1000 – 13FF

3000 – 33FF

F000 – F3FF



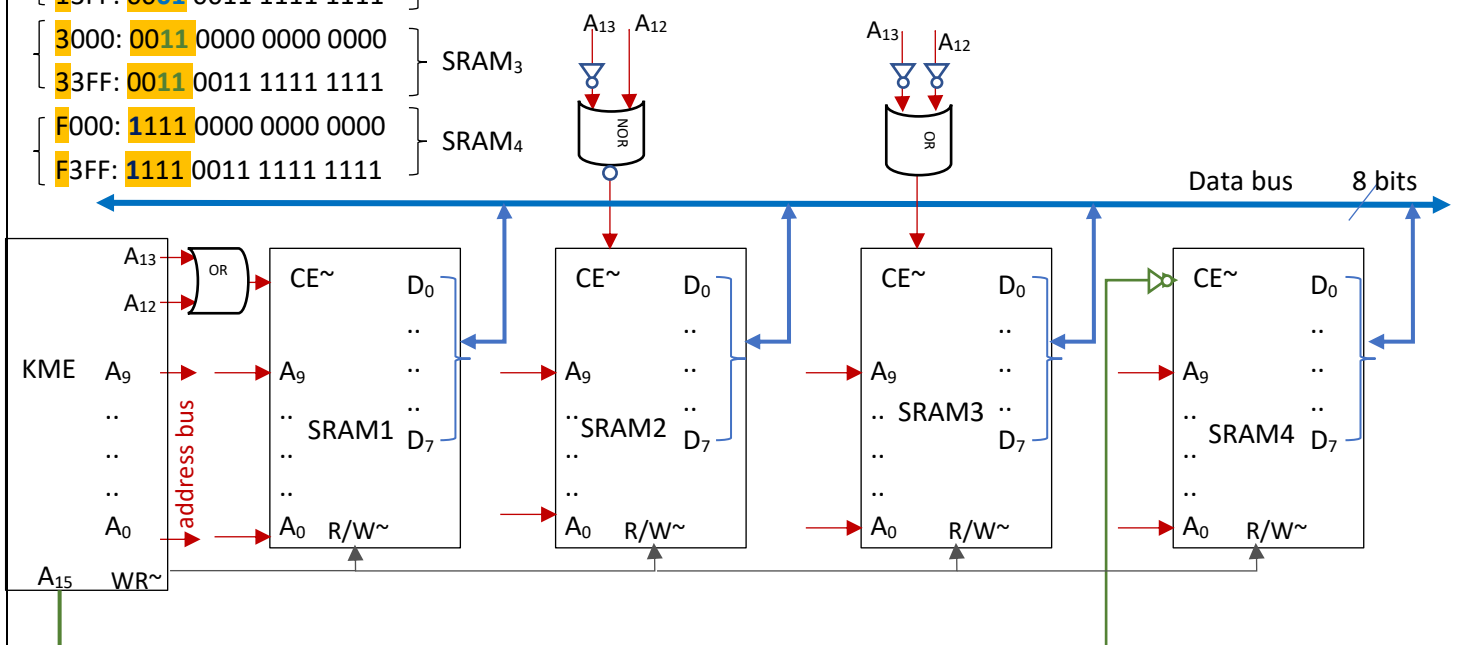
Λύση

Κάθε περιοχή διευθύνσεων μνήμης που δίνεται, έχει συνολική χωρητικότητα $400h = 0100\ 0000\ 0000 = 2^{10}$ θ.μ. = 2^{10} bytes (λόγω της οργάνωσης που έχουμε 1byte/θ.μ.). Αυτό σημαίνει ότι για να γεμίσουμε την κάθε περιοχή χρειαζόμαστε μνήμες SRAM του 1 KB. Για κάθε τέτοια μνήμη χρησιμοποιούμε ένα χάρτη διευθύνσεων (memory map) για να προσδιορίσουμε τα bits από το address bus που χρειάζονται για την επιλογή της κάθε μνήμης με τρόπο μοναδικό.

Χάρτης μνήμης (Memory Map)

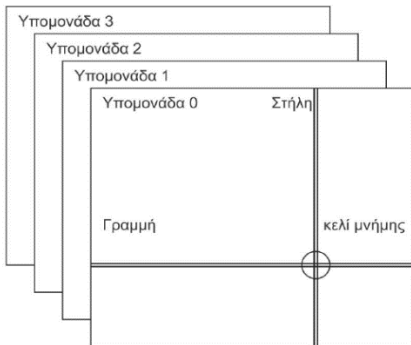
hex	A ₁₅	binary	A ₀	
0000:	0000	0000 0000 0000		SRAM ₁
03FF:	0000	0011 1111 1111		
1000:	0001	0000 0000 0000		SRAM ₂
13FF:	0001	0011 1111 1111		
3000:	0011	0000 0000 0000		SRAM ₃
33FF:	0011	0011 1111 1111		
F000:	1111	0000 0000 0000		SRAM ₄
F3FF:	1111	0011 1111 1111		

Από το χάρτη μνήμης βρίσκουμε τα σταθερά bits για κάθε μια περιοχή διευθύνσεων από τις τέσσερις συνολικά περιοχές που έχουμε, τα οποία χρησιμοποιούνται για την επιλογή του κάθε ΟΚΜ.



3. Κύρια μνήμη DRAM (Dynamic RAM)

Το χαρακτηριστικό μιας δυναμικής μνήμης είναι ότι αποτελείται από **υπομονάδες (banks)** και κάθε υπομονάδα είναι μια συλλογή από γραμμές και στήλες και η διασταύρωση γραμμών και στηλών δίνει το κάθε κελί

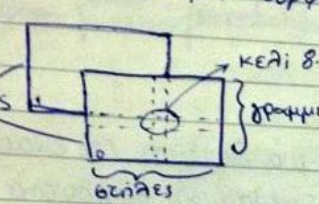


μνήμης (θέση μνήμης). Η διεύθυνση αποτελείται από τρία πεδία:

- **K – bit**: επιλέγει την υπομονάδα. Για να υπολογίσω το μέγεθός του, χρησιμοποιώ τον τύπο $2^x \geq \text{αριθμός υπομονάδων}$, όπου το x = μέγεθος του πεδίου K – bit.
- **Z – bit**: επιλέγει το πλήθος των γραμμών (rows) της κάθε υπομονάδας.
- **Y – bit**: επιλέγει το πλήθος των στηλών (columns) της κάθε υπομονάδας.

Το πόσα bits θα χρησιμοποιήσουμε για την επιλογή των πεδίων **Z – bit** και **Y – bit** είναι αυθαίρετο.

Ασκηση 1 Έστω ότι έχουμε μια φυσική διαύθυνση σε μια κατηγορία μνήμης SDRAM η οποία έχει 32-bit (31...0) και θέλω με αυτή τη διαύθυνση να κάνω φυσική οργάνωση στη μνήμη σύμφωνα με την ακόλουθη αρχιτεκτονική



Η χωρητικότητα ενός κελιού είναι συνήθως 8-bit.
Τυχαιο το πλήθος των bit που αφιερώνω στα γραμμές και στήλες

31		0
K-bit	Z-bit	Y-bit
Υπομονάδες		
1-bit → 2 υπομονάδες	30-21	20-0
31-bit	2^{10}	2^{21}
BANK	ROW	COLUMN

Σε αυτή την περίπτωση έχω λαβεί ότι τα bit είναι 16α με το μέγεθος του δίαυτου διαδίνσεων, αλλιώς συνήθως οι θέσεις μνήμης ορίζονται με λιγότερα bits. (με μοτίβο nx)

Θέμα Φεβρουαρίου 2019

2. α) Μια λογική διεύθυνση SDRAM των 32-bit (A31..A0), αποτελείται από τρία πεδία: Bank (A27..A26), Row (A25..A15), Bank (A14..A0). Να βρείτε το πλήθος των υπομονάδων, των γραμμών και των στηλών, σε κάθε υπομονάδα, (10%).

Λύση

Επειδή δίνεται το μέγεθος του πεδίου K – bit και είναι ίσο με «2» (A27 – A26) αυτό σημαίνει ότι **κατά μέγιστο** θα έχουμε 4 υπομονάδες (θα μπορούσαμε να είχαμε και τρεις). Όσον αφορά το πεδίο **Z – bit** που επιλέγει το πλήθος των γραμμών (rows) της κάθε υπομονάδας, αυτό περιέχει 11 bits, επομένως έχουμε σε κάθε υπομονάδα 2^{11} γραμμές. Όσον αφορά το πεδίο **Y – bit** που επιλέγει το πλήθος των στηλών (columns) της κάθε υπομονάδας, αυτό περιέχει 15 bits, επομένως έχουμε σε κάθε υπομονάδα 2^{15} στήλες.

4. Αρχιτεκτονικές μνήμης Von Neumann και Harvard

Υπάρχουν δύο αρχιτεκτονικές μνήμης που είναι η αρχιτεκτονική Von Neumann (τη χρησιμοποιεί ο Z80) και η αρχιτεκτονική Harvard (τη χρησιμοποιεί ο ATiny 2313). Με τον όρο **λέξη** εννοούμε μια θέση μνήμης.

Von Neumann

0000h-3FFFh EPROM
4000h-DFFFh RAM
E000h-EFFFh Περιφερειακά
F000h-FFFFh BOOT ROM

- Ενιαίος χώρος διευθύνσεων.
- Μικροεπεξεργαστής: 16-bit δίαυλος διευθύνσεων μπορεί να δει μέχρι και 65536 θέσεις μνήμης με το ίδιο μήκος λέξης.
- Τμήματα του χώρου διευθύνσεων για RAM, ROM, ή και περιφερειακά.

Harvard

Μνήμη flash 2 K των 13 bit για την αποθήκευση προγραμμάτων

256 θέσεις δεδομένων των 8 bit

- Ο χώρος διευθύνσεων προγράμματος είναι ανεξάρτητος από εκείνον των δεδομένων.
- Το μήκος των λέξεων στο χώρο προγράμματος μπορεί να είναι διαφορετικό από το μήκος των λέξεων στο χώρο των δεδομένων.
- Οι εντολές μπορούν να έχουν ως ορίσματα διευθύνσεις του χώρου των δεδομένων.

Πομπή: Έχουμε μια φυσική οργάνωση της μνήμης με την αρχιτεκτονική Von Neumann και ένα δίαυλο διευθύνσεων 16-bit. Ποιοι από τους παρακάτω συνδυασμούς οργάνωσης μνήμης είναι εφικτοί;

- 1) 64 KByte RAM και 16 KByte ROM
- 2) 32 KB RAM και 32 KB ROM
- 3) 16 KB RAM και 32 KB ROM
- 4) 16 KB RAM και 16 KB ROM

Ο δίαυλος διευθύνσεων είναι 16-bit, οπότε το μέγιστο εύρος είναι $2^{16} = 64 \text{ KByte}$. Επομένως το άθροισμα των μνημών δεν πρέπει να είναι μεγαλύτερο από το μέγιστο εύρος, αλλά πρέπει να είναι και κατά κάποιο ποσοστό μικρότερο, γιατί έχουμε και την EPROM και τα περιφερειακά. Οπότε δεν μπορεί να είναι ποτέ από 64 KB, αλλά ούτε και 160. Άρα οι συνδυασμοί (1) και (2) είναι αδύνατοι.

EPROM
RAM
ROM
Περιφερειακά

Με 16 bit στο δίαυλο διευθύνσεων (address bus) μπορούμε να έχουμε 2^{16} θ.μ. = 2^{16} bytes (λόγω της οργάνωσης που έχουμε 1 byte/θ.μ.) = $2^6 \times 2^{10}$ bytes = 64KB.



5. Κύκλοι εντολής – μηχανής – ρολογιού – υπολογισμός bytes εντολής

Κανόνας υπολογισμού bytes μιας εντολής

Για τον υπολογισμό των bytes μιας εντολής, ισχύει ο ακόλουθος κανόνας: χρησιμοποιούμε εξ' ορισμού ένα **byte για την αποθήκευση του κωδικού λειτουργίας της εντολής (opcode)** και τόσα επιπλέον bytes, όσοι είναι οι δεκαεξαδικοί αριθμοί που εμφανίζονται στην εντολή (δηλαδή για κάθε δύο δεκαεξαδικά ψηφία χρειαζόμαστε 1 byte, αφού αυτά είναι μεγέθους 8 bits).

STA A68BH → 3 bytes (1-byte για το opcode και 2 bytes για αποθήκευση του αριθμού A68Bh = 0xA68B).

ADD A, (3000H) → 3 bytes (1-byte για opcode και 2 bytes για αποθήκευση του αριθμού 3000h = 0x3000).

ADD A, B → 1 byte (μόνο για το opcode).

OUT (C), B → 1 byte (μόνο για το opcode).

LD HL, 4f28h → 3 bytes (1-byte για το opcode και 2 bytes για αποθήκευση του αριθμού 4f28h).

INDR → 1 byte (μόνο για το opcode).

Κύκλοι εντολής - μηχανής – ρολογιού

Ισχύει ότι **1 κύκλος εντολής** (instruction cycle) αποτελείται από έναν ή περισσότερους **κύκλους μηχανής** (MC – Machine Cycles) και ένας MC αποτελείται από έναν ή περισσότερους κύκλους **ρολογιού** (T).

Κανόνας υπολογισμού κύκλων μηχανής μιας εντολής

Κάθε εντολή εκτελείται σε έναν κύκλο εντολής. Για κάθε εντολή απαιτείται εξ' ορισμού ένας MC για την προσκόμιση και αποκωδικοποίηση της εντολής (fetch + decode), επίσης ένας MC για το διάβασμα δύο δεκαεξαδικών ψηφίων και τόσοι επιπλέον MC όσες είναι οι μεταφορές μεταξύ ΚΜΕ και μνήμης ή μεταξύ ΚΜΕ και περιφερειακών συσκευών.

STA A68BH → **4 MC** (1 MC εξ' ορισμού για fetch + decode, ένας δεύτερος MC για το διάβασμα του αριθμού A6, ένας τρίτος MC για το διάβασμα του αριθμού 8B και ένας τέταρτος MC για την αποθήκευση του A στη διεύθυνση A68BH). **Η συγκεκριμένη εντολή δεν ισχύει για τον επεξεργαστή Z80, αλλά για τον επεξεργαστή 8080 (διότι ο Z80 δεν έχει εντολή STORE).**

ADD A, (3000H) → **4 MC** (1 MC εξ' ορισμού για fetch + decode, ένας δεύτερος MC για το διάβασμα του αριθμού 30, ένας τρίτος MC για το διάβασμα του αριθμού 00 και ένας τέταρτος MC για την μεταφορά του περιεχομένου της θέσης μνήμης με διεύθυνση 3000H στον επεξεργαστή Z80, προκειμένου να αθροιστεί στη συνέχεια με το συσσωρευτή A).

ADD A, B → **1 MC** (1 MC εξ' ορισμού για fetch + decode). **Δεν** χρειάζονται επιπλέον MC, διότι δεν γίνεται κάποια μεταφορά μεταξύ ΚΜΕ/μνήμης ή μεταξύ ΚΜΕ/περιφερειακών συσκευών.



OUT (C), B → **2 MC** (1 MC εξ' ορισμού για fetch + decode και ένας ακόμη για τη μεταφορά του περιεχομένου του καταχωρητή B στην περιφερειακή συσκευή με διεύθυνση αυτή που δείχνει ο καταχωρητής C).

LD HL, 4f28h → **3 MC** (1 MC εξ' ορισμού για fetch + decode, ένας για το διάβασμα του αριθμού 4f και ένας τρίτος MC για το διάβασμα του αριθμού 28).

Κανόνας υπολογισμού κύκλων ρολογιού μιας εντολής

Για τον πρώτο MC απαιτούνται 4 κύκλοι ρολογιού (T) και για τους υπόλοιπους MC απαιτούνται συνήθως 3 T (κ.ρ.), εκτός και αν έχουμε μεταφορά από/προς περιφερειακή συσκευή (εντολές IN, OUT), οπότε τότε απαιτούνται 4 κ.ρ.

STA A68BH → **13T** (4T για τον πρώτο MC και για τους υπόλοιπους τρεις MC απαιτούνται τρεις T για τον καθένα, άρα συνολικά $4T + 3 \times 3T = 13T$).

ADD A, (3000H) → **13T** (4T για τον πρώτο MC και για τους υπόλοιπους τρεις MC απαιτούνται τρεις T για τον καθένα, άρα συνολικά $4T + 3 \times 3T = 13T$).

ADD A, B → **4T** (υπάρχει μόνο ένας MC).

OUT (C), B → **8T** (4T για τον πρώτο MC και 4T για το δεύτερο MC).

LD HL, 4f28h → **10T** (4T για τον πρώτο MC και για τους υπόλοιπους δύο MC απαιτούνται τρεις T για τον καθένα, άρα συνολικά $4T + 2 \times 3T = 10T$).

Στο επόμενο σχήμα φαίνεται η υποδιαίρεση μιας εντολής σε κύκλους μηχανής και η υποδιαίρεση κάθε κύκλου μηχανής σε κύκλους ρολογιού.

Ανάκληση Εντολής				Αποκωδικ οποίηση				Ανάγνωση Μνήμης				Ανάγνωση Μνήμης				Εγγραφή Μνήμης			
MC ₁				MC ₂				MC ₃				MC ₄							
T ₁	T ₂	T ₃	T ₄	T ₁	T ₂	T ₃		T ₁	T ₂	T ₃		T ₁	T ₂	T ₃		T ₁	T ₂	T ₃	

Εικόνα 1: Κύκλοι μηχανής και ρολογιού



7. Τρόποι διευθυνσιοδότησης Z80

LD A, (4F00h) ; $A \leftarrow (4F00h)$, **απόλυτη διευθυνσιοδότηση (absolute)**, όταν περιέχεται ένας αριθμός μέσα σε ().

LD A, 3Fh ; $A \leftarrow 3Fh$, **άμεση ή απευθείας διευθυνσιοδότηση**, όταν φορτώνουμε έναν καταχωρητή με αριθμό (συγκεκριμένη τιμή).

LD HL, 4f28H ; **άμεση ή απευθείας διευθυνσιοδότηση**, όταν φορτώνουμε τους καταχωρητές H, L με τις τιμές που φαίνονται στη συνέχεια: $H \leftarrow 4fH$, $L \leftarrow 28H$.

ADD A, B ; $A \leftarrow A + B$, **υπονοούμενη διευθυνσιοδότηση**, όταν υπονοείται ο A.

SUB B ; $A \leftarrow A - B$, **υπονοούμενη διευθυνσιοδότηση**, αφού και πάλι υπονοείται ο A.

INDR ; **υπονοούμενη διευθυνσιοδότηση**, μεταφέρεται το περιεχόμενο της θέσης μνήμης που δείχνει ο καταχωρητής C, στη θέση μνήμης που δείχνει το ζεύγος HL και στη συνέχεια αυξάνεται ο καταχωρητής B κατά «1».

JR CC, NN ; **σχετική διευθυνσιοδότηση**, αυτή χρησιμοποιείται πάντα από εντολές διακλάδωσης, αν ισχύει η συνθήκη CC, γίνεται μετάβαση στη θέση NN (ετικέτα). Το JR = Jump.

JR NZ, loop ; για όσο διάστημα το Z = 0 (δηλ. ισχύει το **NZ**), επιστροφή στην ετικέτα loop

JR Z, loop ; για όσο διάστημα το Z = 1 (δηλ. ισχύει το **Z**), επιστροφή στην ετικέτα loop

JR NC, loop ; για όσο διάστημα το C = 0 (δηλ. ισχύει το **NC**), επιστροφή στην ετικέτα loop

JR C, loop ; για όσο διάστημα το C = 1 (δηλ. ισχύει το **C**), επιστροφή στην ετικέτα loop

LD A, (BC + offset) ; **δεικτοδοτούμενη διευθυνσιοδότηση**. Φόρτωσε τον A με το περιεχόμενο της μνήμης (BC + offset). Δηλαδή, αν το ζεύγος καταχωρητών BC δείχνει στη διεύθυνση 100H και το offset (μετατόπιση) είναι 7H, τότε θα γίνει το άθροισμα $100 + 7 = 107H$ μέσα στις παρενθέσεις, θα εντοπιστεί το περιεχόμενο της θ.μ. με διεύθυνση 107H και αυτό θα μπει ως περιεχόμενο στον A (Accumulator).

RLC (IX + 4) ; **δεικτοδοτούμενη διευθυνσιοδότηση**. Κάνε αριστερή κυκλική ολίσθηση του περιεχομένου της θέσης μνήμης με διεύθυνση (IX + 4), δηλ. με διεύθυνση αυτή που δείχνει ο καταχωρητής IX αυξημένη κατά «4».

LD A, (BC) ; **έμμεση με καταχωρητή**. Φόρτωσε τον A με το περιεχόμενο της θέσης μνήμης που δείχνει το ζεύγος καταχωρητών BC. Μπορεί αντί για το ζεύγος BC να χρησιμοποιηθούν και τα ζεύγη HL και DE.

BIT 3, (HL) ; **έμμεση με καταχωρητή**. Εξετάζει το 3^ο bit της θ.μ. που δείχνει το ζεύγος HL.

OUT (C), B ; **έμμεση με καταχωρητή**. Εξάγει το περιεχόμενο του καταχωρητή B στην περιφερειακή συσκευή με διεύθυνση αυτή που δείχνει ο καταχωρητής C.

SET 7, A ; **bit address**. Θέτουμε το 7^ο bit του καταχωρητή A.

RESET 5, B ; **bit address**. Μηδενίζουμε το 5^ο bit του καταχωρητή B.



Θέμα Ιουνίου 2020

Ποια λειτουργία επιτελούν, ποια μέθοδο διευθυνσιοδότησης ακολουθούν, πόσα byte(s) μνήμης προγράμματος και ποίους κύκλους μηχανής χρειάζεται οι παρακάτω εντολές: 1. RLC (IX+4) 2. OUT (C), B

≡ B / U A - A -

Λύση

RLC (IX + 4) → δεικτοδοτούμενη διευθυνσιοδότηση.

2 bytes (ένα για το opcode και ένα για τον αριθμό 4),

3 MC (ένα MC για fetch + decode, ένα MC για το διάβασμα του 4 και ένα MC για τη μεταφορά της θ.μ. με διεύθυνση αυτή που δείχνει το άθροισμα IX + 4).

Λειτουργία: κάνει αριστερή κυκλική ολίσθηση του περιεχομένου της θέσης μνήμης με διεύθυνση (IX + 4), δηλ. με διεύθυνση αυτή που δείχνει ο καταχωρητής IX αυξημένη κατά «4».

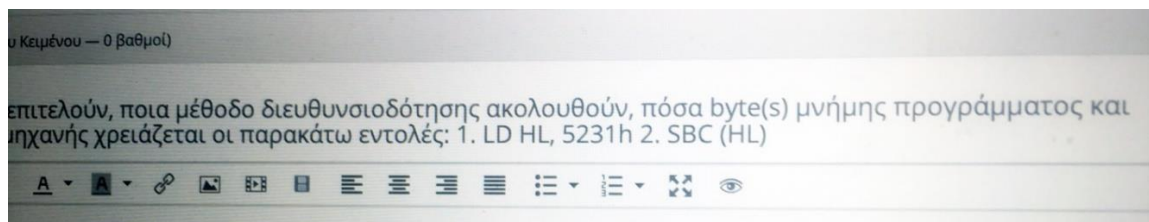
OUT (C), B → έμμεση με καταχωρητή.

1 byte (ένα για το opcode)

2 MC (ένα MC για fetch + decode, ένα για τη μεταφορά στην περιφερειακή με διεύθυνση αυτή που δείχνει ο καταχωρητής C).

Λειτουργία: Εξάγει το περιεχόμενο του καταχωρητή B στην περιφερειακή συσκευή με διεύθυνση αυτή που δείχνει ο καταχωρητής C.

Θέμα Σεπτεμβρίου 2020



Λύση

LD HL, 5231h → άμεση ή απευθείας διευθυνσιοδότηση, όταν φορτώνουμε τους καταχωρητές H, L με τις τιμές που φαίνονται στη συνέχεια: $H \leftarrow 52h$, $L \leftarrow 31h$,

3 bytes (ένα για το opcode, ένα byte για το 52h και ένα byte για το 31h),

3 MC (ένα MC για fetch + decode, ένα MC για το διάβασμα του 52h και ένα MC για το διάβασμα του 31h).

Λειτουργία: απευθείας φόρτωση του ζεύγους καταχωρητών H, L με συγκεκριμένες τιμές.

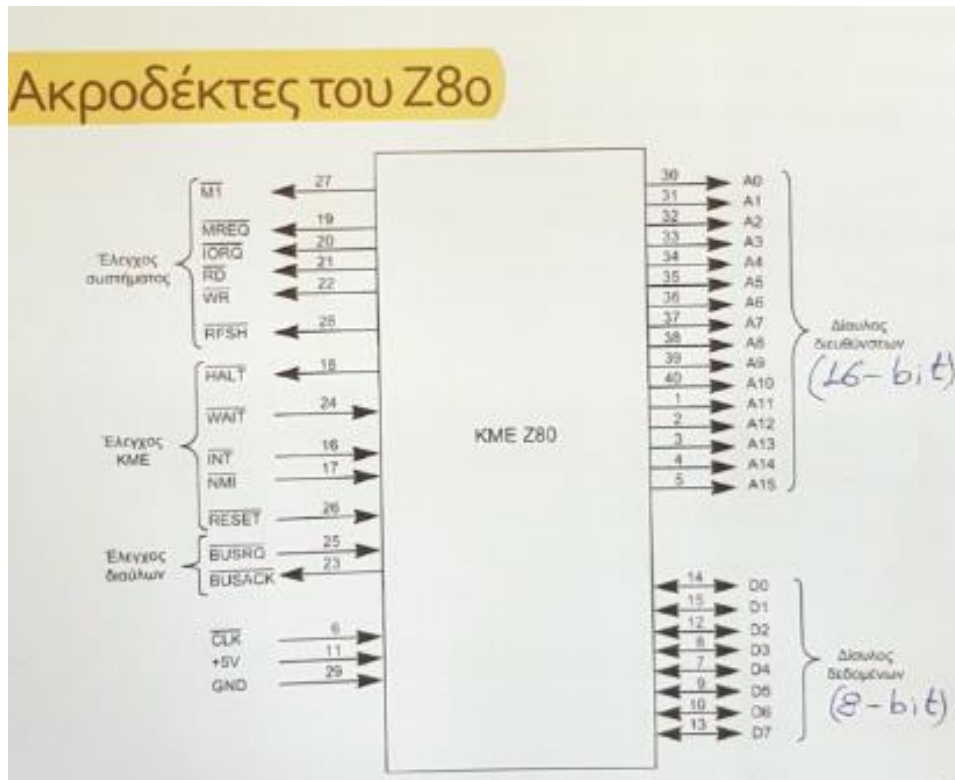
SBC (HL) → έμμεση με καταχωρητή, δηλ. $A \leftarrow A - (HL) - C$

1 byte (ένα για το opcode)

2 MC (ένα MC για fetch + decode και ένα για τη μεταφορά του περιεχομένου της θέσης μνήμης που δείχνει το ζεύγος HL στον επεξεργαστή).

Λειτουργία: Από το περιεχόμενο του A (Accumulator) αφαιρείται το περιεχόμενο της θέσης μνήμης που δείχνει το ζεύγος BC καθώς και το C.

8. Ακροδέκτες και εσωτερική δομή του Z80



Στα δεξιά υπάρχει ο 16 – bit δίαυλος διευθύνσεων (address bus) και ο 8 – bit δίαυλος δεδομένων (data bus). Στα αριστερά υπάρχουν:

CLK~: ρολόι του συστήματος

+5V: τάση λειτουργίας

GND: γείωση

BUSRQ~: Bus Request = Αίτηση παραχώρησης διαύλων (το σήμα αυτό προέρχεται από τον ελεγκτή ΑΠΜ).

BUSACK~: Bus Acknowledge = Αναγνώριση παραχώρησης διαύλων (το σήμα αυτό προέρχεται από την ΚΜΕ).

Reset~: σήμα διακοπής που προκαλεί επανεκκίνηση του επεξεργαστή

INT~: σήμα διακοπής προερχόμενο από μια περιφερειακή συσκευή (συσκευή E/E).

NMI~: σήμα διακοπής υψηλής προτεραιότητας που δεν μασκάρεται (παρεμποδίζεται).

Wait~: σήμα που εισάγει κύκλους αναμονής ή καθυστέρησης (wait states ή wait cycles) στον επεξεργαστή, προκειμένου να μπορέσει ο τελευταίος να επικοινωνήσει με αργές περιφερειακές συσκευές.

Halt~: σήμα εξόδου που δείχνει ότι ο επεξεργαστής έχει εισέλθει σε κατάσταση χαμηλής κατανάλωσης ενέργειας.

RFSH~: σήμα αναζωογόνησης (refresh) που διατηρεί τα περιεχόμενα της κύριας μνήμης αναλλοίωτα.

WR~: σήμα εγγραφής (αποθήκευσης) σε περιφερειακή συσκευή ή στην κύρια μνήμη.

RD~: σήμα ανάγνωσης από περιφερειακή συσκευή ή κύρια μνήμη.

IORQ~: σήμα που δείχνει ότι η προσπέλαση γίνεται σε περιφερειακή συσκευή (είτε για ανάγνωση, είτε για εγγραφή).

MREQ~: σήμα που δείχνει ότι η προσπέλαση γίνεται στην κύρια μνήμη (είτε για ανάγνωση, είτε για εγγραφή).

M1~: ειδικό σήμα ελέγχου, που ενεργοποιείται σε ειδικές περιπτώσεις.



Καταχωρητές Z80

Ο Z80 περιέχει δύο ειδών καταχωρητές: **γενικού** και **ειδικού** σκοπού.

ΔΙΑΝΥΣΜΑ ΔΙΑΚΟΠΩΝ IV (8-bit)	ΑΝΑΖΩΓΟΗΝΗΣΗ ΜΝΗΜΗΣ R (8-bit)
ΚΑΤΑΧΩΡΗΤΗΣΗΣ ΔΕΙΚΤΗ IX (16-bit)	
ΚΑΤΑΧΩΡΗΤΗΣΗΣ ΔΕΙΚΤΗ IY (16-bit)	
ΔΕΙΚΤΗΣ ΣΤΟΙΒΑ SP (16-bit)	
ΜΕΤΡΗΤΗΣ ΠΡΟΓΡΑΜΜΑΤΟΣ PC (16-bit)	

Εικόνα 2: Καταχωρητές ειδικού σκοπού

- **Μετρητής Προγράμματος (Program Counter PC):** 16-bit, πιο πρόσφατη εντολή που εκτελέστηκε στη μνήμη, για εντολές άλματος ή ανάγνωσης η νέα τιμή φορτώνεται αυτόματα στον PC.
- **Δείκτης Στοίβας (Stack Pointer SP):** 16-bit, η διεύθυνση της κορυφής της στοίβας. Δεδομένα μεταφέρονται στην κορυφή της στοίβας (λειτουργία push), δεδομένα μπορούν να ανακτηθούν από την κορυφή της στοίβας (λειτουργία pop).
- **Καταχωρητές Δείκτη (Index Registers – IX, IY):** 16-bit, διευθύνσεις για δεικτοδοτούμενους τρόπους διευθυνσιοδότησης.
- **Καταχωρητής Διανύσματος Διακοπών (Interrupt Vector Register - IV):** χρησιμοποιείται για την αποθήκευση των 8 MSBs της διεύθυνσης χειρισμού του εξωτερικού σήματος διακοπής (Interrupt Service Routine - ISR), τα 8 LSBs της διεύθυνσης αυτής παρέχονται από την περιφερειακή συσκευή, που ζητά εξυπηρέτηση με συγκεκριμένη διακοπή.
- **Καταχωρητής Ανανέωσης Μνήμης (Memory Refresh Register - R):** 8-bit, αποθηκεύεται ένας μετρητής υπεύθυνος για την ανανέωση (refresh) των μνημών
- **Καταχωρητής Κατάστασης (Status Register – F):** παρέχει διάφορες σημαίες (C, Z, N κ.λ.π.) για τις διάφορες καταστάσεις λειτουργίας.

ΚΥΡΙΟ ΣΥΝΟΛΟ ΚΑΤΑΧΩΡΗΤΩΝ	
ACUMMULATOR	FLAGS
A	F
B	C
D	E
H	L

Εικόνα 3: Καταχωρητές γενικού σκοπού

Οι καταχωρητές BC χρησιμοποιούνται είτε μεμονωμένα (δηλαδή ως καταχωρητές 8 – bit), είτε ως ζεύγος καταχωρητών (δηλαδή ως καταχωρητές 16 – bit). Το ίδιο ακριβώς ισχύει και για τα ζεύγη DE και HL. Ο A είναι ο συσσωρευτής (Accumulator) και χρησιμοποιείται πάντα μεμονωμένα και αποτελεί το βασικό καταχωρητή μέσω του οποίου γίνονται οι περισσότερες λειτουργίες, ενώ ο F είναι ο καταχωρητής κατάστασης (**Status Register**) με τα **Flags**.

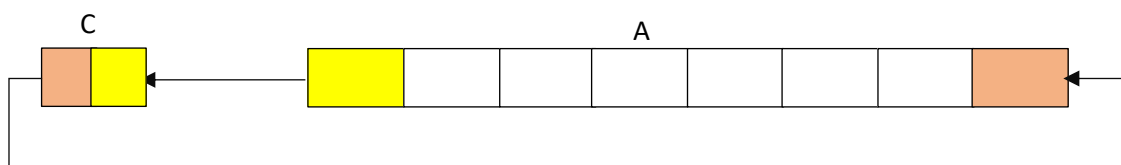
9. Κώδικες Assembly Z80

Κώδικας 1

Να αναπτυχθεί πρόγραμμα σε συμβολική γλώσσα του Z80 που να ελέγχει το πιο **σημαντικό** bit του αριθμού που είναι αποθηκευμένος στη θέση 3000h. Στην περίπτωση που το ψηφίο αυτό είναι «1», να αντιστρέφει τα 4 λιγότερο σημαντικά bits, αλλιώς να προσθέτει την τιμή «3» στον αριθμό αυτό και να αποθηκεύει το αποτέλεσμα στη διεύθυνση 3001h.

Λύση

LD A, (3000h) ; A ← (3000 h) δηλ. φορτώνουμε στον A το **περιεχόμενο της θ.μ. με διεύθυνση 3000h** (load)
 RLA ; Αριστερή κυκλική ολίσθηση του περιεχομένου του A, μέσω κρατούμενου C. Με τον τρόπο αυτό, το πιο σημαντικό ψηφίο του A (πιο αριστερό) μεταφέρεται στο Carry. RLA = Rotate Left



JR C next ; Αν το C = 1 (δηλ. αν το MSB = 1), μετάβαση στην ετικέτα next

ADD A, 03h ; Αλλιώς, αν το C = 0, πρόσθεση του «3» στον αριθμό που περιέχεται στον A

JP end ; άλμα στην ετικέτα end

next:

XOR 0Fh ; αντιστροφή των τεσσάρων λιγότερο σημαντικών ψηφίων του A, δηλ. $A \leftarrow A \text{ XOR } 0000\mathbf{1111}$, επομένως τα 4 λιγότερο σημαντικά bits του A αντιστρέφονται (ιδιότητα πύλης XOR).

end:

LD (3001h), A ; (3001h) ← A, αποθήκευση του περιεχομένου του A στη θ.μ. με διεύθυνση 3001h (store).

HLT ; τέλος προγράμματος

Παρατήρηση 1: Υπενθυμίζουμε ότι στις πύλες XOR ισχύει ότι:



Παρατήρηση 2: Σε περίπτωση που ζητηθεί να **τεθούν (να γίνουν “1”)/μηδενιστούν τα 4 λιγότερο σημαντικά bits** του αριθμού, θα αντικαταστήσουμε την εντολή XOR με την εντολή OR/AND αντίστοιχα, ως εξής:

$\text{XOR } 0Fh \leftrightarrow \text{OR } 0Fh$ ή $\text{XOR } 0Fh \leftrightarrow \text{AND } F0h$

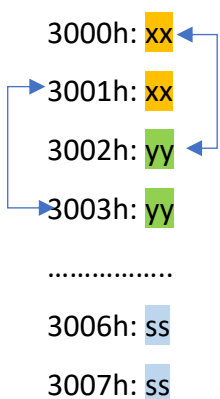


Σημείωση: στην Assembly του Z80 είναι **διαφορετικές οι εντολές άλματος JR και JP**. Η πρώτη εντολή χρησιμοποιείται πάντα σε συνδυασμό με τα flags, π.χ. η εντολή **JR C, label** (άλμα αν C = 1), η εντολή **JR NC, label** (άλμα αν C = 0), η εντολή **JR Z, label** (άλμα αν Z = 1), η εντολή **JR NZ, label** (άλμα αν Z = 0) κ.ο.κ. Η δεύτερη εντολή χρησιμοποιείται όταν κάνουμε άλματα (διακλαδώσεις) χωρίς να εξετάζεται η τιμή κάποιου flag, π.χ. η εντολή **JP (HL)** σημαίνει ότι στον $PC \leftarrow HL$, η εντολή **JP (IX)** σημαίνει ότι στον $PC \leftarrow IX$, η εντολή **JP nn** σημαίνει ότι στον $PC \leftarrow nn$ (label) κ.ο.κ.

Κώδικας 2

Να αναπτυχθεί πρόγραμμα σε συμβολική γλώσσα του Z80 που να αθροίζει **δύο 16 – bit αριθμούς** που είναι αποθηκευμένοι στις θέσεις μνήμης με διευθύνσεις 3000h και 3002h και να αποθηκεύει το άθροισμά τους στη διεύθυνση 3006h.

Λύση



Με κίτρινο χρώμα συμβολίζεται ο ένας 16-bit αριθμός και με πράσινο ο άλλος 16-bit αριθμός. Με γαλάζιο χρώμα συμβολίζεται το άθροισμά τους.

Κάθε θέση μνήμης στον Z80 έχει μήκος 8 bits. **Αυτό σημαίνει ότι κάθε 16 – bit αριθμός καταλαμβάνει δύο θ.μ.** Επομένως ο πρώτος αριθμός είναι αποθηκευμένος στις θέσεις μνήμης με διευθύνσεις 3000h – 3001h και ο δεύτερος αριθμός είναι αποθηκευμένος στις θέσεις μνήμης με διευθύνσεις 3002h – 3003h. **Επιπλέον, θα πρέπει να αθροίσουμε τα bytes ίδιας τάξης μεταξύ τους**, δηλ. τα λιγότερο σημαντικά bytes και τα περισσότερα σημαντικά bytes.

LD A, (3000h)	; A ← το λιγότερο σημαντικό τμήμα (byte) του 1 ^{ου} ορίσματος
LD B, (3002h)	; B ← το λιγότερο σημαντικό τμήμα (byte) του 2 ^{ου} ορίσματος
ADD A, B	; A ← A + B, άθροισμα των δύο λιγότερων σημαντικών bytes των δύο αριθμών
LD (3006h), A	; (3006h) ← A (άθροισμα των δύο λιγότερο σημαντικών τμημάτων των 2 αριθμών)
LD A, (3001h)	; A ← το πιο σημαντικό τμήμα του 1 ^{ου} ορίσματος
LD B, (3003h)	; B ← το πιο σημαντικό τμήμα του 2 ^{ου} ορίσματος
ADC A, B	; A ← A + B + Carry (τυχόν κρατούμενο που προέκυψε από την 1 ^η άθροιση)
LD (3007h), A	; (3007h) ← A (άθροισμα των δύο πιο σημαντικών τμημάτων των 2 αριθμών)
HLT	



Παρατήρηση: Στον Z80 δεν υπάρχουν εντολές LOAD και STORE, υπάρχει εντολή μόνο LOAD (LD), η οποία χρησιμοποιείται και ως εντολή STORE. Έχει σημασία η **σειρά των ορισμάτων των εντολών**, για να προσδιορίσουμε το ποια από τις δύο εντολές εκτελείται. Πιο συγκεκριμένα, αν η εντολή LD έχει τη μορφή:

load: LD καταχωρητής, θέση μνήμης

store: LD θέση μνήμης, καταχωρητής

Κώδικας 3

Να αναπτυχθεί πρόγραμμα σε συμβολική γλώσσα του Z80 που να αθροίζει **δύο 32 – bit αριθμούς** που είναι αποθηκευμένοι στις θέσεις μνήμης με διευθύνσεις 3000h και 3004h και να αποθηκεύει το άθροισμά τους στη διεύθυνση 3008h.

Λύση

Κάθε θέση μνήμης στον Z80 έχει μήκος 8 bits. Αυτό σημαίνει ότι κάθε 32 – bit αριθμός καταλαμβάνει τέσσερις θ.μ. Επομένως ο **πρώτος αριθμός** είναι αποθηκευμένος στις θέσεις μνήμης με διευθύνσεις 3000h – 3003h και ο **δεύτερος αριθμός** είναι αποθηκευμένος στις θέσεις μνήμης με διευθύνσεις 3004h – 3007h. Επιπλέον, θα πρέπει -όπως πριν- να αθροίσουμε **τα bytes ίδιας τάξης μεταξύ τους**.

LD C, 04h ; Ο καταχωρητής C θα χρησιμοποιηθεί ως **μετρητή επαναλήψεων**, αρχικοποίηση του μετρητή με «4».

LD HL, 3000h ; HL $\leftarrow$ τιμή 3000h, είναι η διεύθυνση έναρξης

LD DE, 3008h ; DE $\leftarrow$ τιμή 3008h, είναι η διεύθυνση προορισμού του αθροίσματος

SCF ; τίθεται η σημαία του κρατούμενου (γίνεται «1»), SCF = Set Carry Flag

CCF ; συμπληρώνεται η τιμή του κρατούμενου ($C \leftarrow C'$), επομένως αρχικά μηδενίζεται, CCF = Clear Carry Flag

loop:

LD A, (HL) ; A $\leftarrow$ (HL), φορτώνει στον A το **περιεχόμενο** της θ.μ. με διεύθυνση **3000h, 3001h, 3002h, 3003h**

ADD HL, 4 ; HL $\leftarrow$ HL + 4, για μετάβαση στη θ.μ. με διεύθυνση **3004h, 3005h, 3006h, 3007h**

LD B, (HL) ; B $\leftarrow$ (HL), φορτώνει στον B το περιεχόμενο της θ.μ. με διεύθυνση **3004h, 3005h, 3006h, 3007h**

ADC A, B ; A $\leftarrow$ A + B + Carry, αρχικά το κρατούμενο είναι «0», διότι από πριν το μηδενίσαμε.

LD (DE), A ; (DE) $\leftarrow$ A, αρχικά το άθροισμα θα αποθηκευτεί στη διεύθυνση **30008h**, μετά στη διεύθυνση **3009h**, μετά στη διεύθυνση **300Ah** και τέλος στη διεύθυνση **300Bh**.

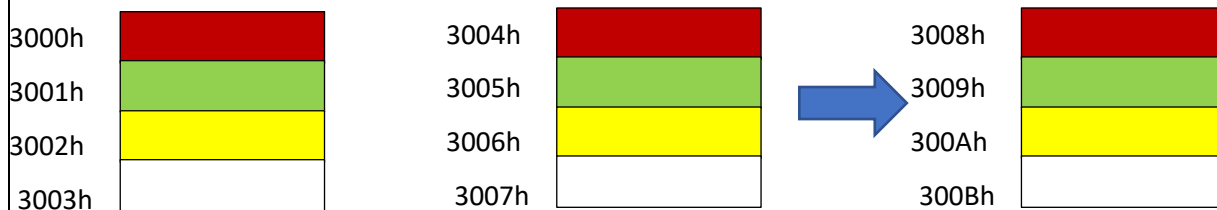
INC DE ; DE $\leftarrow$ DE + 1, αύξηση της διεύθυνσης προορισμού κατά 1 θέση

SUB HL, 3 ; HL $\leftarrow$ HL – 3, αρχικά με αυτή τη μείωση το ζεύγος HL δείχνει στη διεύθυνση 3001h, μετά στην 3002h κ.ο.κ.

DEC C ; C $\leftarrow$ C – 1, μείωση μετρητή επαναλήψεων μετά από κάθε άθροιση κατά «1».

JR NZ, loop ; Για όσο διάστημα ο καταχωρητής C $\neq$ 0, επιστροφή στην ετικέτα loop, για την επόμενη άθροιση
; NZ = No Zero, δηλ. το Z = 0, κάτι που σημαίνει ότι οι επαναλήψεις **δεν** έχουν ολοκληρωθεί. Όταν, μετά από τις διαδοχικές μειώσεις, ο καταχωρητής C = 0, τότε το Z = 1, και σταματά η επανάληψη.

HLT



Κώδικας 4

2. Να γραφεί και να σχολιαστεί ο κώδικας που να αντιγράφει τα περιεχόμενα των θέσεων 3000(hex)-300F(hex) στις θέσεις 4000(hex)-4000F(hex) αντίστοιχα (μικροεπεξεργαστής Z80). (Βαθμολογία: 2.50)

Λύση

Η μεταφορά αυτή θα πραγματοποιηθεί **μέσω του A**, byte προς byte.

```
LD BC, 0Fh      ;BC ← 0Fh (πλήθος επαναλήψεων), άρα το ζεύγος αυτό χρησιμοποιείται ως μετρ. επαναλήψ.
LD HL, 3000h    ;HL ← 3000h (διεύθυνση έναρξης της μεταφοράς)
LD DE, 4000h    ;DE ← 4000h (διεύθυνση προορισμού της μεταφοράς)

loop:
LD A, (HL)      ;A ← (HL), δηλ. φορτώνουμε στον A το περιεχόμενο της θ.μ. με διεύθυνση αυτή που δείχνει το HL
LD (DE), A      ;(DE) ← A, μεταφορά του περιεχομένου του A στη διεύθυνση που δείχνει το DE
INC HL          ;HL ← HL + 1 αύξηση της διεύθυνσης έναρξης
INC DE          ;DE ← DE + 1 αύξηση της διεύθυνσης προορισμού
DEC BC          ;BC ← BC - 1, μετά από κάθε μεταφορά, το ζεύγος BC μειώνεται κατά «1»
JR NZ, loop     ;Αν έχουμε NZ = No Zero, δηλ. Z = 0 (δεν έχουν ολοκληρωθεί οι επαναλήψεις), επιστροφή στο loop
HLT
```

Παρατήρηση: Η όλη προηγούμενη διαδικασία μεταφοράς, μπορεί να γίνει πιο **σύντομα με την εντολή LDIR**, η οποία εκτελεί την εξής λειτουργία: $(DE) \leftarrow (HL)$, $DE \leftarrow DE + 1$, $HL \leftarrow HL + 1$, $BC \leftarrow BC - 1$

```
LD BC, 0Fh
LD HL, 3000h
LD DE, 4000h
LDIR
```

Κώδικας 5

Να αναπτύξετε κώδικα σε συμβολική γλώσσα του Z80 που να αναζητά στους αριθμούς που είναι αποθηκευμένοι στις θέσεις **3002h μέχρι 3002h + N** **τον αριθμό που είναι αποθηκευμένος στη θέση 3000h**. Το πλήθος των αριθμών N είναι αποθηκευμένο στη θέση 3001h.

Λύση

```
LD B, (3001h)      ;B ← (3001h), ο καταχωρητής B χρησιμοποιείται ως μετρητής επαναλήψεων (N)
```



LD A, (3000h) ;A $\leftarrow$ (3000h), δηλ. στον A καταχωρείται ο αριθμός που θα αναζητηθεί

LD HL, 3002h ;HL $\leftarrow$ 3002h, αρχική θέση αναζήτησης

loop:

CP (HL) ; A - (HL), σύγκριση περιεχομένου θέσης μνήμης που δείχνει το HL με τον A

JR Z, end ;Αν A = (HL), τότε η διαφορά A - (HL) = 0, οπότε Z = 1 και άλμα στην ετικέτα end.

INC HL ;αλλιώς, αν δεν βρέθηκε ο αριθμός τότε HL $\leftarrow$ HL + 1

DEC B ;B $\leftarrow$ B - 1, μετά από κάθε μεταφορά, ο B μειώνεται κατά «1»

JR NZ, loop ;Για όσο ο καταχωρητής B $\neq$ 0, το Zero flag δεν ενεργοποιείται, δηλαδή Z = 0, δηλ. NZ, οπότε δεν έχουν ολοκληρωθεί οι επαναλήψεις και επιστροφή στο loop

end:

HLT

Θέμα Ιουνίου 2020

1. Θεωρείστε ένα μικροπολογιστικό σύστημα, με χρήση του Z80 . Για τα τέσσερα αριστερότερα ψηφία του αριθμού μητρώου σας: K3K2K1K0, γράψτε τον κώδικα της βέλτιστης λύσης για τη δημιουργία ενός αντιγράφου K1K0 (δεκαδικό σύστημα) θέσεων, ξεκινώντας από τη θέση K3K200 (δεκαεξαδικό σύστημα). Σημείωση: δεν θεωρείτε σωστή καμία άλλη λύση, εκτός από τη βέλτιστη.

Λύση

LD BC, K1K0 ;πλήθος θέσεων αναζήτησης

LD HL, K3K200h ;αρχική διεύθυνση αναζήτησης

LD DE, K3K2K1K0h ;αφού πρώτα μετατρέψουμε τον δεκαδικό K1K0 σε δεκαεξαδικό K1K0.

LDIR

π.χ. για AM = 1058100, έχουμε K3K2K1K0 = 1058 και ο κώδικας είναι ο εξής:

LD BC, 3Ah ;58₁₀ = 3Ah

LD HL, 1000h

LD DE, 103Ah ;αφού πρώτα μετατρέψουμε τον δεκαδικό K1K0 σε δεκαεξαδικό K1K0, δηλ. 58₁₀ = 3Ah.

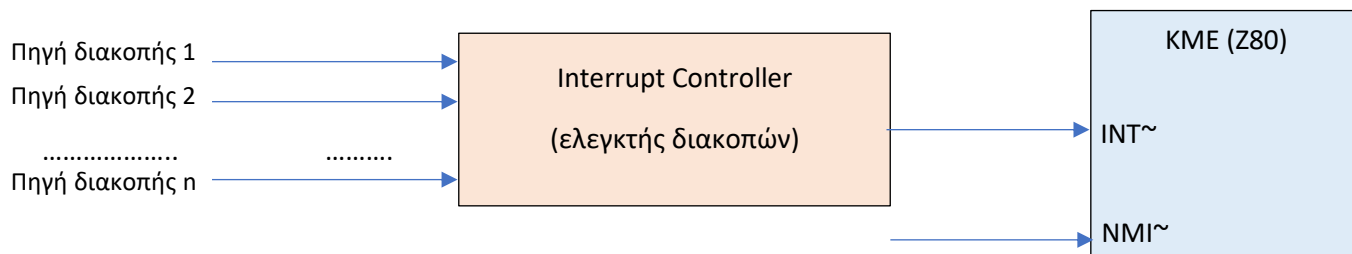
LDIR

10. Διακοπές Z80

Οι διακοπές αποτελούν έναν τρόπο για να διακοπεί η ροή ενός προγράμματος, έτσι ώστε να εξυπηρετηθεί ένα έκτακτο γεγονός. Οι διακοπές προέρχονται από περιφερειακές συσκευές (ή αλλιώς συσκευές E/E) και κατευθύνονται προς την ΚΜΕ. **Η διακοπή πάντα αναφέρεται στη διακοπή λειτουργίας της ΚΜΕ.** Προκειμένου να **εξυπηρετηθεί μια διακοπή**, αποθηκεύει η ΚΜΕ την τρέχουσα διεύθυνση του PC (ΜΠ) καθώς και τα περιεχόμενα των καταχωρητών γενικού σκοπού και κάνει άλμα στη διεύθυνση της **Ρουτίνας Εξυπηρέτησης Διακοπής (ISR = Interrupt Service Routine)**, όπου και εκτελεί μια έτοιμη υπορουτίνα. Πιο συγκεκριμένα, **πριν** γίνει η μετάβαση στη διεύθυνση μιας ISR στην κύρια μνήμη, θα πρέπει να γίνει αποθήκευση των περιεχομένων των καταχωρητών και αυτό πραγματοποιείται με την εκτέλεση δύο συνεχόμενων εντολών:

EX AF, AF'
EXX

Αφού ολοκληρωθεί η εξυπηρέτηση μιας διακοπής, μέσω της εκτέλεσης της υπορουτίνας, επιστρέφει ο έλεγχος του προγράμματος στη διεύθυνση της επόμενης εντολής από εκείνη που εκτελούνταν όταν συνέβη η διακοπή. Επειδή πολλές πηγές διακοπών πρέπει να οδηγούν τη μοναδική γραμμή (INT~) ενός επεξεργαστή, θα πρέπει να γίνει χρήση ενός **ελεγκτή διακοπών (Interrupt Controller)**, ο οποίος επιτρέπει την **ιεράρχηση των διακοπών** με διάφορες μεθόδους (round robin, απόλυτη ιεράρχηση κ.λ.π). Αυτό σημαίνει ότι κάθε φορά εξυπηρετείται **μόνο μια διακοπή** και πιο συγκεκριμένα αυτή που έχει τη **μεγαλύτερη** προτεραιότητα.



Κάθε διακοπή σχετίζεται -όπως προαναφέρθηκε- με μια συγκεκριμένη και ταυτόχρονα διαφορετική Ρουτίνα Εξυπηρέτησης Διακοπής (ISR). Ο **καθορισμός της 16 – bit διεύθυνσης της Ρουτίνας Εξυπηρέτησης Διακοπής (Z80)** προκύπτει από:

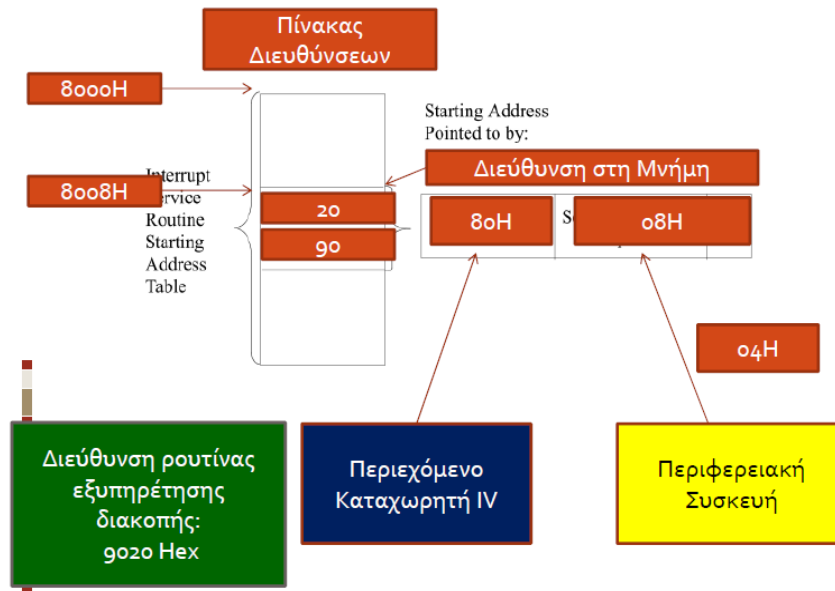
1. Την περιφερειακή συσκευή που προκάλεσε τη διακοπή (8 λιγότερο σημαντικά ψηφία της διεύθυνσης)
2. Από τον πίνακα διακοπών (Interrupt Vector) (8 περισσότερο σημαντικά ψηφία της διεύθυνσης)
3. Από τα δύο προηγούμενα, σχηματίζεται μια σταθερή διεύθυνση.

Για να προκύψει η διεύθυνση μιας ISR θα πρέπει πρώτα να σχηματίσουμε την 16-bit διεύθυνση από τη συσκευή που προκάλεσε τη διακοπή και από τον καταχωρητή IV (Interrupt Vector) και στη συνέχεια με βάση αυτή τη διεύθυνση που προέκυψε, να μεταβούμε μέσα σε ένα πίνακα διακοπών από όπου θα εντοπίσουμε τη διεύθυνση της ISR (με χρήση little – endian αποθήκευσης), όπως φαίνεται από το παράδειγμα που ακολουθεί. Σε αυτό υποθέτουμε ότι το **περιεχόμενο του IV είναι 80H** και το **περιεχόμενο της περιφερειακής συσκευής που προκάλεσε τη διακοπή είναι 04H**. Σε αυτό το περιεχόμενο πραγματοποιείται αριστερή λογική ολίσθηση,



οπότε διπλασιάζεται. Σχηματίζεται η διεύθυνση 8008H, η οποία μέσα στον πίνακα διευθύνσεων περιέχει την τιμή 20H και η επόμενη θέση μνήμης περιέχει την τιμή 90H. Η τελική διεύθυνση της ISR είναι η 9020H, διότι χρησιμοποιείται η αρχιτεκτονική little - endian.

Παράδειγμα: INT Mode 2 (Z80)



Εικόνα 4: Τρόπος δημιουργίας διεύθυνσης ISR

Υπάρχουν **τρία είδη διακοπών** στον Z80:

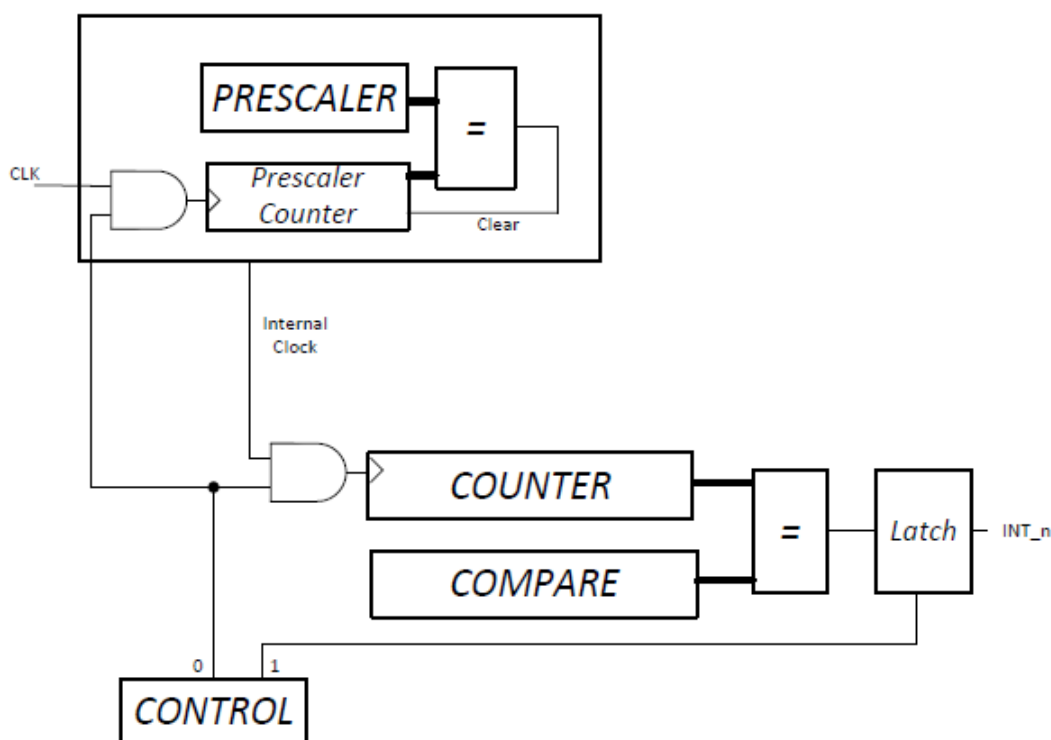
- διακοπές τύπου **INT[~] (Interrupt)**, οι οποίες μασκάρονται (παρεμποδίζονται).
 - o IM0: Interrupt Mode 0.
 - o IM1: Interrupt Mode 1.
 - o IM2: Interrupt Mode 2. Υπάρχουν 127 διαφορετικές ISRs (πιο συνήθης κατηγορία διακοπών).
- διακοπές τύπου **NMI[~] (Non Maskable Interrupts)**, οι οποίες δεν μασκάρονται (παρεμποδίζονται), επειδή όταν συμβούν, πρέπει οπωσδήποτε να εξυπηρετηθούν (έχουν μεγαλύτερη προτεραιότητα από τις διακοπές τύπου INT).
- διακοπές τύπου **RST[~]**, η επανεκκίνηση θεωρείται είδος διακοπής.

Η διακοπή INT[~] αναγνωρίζεται (εξυπηρετείται) μόνο αν αυτό έχει επιτραπεί, με ενεργοποίηση του αντίστοιχου εσωτερικού Flip/Flop (IFF1). Όταν το IFF1 γίνεται «0» τότε αδρανοποιούνται (μασκάρονται) οι διακοπές τύπου INT[~], για να δοθεί προτεραιότητα στις διακοπές τύπου NMI[~]. Αυτό συμβαίνει διότι μια εντολή INT[~] μπορεί και μασκάρεται. Από τις τρεις καταστάσεις λειτουργίας του INT (IM0, IM1, IM2), η πιο συνήθης είναι η IM2. Είναι αυτή που περιέχει 127 διαφορετικές ρουτίνες εξυπηρέτησης διακοπής. Για να μπορέσει να προκύψει μια διακοπή INT[~] τύπου mode – 2, πρέπει να έχει εκτελεστεί η εντολή IM2, να έχει ενεργοποιηθεί το IFF1 (να γίνει «1») και επίσης και η γραμμή INT[~].

11. Μετρητές και χρονιστές

Λογικά κυκλώματα μέτρησης και παράλληλης φόρτωσης, των οποίων η τιμή μεταβάλλεται με βάση κάποιους παλμούς. Ένα περιφερειακό κύκλωμα του μικροεπεξεργαστή Z80 που ονομάζεται **CTC** (Clock – Timer Circuit) μπορεί να λειτουργήσει είτε ως **μετρητής παλμών** στην είσοδό του είτε ως **χρονιστής**, ανάλογα με το πως τον έχουμε προγραμματίσει.

- **Χρονιστής (timer):** η τιμή του μεταβάλλεται με βάση κάποιο εσωτερικό ρολόι, σταθερής συχνότητας. Υπάρχουν χρονιστές των 8 - bit και των 16 - bit: συνοδεύονται συχνά από ένα **διαιρέτη συχνότητας (Prescaler)**, π.χ. 1:32. Αυτός ο διαιρέτης συχνότητας που υπάρχει στον Timer παίρνει ένα σήμα εξωτερικού ρολογιού (CLK) και το υποδιαιρεί σε μικρότερες συχνότητες ρολογιού (internal clock). Ένας χρονιστής προκαλεί διακοπή στον επεξεργαστή (INT_n) όταν η τιμή ενός καταχωρητή του μετρητή (COUNTER) γίνει ίση με μια προκαθορισμένη τιμή (COMPARE), λόγω αύξησης ή μείωσης της.
- **Μετρητής (counter):** η τιμή του αλλάζει με βάση παλμούς που δίνονται σε έναν εξωτερικό ακροδέκτη, (άρα ουσιαστικά μετράει παλμούς). Ένας μετρητής προκαλεί διακοπή στον επεξεργαστή όταν συμβεί υπερχειλίση, δηλαδή όταν η τιμή τους φτάσει το μηδέν ή ξεπεράσει το μέγιστο (π.χ. 255 για 8-bit)



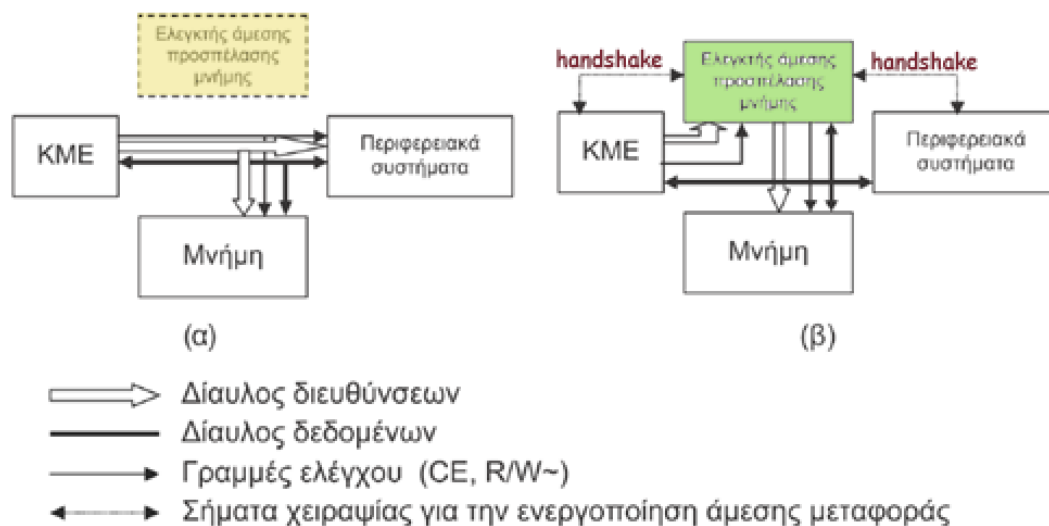
Εικόνα 5: Η σημασία του διαιρέτη συχνότητας (Prescaler)

12. Απευθείας (άμεση) προσπέλαση μνήμης (DMA) - ΑΠΜ

Η χρήση της **απευθείας προσπέλασης μνήμης** είναι ένα χαρακτηριστικό που χρησιμοποιείται όταν πρέπει να πραγματοποιηθεί μεταφορά δεδομένων μεταξύ κύριας μνήμης και περιφερειακών συσκευών, χωρίς τη μεσολάβηση της ΚΜΕ. Σε ένα σύστημα που **δεν** υποστηρίζει την άμεση προσπέλαση της μνήμης από τις περιφερειακές συσκευές, η μεταφορά των δεδομένων θα πρέπει να γίνει από την ΚΜΕ.

- Μεταφορά 1-byte: Περιφερειακή Συσκευή – ΚΜΕ – Εσωτερικός Καταχωρητής – Κύρια Μνήμη.
- Επαναλαμβανόμενη διαδικασία που εισάγει **καθυστέρηση**, ακόμα και αν η ΚΜΕ δεν έχει άλλη εργασία να επιτελέσει.
- Ο χρόνος απόκρισης της ΚΜΕ είναι ένα επιπρόσθετο μειονέκτημα, σε μια αίτηση μεταφοράς από μια περιφερειακή συσκευή.

Στην Εικόνα που ακολουθεί φαίνεται η χρήση της τεχνικής ΑΠΜ.



α) χωρίς τη χρήση άμεσης προσπέλασης,

β) με τη χρήση ελεγκτή άμεσης προσπέλασης.

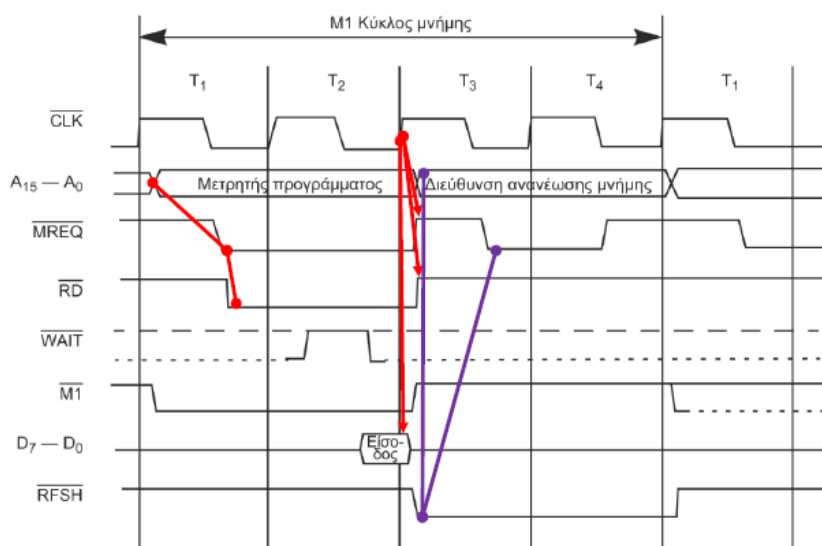
Εικόνα 6: Σύγκριση από τη μη χρήση και τη χρήση της τεχνικής ΑΠΜ (DMA)

Στην περίπτωση που **χρησιμοποιείται η τεχνική ΑΠΜ ή DMA (Direct memory Access)**, ο ελεγκτής άμεσης προσπέλασης μνήμης (DMAC = DMA Controller) αναλαμβάνει τον έλεγχο του διαύλου διευθύνσεων και των γραμμών ελέγχου. Η ΚΜΕ αρχικοποιεί κατάλληλα τον ελεγκτή, ώστε να γνωρίζει ο τελευταίος το πλήθος των δεδομένων και τις κατάλληλες διευθύνσεις, από τις οποίες θα γίνει η ανάγνωση/εγγραφή των δεδομένων αυτών. Πραγματοποιείται η μεταφορά των δεδομένων μεταξύ του περιφερειακού συστήματος και της μνήμης. Η μεταφορά των δεδομένων ξεκινάει με τα κατάλληλα σήματα χειραψίας (handshake). Μέσω αυτών των σημάτων παραχωρείται ο έλεγχος των διαύλων από την ΚΜΕ στον ελεγκτή ΑΠΜ (DMAC). Ο ελεγκτής αξιοποιεί τους διαύλους σε χρονικές στιγμές που δεν χρησιμοποιούνται από την ΚΜΕ ή απομονώνει την ΚΜΕ για όσο διάστημα διαρκεί η μεταφορά που αυτός πραγματοποιεί.

13. Κύκλοι εντολής Z80

Κύκλος προσκόμισης εντολής

Προσκόμιση Εντολής ...

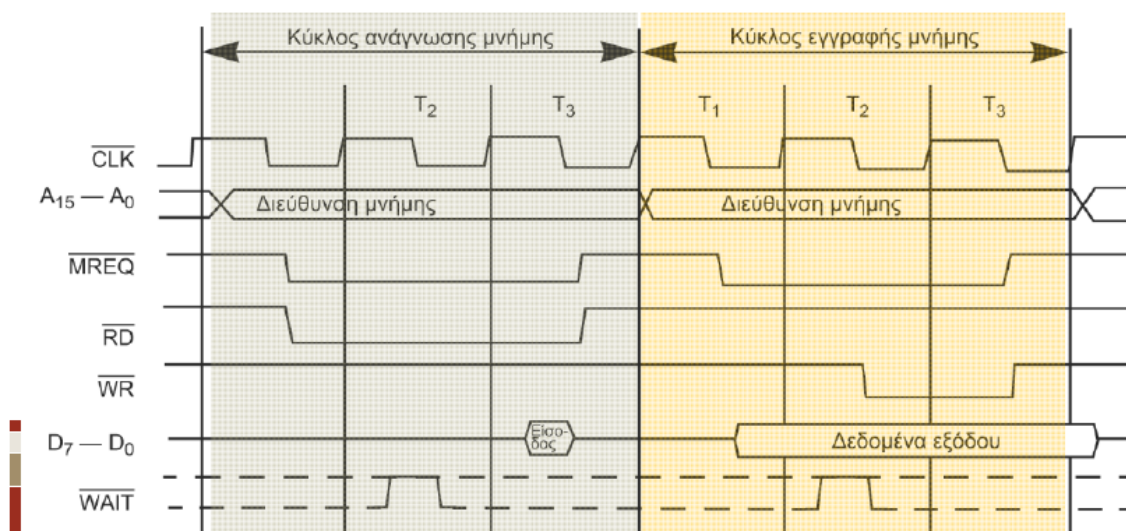


- **T1, T2:** Προσκόμιση Εντολής (Opcode Εντολής → D7-D0)
- **T3, T4: Refresh**
 - Ενεργοποίηση κύκλου RFSH~, εσωτερικές λειτουργίες στον Z80 (instruction decode),
 - Δεν γίνεται ενεργοποίηση του σήματος RD~ (καμία ανάγνωση από μνήμη).

Ο πρώτος κύκλος μηχανής αποτελείται από 4T (κύκλους ρολογιού). Στο address bus $A_{15} - A_0$, ο μετρητής προγράμματος (PC) μεταφέρει στους δύο πρώτους κύκλους ρολογιού (T_1, T_2) τη διεύθυνση της εντολής που εκτελείται και στους υπόλοιπους δύο κύκλους ρολογιού (T_3, T_4) το address bus μεταφέρει τη διεύθυνση ανανέωσης της κύριας μνήμης. Το σήμα **MREQ'** όταν γίνει ενεργό («0»), δηλώνει **προσπέλαση κύριας μνήμης** και αυτό συμβαίνει σε δύο περιπτώσεις: στην αρχή όταν εντοπίζεται από τον PC η διεύθυνση της εντολής που θα εκτελεστεί και στους κύκλους ρολογιού T_3 και T_4 όταν γίνεται ανανέωση των περιεχομένων της κύριας μνήμης. Το σήμα **RD'** ενεργοποιείται στην αρχή για ανάγνωση του κωδικού λειτουργίας της εντολής και το data bus $D_7 - D_0$ μεταφέρει τον κωδικό λειτουργίας της εντολής που θα εκτελεστεί στην ΚΜΕ. Αυτή η μεταφορά είναι έγκυρη στο πεδίο Είσοδος. Το σήμα **RFSH'** ενεργοποιείται όταν γίνεται η ανανέωση των περιεχομένων της κύριας μνήμης. Τέλος, το σήμα **M1'** ενεργοποιείται («0») σε ειδικές περιπτώσεις και μια από αυτές είναι ο κύκλος προσκόμισης και αποκωδικοποίησης της εντολής, που αποτελείται από τους κύκλους ρολογιού T_1 και T_2 . Το σήμα **Wait** ενεργοποιείται σε ειδικές περιπτώσεις, όταν πρέπει να γίνει διακοπή της ΚΜΕ από μια περιφερειακή συσκευή που ζητά εξυπηρέτηση.

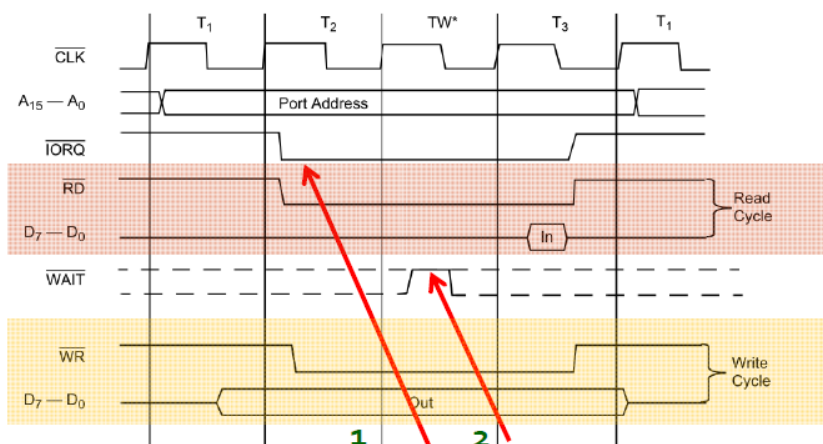
Κύκλος ανάγνωσης/εγγραφής μνήμης

|| Ζ80: Κύκλος Ανάγνωσης & Εγγραφής



Οποιοσδήποτε από τους δύο κύκλους μνήμης (είτε για ανάγνωση, είτε για εγγραφή) απαιτεί την προσπέ-
λαση της μνήμης, δηλ. τον εντοπισμό της διεύθυνσης μνήμης όπου θα γράψουμε (αποθηκεύσουμε) ή θα δια-
βάσουμε. Αυτό γίνεται με το address bus $A_{15} - A_0$. Τόσο για την ανάγνωση όσο και για την εγγραφή, είναι
απαραίτητο να ενεργοποιηθεί το σήμα $MREQ'$, που δείχνει προσπέλαση μνήμης. Η διαφοροποίηση των δύο
περιπτώσεων φαίνεται με τα σήματα RD' και WR' , όπου ανάλογα με την περίπτωση, ενεργοποιείται το ένα
από τα δύο. Στην περίπτωση της **ανάγνωσης μνήμης**, το data bus $D_7 - D_0$ μεταφέρει στην ΚΜΕ (Είσοδος) τα
περιεχόμενα της θ.μ. που διαβάστηκε, ενώ στην περίπτωση της **εγγραφής μνήμης**, το data bus $D_7 - D_0$ μεταφέ-
ρει από την ΚΜΕ (Εξόδος) το περιεχόμενο που θα αποθηκευτεί στη θ.μ. που προσπελάστηκε νωρίτερα. **Από το
data bus είναι εμφανές ότι η λειτουργία εγγραφής στη μνήμη είναι πιο αργή (χρονοβόρα) σε σχέση με τη
λειτουργία της ανάγνωσης.** Γενικότερα, η είσοδος/έξοδος έχουν να κάνουν με την ΚΜΕ.

■ Ανάγνωση/Εγγραφή Εισόδου Εξόδου

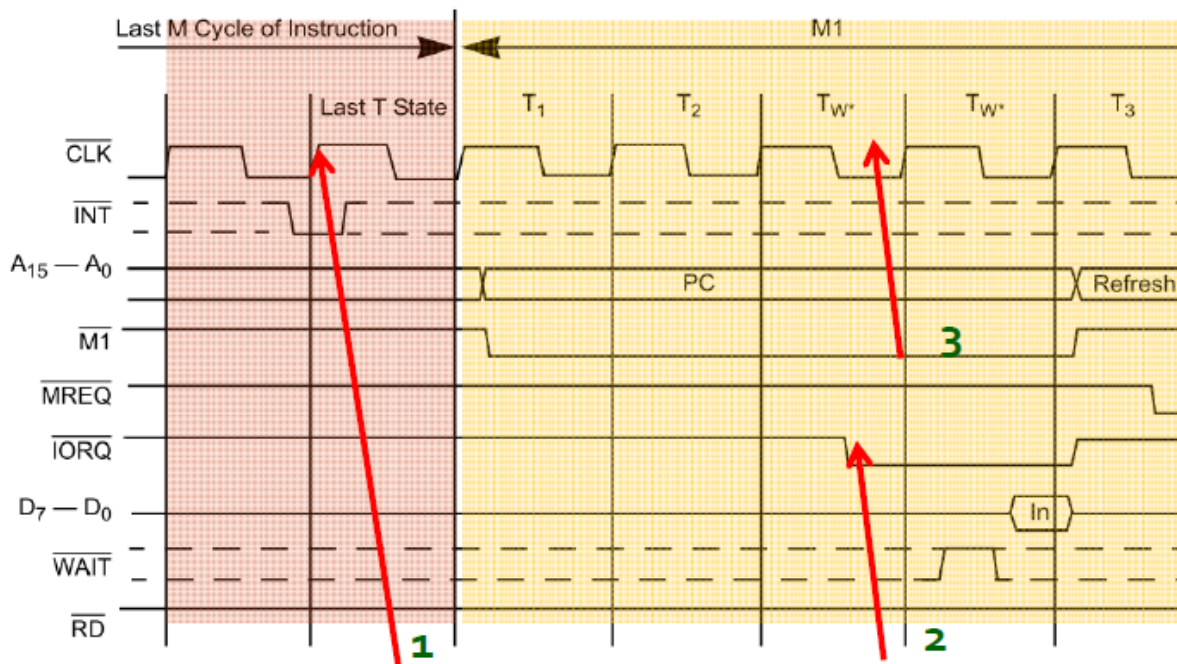


1. Ενεργοποίηση του IORQ~ αντί για το MREQ~
2. Αυτόματη εισαγωγή κύκλου αναμονής (Wait State), για ορθή απόκριση της συσκευής,
3. Για λειτουργίες εγγραφής από περιφερειακές συσκευές, το σήμα WR~ χρησιμοποιείται ως ρολόι στη συσκευή εισόδου/εξόδου.

13

Στην περίπτωση αυτή γίνεται ανάγνωση από περιφερειακή συσκευή και εγγραφή σε περιφερειακή συσκευή. Επειδή στην περίπτωση αυτή προσπελαύνεται περιφερειακή συσκευή και όχι μνήμη, **εισάγονται κύκλοι καθυστέρησης (Wait), για ορθή απόκριση της συσκευής**. Το address bus $A_{15} - A_0$ μεταφέρει στην περίπτωση αυτή τη διεύθυνση της περιφερειακής συσκευής που θα διαβαστεί ή θα γραφτεί. Και οι περιφερειακές συσκευές έχουν μια διεύθυνση μέσω της οποίας εντοπίζονται. Αντί για το σήμα MREQ' χρησιμοποιείται το σήμα IORQ', που δείχνει προσπέλαση σε περιφερειακή συσκευή. Στην περίπτωση της ανάγνωσης από περιφερειακή συσκευή, ενεργοποιείται μαζί με το σήμα το σήμα IORQ' και το σήμα RD' για ανάγνωση, ενώ στην περίπτωση της εγγραφής, ενεργοποιείται μαζί με το σήμα το σήμα IORQ' και το σήμα WR'. Στην περίπτωση της ανάγνωσης, το Data Bus $D_7 - D_0$ μεταφέρει την πληροφορία που διαβάστηκε από την περιφερειακή συσκευή στον επεξεργαστή (In). Στην περίπτωση της εγγραφής, το Data Bus $D_7 - D_0$ μεταφέρει την πληροφορία που θα γραφτεί από τον επεξεργαστή στην περιφερειακή συσκευή (Out). Όπως και στην περίπτωση της μνήμης, έτσι και εδώ η διάρκεια της εγγραφής είναι μεγαλύτερη από αυτή της ανάγνωσης.

Κύκλος διακοπής INT

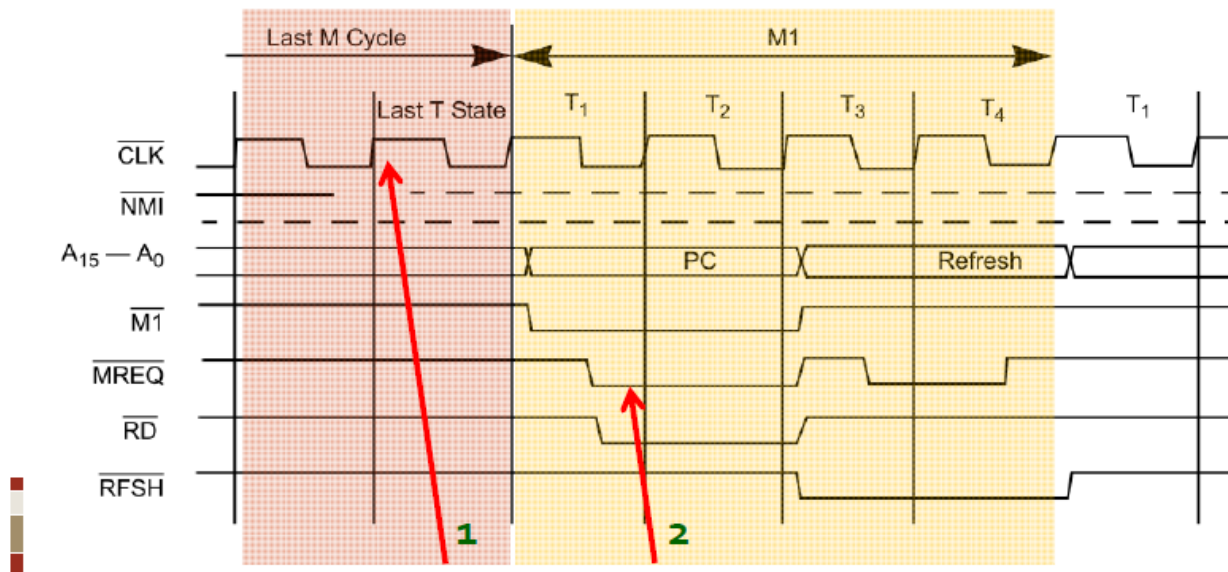


Στον **τελευταίο κύκλο μηχανής (Last M Cycle of Instruction)** και συγκεκριμένα στον τελευταίο **κύκλο ρολογιού (Last T State)** αυτού του κύκλου μηχανής, εξετάζεται το αν έχει ενεργοποιηθεί το σήμα INT'. Σε περίπτωση που διαπιστωθεί ότι έχει ενεργοποιηθεί (δηλ. έχει γίνει «0») ξεκινά ο κύκλος εξυπηρέτησης της διακοπής με όνομα M1. Αυτός περιέχει εκτός από τους κανονικούς κύκλους ρολογιού T₁, T₂ και T₃, και κύκλους αναμονής ή καθυστέρησης (Tw – Waite State). Αυτό γίνεται για να υπάρχει η απαιτούμενη άνεση χρόνου προκειμένου να εξυπηρετηθεί η διακοπή. Για το λόγο αυτό εκτελείται κάθε φορά (ανάλογα με τη διακοπή που προέκυψε) μια διαφορετική ρουτίνα εξυπηρέτησης της διακοπής (ISR – Interrupt Service Routine)¹. Μια ISR είναι ένα έτοιμο πρόγραμμα που βρίσκεται σε προκαθορισμένη θέση στην κύρια μνήμη και εκτελείται προκειμένου να εξυπηρετηθεί η περιφερειακή συσκευή που έστειλε σήμα διακοπής INT'. Για το λόγο αυτό, κατά τη διάρκεια του κύκλου εξυπηρέτησης της διακοπής M1, ο PC που φαίνεται στο Address Bus A₁₅ – A₀, μεταφέρει τη διεύθυνση εντοπισμού της ISR. Επειδή προσπελάσσεται E/E (Είσοδος/Εξόδος), ενεργοποιείται (δηλ. γίνεται «0») το σήμα IORQ' (Input - Output Request) για να δείξει προσπέλαση σε συσκευή Εισόδου – Εξόδου (I/O). Ο σκοπός ενεργοποίησης του σήματος IORQ' αντί για το MREQ' είναι ότι η συσκευή που προκάλεσε τη διακοπή πρέπει να τοποθετήσει τα 8-bit LSB της διεύθυνσης της ρουτίνας εξυπηρέτησης διακοπής (η διεύθυνση της οποίας είναι 16 bit, στο δίαυλο δεδομένων. Το **Data Bus D₇ – D₀** μεταφέρει αυτά τα 8-bit LSB της διεύθυνσης της ISR. Τέλος ενεργοποιείται στον κύκλο εξυπηρέτησης της διακοπής το ειδικό σήμα M1'. Υπενθυμίζεται ότι το σήμα αυτό ενεργοποιείται επίσης και κατά την προσκόμιση εντολής.

¹ Υπάρχουν 127 διαφορετικές ISRs για διακοπές τύπου INT Mode 2.

Κύκλος διακοπής NMI

Αναγνώριση Non Maskable Interrupt



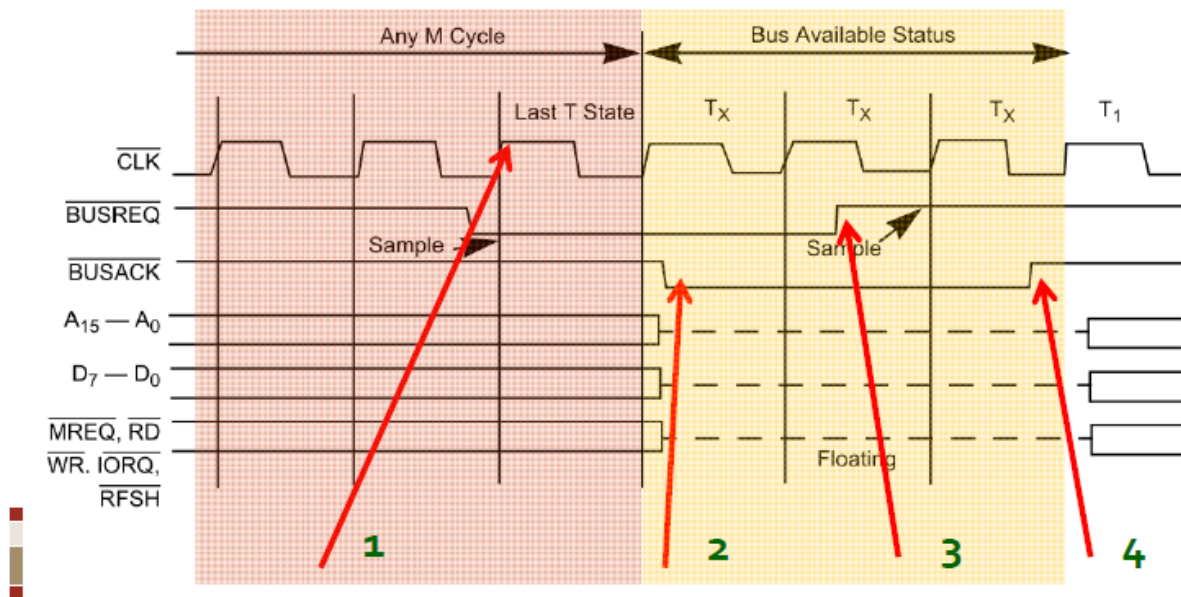
Στον **τελευταίο κύκλο μηχανής (Last M Cycle of Instruction)** και συγκεκριμένα στον τελευταίο κύκλο ρολογιού (**Last T State**) αυτού του κύκλου μηχανής, εξετάζεται το αν έχει ενεργοποιηθεί το σήμα NMI'. Σε περίπτωση που διαπιστωθεί ότι έχει ενεργοποιηθεί (δηλ. έχει γίνει «0») ξεκινά ο κύκλος εξυπηρέτησης της διακοπής με όνομα M1. Αυτός, σε αντίθεση με τον κύκλο εξυπηρέτησης διακοπής INT', περιέχει μόνο κανονικούς κύκλους ρολογιού T₁, T₂ και T₃, και όχι κύκλους αναμονής ή καθυστέρησης (Tw – Waite State), γιατί τώρα **δεν υπάρχει η ανάγκη επιμήκυνσης του χρόνου εξυπηρέτησης της διακοπής**. Για την εξυπηρέτηση μιας διακοπής τύπου NMI, υπάρχει μόνο μια ISR, η οποία μάλιστα βρίσκεται στην προκαθορισμένη διεύθυνση 0066H. Η διακοπή αυτή, όπως υποδηλώνει και το όνομά της (Non Maskable Interrupt) δεν μασκάρεται, δηλ. δεν παρεμποδίζεται και όταν συμβεί, πρέπει οπωσδήποτε να εξυπηρετηθεί. Κατά τη διάρκεια του κύκλου εξυπηρέτησης της διακοπής M1, ο PC που φαίνεται στο Address Bus A₁₅ – A₀, μεταφέρει τη διεύθυνση εντοπισμού της ISR, που όπως προαναφέρθηκε είναι η 0066H. Τώρα, **ενεργοποιείται το σήμα MREQ' και όχι το IORQ'** γιατί αυτή η διακοπή δεν προσπελαύνει E/E, αλλά μνήμη. Επίσης, ενεργοποιείται και το σήμα RD' για να δείξει ανάγνωση της διεύθυνσης της ISR. Τέλος ενεργοποιείται στον κύκλο εξυπηρέτησης της διακοπής το ειδικό σήμα M1'. Υπενθυμίζεται ότι το σήμα αυτό ενεργοποιείται επίσης και κατά την προσκόμιση εντολής.

Ομοιότητες – Διαφορές NMI και INT

- Η διακοπή NMI δεν μασκάρεται (παρεμποδίζεται) - Η διακοπή INT μασκάρεται.
- Η διακοπή NMI εξυπηρετείται από μια μόνο ISR - Η διακοπή INT μπορεί να έχει έως 127 διαφορετικές ISRs
- Η διακοπή NMI ενεργοποιεί το σήμα MREQ' - Η διακοπή INT ενεργοποιεί το σήμα IORQ'.
- Η διακοπή NMI δεν εισάγει wait states κατά τον κύκλο εξυπηρέτησης της διακοπής - Η διακοπή INT εισάγει.
- Στη διακοπή NMI γίνεται ανανέωση των περιεχομένων της Κ.Μ. - Στη διακοπή INT όχι.

Κύκλος διακοπής DMA

Bus Request για DMA

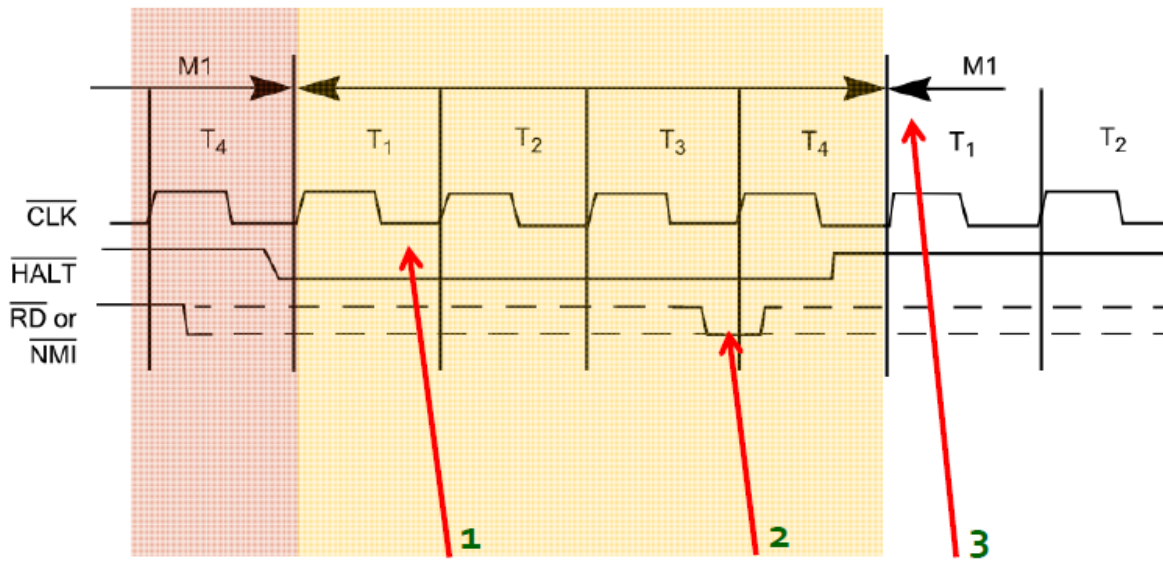


Στον τελευταίο κύκλο ρολογιού (**Last T State**) **οποιοδήποτε κύκλου μηχανής (Any M Cycle)**, εξετάζεται το αν έχει ενεργοποιηθεί το σήμα **BUSREQ'** (Bus Request = απαίτηση παραχώρησης διαύλων). Το σήμα αυτό προέρχεται από κάποιο ελεγκτή ΑΠΜ (Απευθείας Προσπέλασης Μνήμης – DMAC = DMA Controller), ο οποίος ζητά από την ΚΜΕ την παραχώρηση των τριών διαύλων του συστήματος (Address Bus, Data Bus, Control Bus²), προκειμένου να πραγματοποιήσει μεταφορές εντολών/δεδομένων μεταξύ Κύριας Μνήμης και συσκευών Ε/Ε. Σε περίπτωση που διαπιστωθεί ότι το σήμα αυτό έχει ενεργοποιηθεί (δηλ. έχει γίνει «0»), παραχωρούνται οι τρεις δίαυλοι του συστήματος στον ελεγκτή ΑΠΜ. Αυτό διαπιστώνεται από το γεγονός ότι η ΚΜΕ απαντά σε αυτή την απαίτηση, θέτοντας τη γραμμή **BUSACK'** (Bus Acknowledge = αναγνώριση παραχώρησης διαύλου) σε χαμηλό δυναμικό. Αυτό δηλώνει η ένδειξη «2». Από τη στιγμή που ο έλεγχος των διαύλων περνά στον ελεγκτή ΑΠΜ, εμφανίζεται η ένδειξη *floating*, που σημαίνει υψηλή εμπέδηση σημάτων και διαύλων από την ΚΜΕ (δηλ. δεν μπορούν να χρησιμοποιηθούν πλέον οι δίαυλοι και τα σήματα από την ΚΜΕ), διότι χρησιμοποιούνται από τον DMAC για μεταφορά δεδομένων ανάμεσα σε μνήμες και περιφερειακές συσκευές. Η ένδειξη «3», δηλαδή η απενεργοποίηση του σήματος **BUSREQ'**, υποδηλώνει ότι **δεν** είναι πλέον αναγκαία η χρήση των διαύλων από τον DMAC (έχει ολοκληρώσει την εργασία με την οποία ήταν επιφορτισμένος) και για αυτό μετά την παρέλευση κάποιου χρονικού διαστήματος απενεργοποιείται και το σήμα **BUSACK'**. Η ένδειξη «4» δείχνει ότι η ΚΜΕ παίρνει και πάλι τον έλεγχο των διαύλων. Ο κύκλος εξυπηρέτησης της ΑΠΜ ονομάζεται **Bus Available Status** και περιέχει μεταβλητό αριθμό κύκλων ρολογιού, διότι εξαρτάται από το μέγεθος των πληροφοριών που κάθε φορά μεταφέρονται.

² Αποτελείται από τα σήματα **MREQ', IORQ', RD' WR', RFSH'**

Κύκλος διακοπής HALT

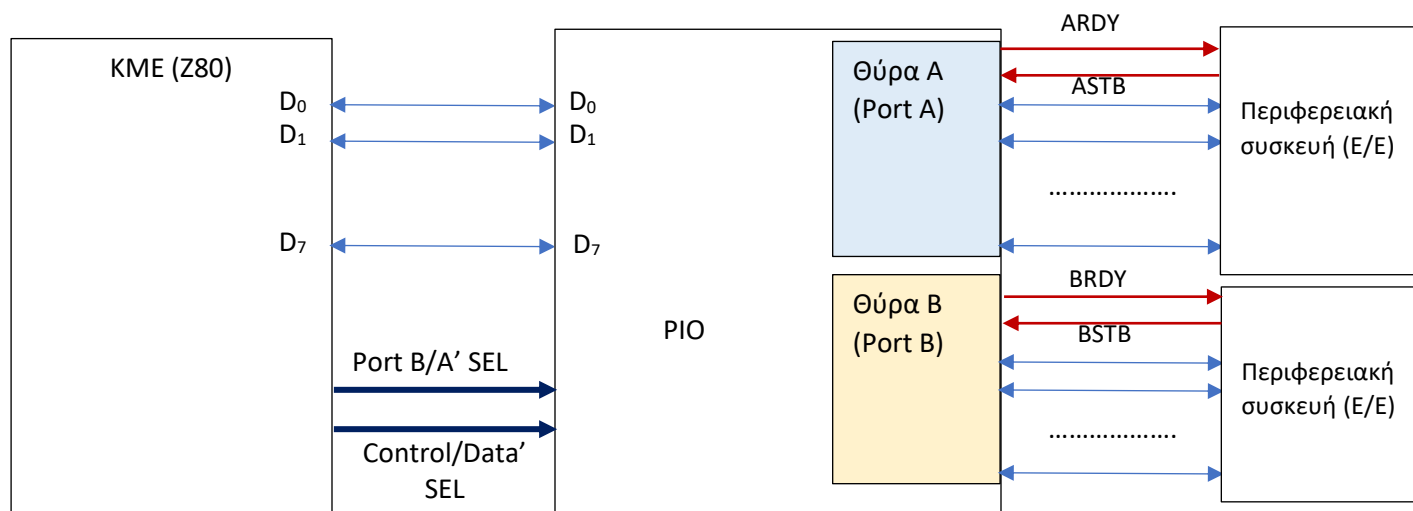
Εκτέλεση της εντολής Halt



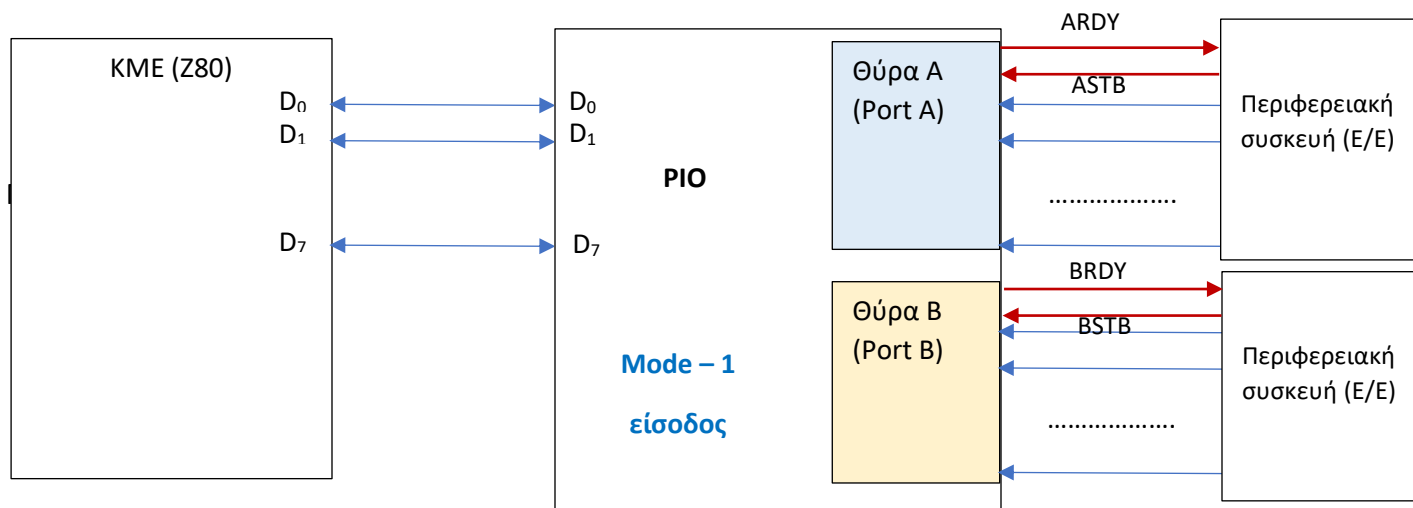
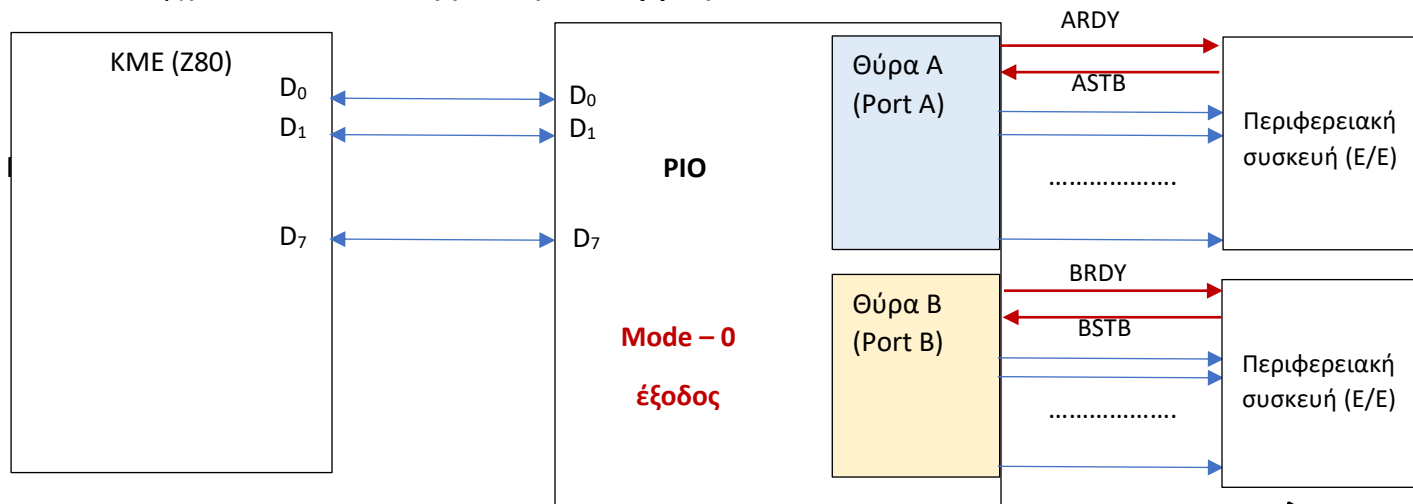
Η κατάσταση Halt είναι μια κατάσταση χαμηλής κατανάλωσης ενέργειας στην οποία εισέρχεται ο Z80, όταν υπολειτουργεί. Το πέρασμα στην κατάσταση αυτή σηματοδοτείται από τη μετάβαση του σήματος $\overline{\text{HALT}}$ σε χαμηλό δυναμικό. Στην κατάσταση αυτή η ΚΜΕ εκτελεί εντολές NOP, για να διατηρεί ενεργό το σήμα Refresh της μνήμης. Υπάρχει συνεχής δειγματοληψία των γραμμών διακοπών ($\overline{\text{INT}}$, $\overline{\text{NMI}}$) κατά την ανοδική ακμή του CLK για να διαπιστωθεί αν προκύψει κάποια διακοπή από εξωτερική πηγή (κύκλος ρολογιού T4). Σε μια τέτοια περίπτωση, ο επόμενος κύκλος (M1) λειτουργεί ως κύκλος εξυπηρέτησης της διακοπής (Interrupt Acknowledge ή αλλιώς ISR), για την εξυπηρέτηση της διακοπής που δέχτηκε η ΚΜΕ. Αυτό φαίνεται με την ένδειξη «3».

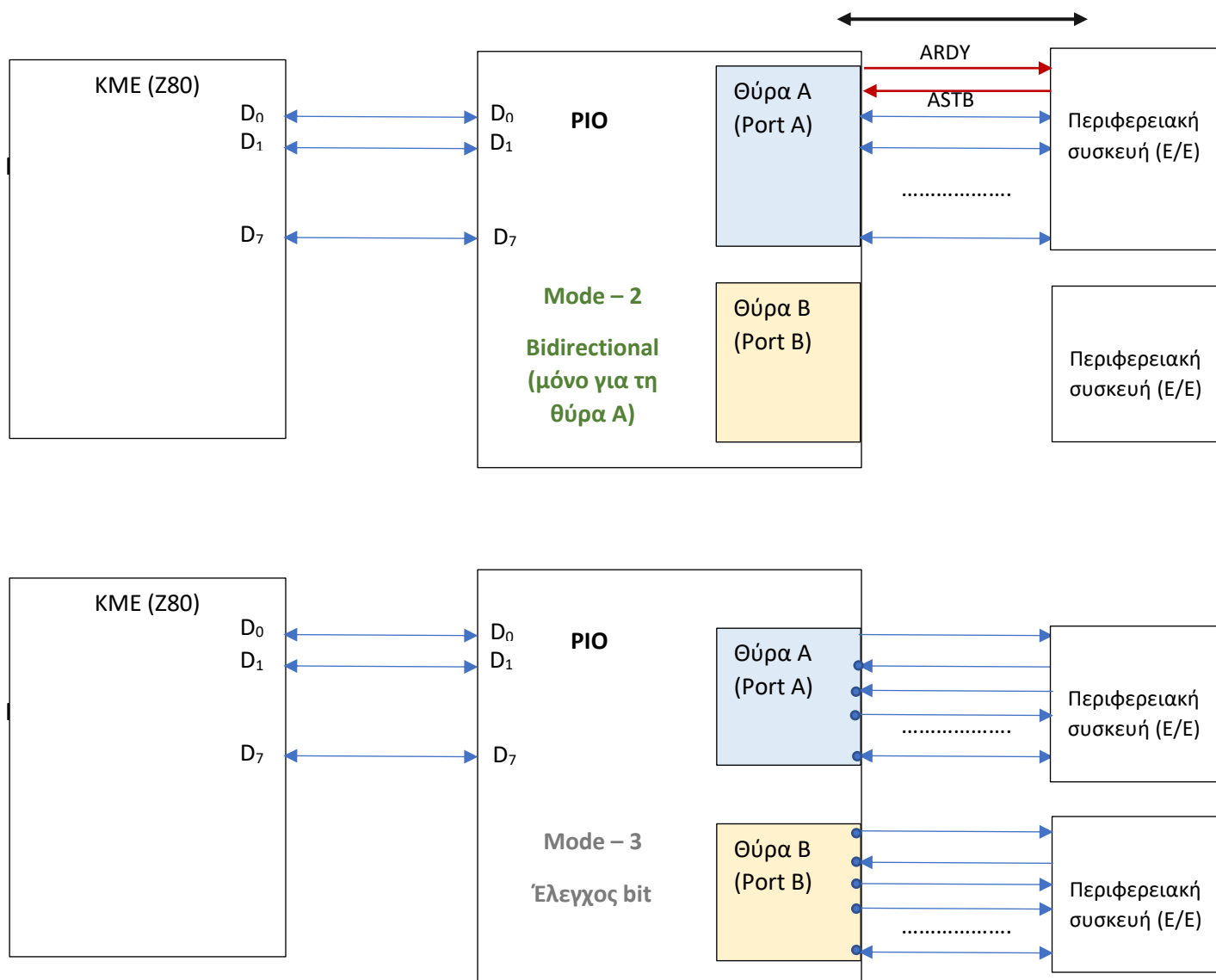
14. PIO (parallel Input – Output)

Mode λειτουργίας PIO



Στο PIO υπάρχουν **4 καταστάσεις (mode) λειτουργίας**:

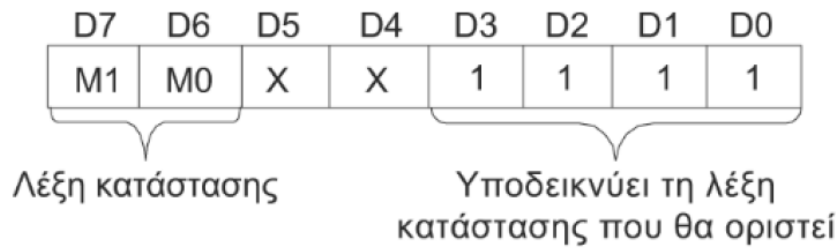




Για να λειτουργήσει σωστά το PIO απαιτούνται τα σήματα χειραψίας READY και STROBE στα mode λειτουργίας από «0» έως και «2». Αυτά τα σήματα χειραψίας αναφέρονται ξεχωριστά σε κάθε θύρα, γι' αυτό και ονομάζονται **ARDY** και **ASTB** και αντίστοιχα **BRDY** και **BSTB**.

Ο πιο συνηθισμένος τρόπος λειτουργίας του PIO είναι το **Mode – 3**, όπου εκεί χρειάζεται ο προγραμματισμός **όλων** των καταχωρητών του. Στους τρεις πρώτους τρόπους λειτουργίας (mode 0 έως mode 2) απαιτούνται από μια έως τρεις λέξεις (καταχωρητές) ελέγχου, ενώ στον τελευταίο τρόπο, απαιτούνται και οι πέντε λέξεις (καταχωρητές) που έχει το PIO. Η κατάσταση αυτή αφορά και τις δύο θύρες. Στο **Mode – 3** τα bit αναγνωρίζονται ξεχωριστά ως bit εισόδου, είτε ως bit εξόδου. **Δεν χρησιμοποιούνται τα σήματα χειραψίας Ready και Strobe σε αυτή την κατάσταση λειτουργίας.** Ακολουθεί ο προγραμματισμός των καταχωρητών του PIO. Αυτοί -όπως αναφέρθηκε- είναι συνολικά πέντε και χρησιμοποιούνται όλοι μόνο στο Mode – 3 (έλεγχος bit). Πάντα, ανεξάρτητα από το Mode λειτουργίας που χρησιμοποιείται, **ως πρώτος καταχωρητής (λέξη ελέγχου) προγραμματίζεται η λέξη ελέγχου κατάστασης.**

Λέξη Ελέγχου Κατάστασης (Mode Control)



- Επιλέγει την κατάσταση της θύρας από τα πεδία M1 και M0.
- Απαιτείται εγγραφή της λέξης, που μπορεί να γίνει οποιαδήποτε στιγμή.
- Η Mode Control Word πρέπει να ακολουθείται από I/O Control Word.

Αυτός ο καταχωρητής χρησιμοποιείται σε όλα τα mode λειτουργίας (mode 0 έως mode 3) γιατί προγραμματίζει το PIO πως θα λειτουργήσει. Αν $M_1M_0=00$, τότε επιλέγεται το mode 0 κ.ο.κ.

Λέξη Ελέγχου Εισόδων/Εξόδων (I/O Register Control Word)

D7	D6	D5	D4	D3	D2	D1	D0
I/O7	I/O6	I/O5	I/O4	I/O3	I/O2	I/O1	I/O0

- Καθορίζει ποιες γραμμές θα είναι έξοδοι και ποιες θα είναι είσοδοι.
- Εγγραφή «0» στο αντίστοιχο bit: έξοδοι.
- Εγγραφή «1» στο αντίστοιχο bit: είσοδοι.

Αυτός ο καταχωρητής χρησιμοποιείται μόνο στο mode 3, γιατί προγραμματίζει το PIO έτσι ώστε άλλοι ακροδέκτες (pins) της επιλεγμένης θύρας (A ή B) να είναι είσοδοι και άλλοι έξοδοι.

Λέξη Διανύσματος Διακοπής

(Interrupt Vector)

D7	D6	D5	D4	D3	D2	D1	D0
V7	V6	V5	V4	V3	V2	V1	0

Υποδεικνύει ότι αυτή η λέξη ελέγχου είναι ένα διάνυσμα διακοπών

- Το PIO είναι σχεδιασμένο να λειτουργεί με τον Z80 στην κατάσταση διακοπής 2 (Interrupt Mode 2).
- Η λέξη αυτή παρέχει τα 8 LSBs της διεύθυνσης της ρουτίνας εξυπηρέτησης διακοπής.
- Όμοια με τα Modes 0, 1, 2.

2

Αυτός ο καταχωρητής χρησιμοποιείται σε όλα τα mode λειτουργίας (mode 0 έως mode 3) γιατί περιέχει τα 8 λιγότερο σημαντικά δυαδικά ψηφία της διεύθυνσης της ρουτίνας εξυπηρέτησης της διακοπής (ISR). Προϋπόθεση για να χρησιμοποιηθεί ο εν' λόγω καταχωρητής είναι να έχει προσδιοριστεί από την εκφώνηση της άσκησης η διεύθυνση αυτή. Για παράδειγμα, αν έχει δοθεί από την εκφώνηση, ότι η διεύθυνση της ISR (Interrupt Service Routine) είναι η F890h, τότε ο καταχωρητής αυτός θα περιέχει το **low byte** της διεύθυνσης της ISR που είναι το 90h, γραμμένο σε δυαδική μορφή:

1	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---

Λέξη Ελέγχου Διακοπής

(Interrupt Control)

D7	D6	D5	D4	D3	D2	D1	D0
Enable Int.	AND/OR	High Low	Masks Follows	0	1	1	1

Χρησιμοποιούνται μόνο στην κατάσταση 3

Υποδεικνύει τη λέξη ελέγχου διακοπής

- Διακοπή λόγω μιας γραμμής που πληροί το επιθυμητό επίπεδο (συνθήκη OR).
- Διακοπή λόγω όλων των γραμμών που πληρούν το επιθυμητό επίπεδο (συνθήκη AND).
- Το D5 ελέγχει εάν το ενεργό επίπεδο μιας γραμμής είναι «0» ή «1».

Αυτός ο καταχωρητής χρησιμοποιείται σε όλα τα mode λειτουργίας (mode 0 έως mode 3), κυρίως όμως στο mode – 3, διότι προγραμματίζει τον τρόπο λειτουργίας των διακοπών. Για παράδειγμα, προσδιορίζει το αν θα συμβούν διακοπές ή όχι (Enable Int.) και το πότε θα συμβούν (AND/OR) και σε ποια τιμή των εισόδων (High/Low). Επίσης, καθορίζει και την ύπαρξη μάσκας (Mask Follows), όπως προσδιορίζεται στη συνέχεια.

Λέξη Ελέγχου Κάλυψης (Mask Control Word)

D7	D6	D5	D4	D3	D2	D1	D0
MB7	MB6	MB5	MB4	MB3	MB2	MB1	MB0

- Αυτή η λέξη καθορίζει ποιες πηγές διακοπών θα επιλεγούν.
- Για να προσπελαστεί η λέξη αυτή θα πρέπει να γίνει 1 το D4 της λέξης ελέγχου διακοπής (Interrupt Control Word).

Αυτός ο καταχωρητής χρησιμοποιείται σε όλα τα mode λειτουργίας (mode 0 έως mode 3) κυρίως όμως στο mode – 3, διότι προγραμματίζει τον τρόπο λειτουργίας των διακοπών, ως επακόλουθος του προηγούμενου καταχωρητή, για να προσδιορίσει ποιοι ακροδέκτες της επιλεγμένης θύρας (A ή B) μπορούν να προκαλέσουν διακοπή (δηλ. ποιοι ακροδέκτες θα αποτελούν τις πηγές διακοπής). Όσα bits του καταχωρητή αυτού έχουν τιμή «0» σημαίνει επίτρεψη, δηλ. σημαίνει ότι μπορούν να προκαλέσουν διακοπή (δηλ. αυτοί δεν είναι μασκαρισμένοι), ενώ όσα bits του καταχωρητή αυτού έχουν τιμή «1» σημαίνει ότι είναι μασκαρισμένοι, δηλ. δεν μπορούν να προκαλέσουν διακοπή.

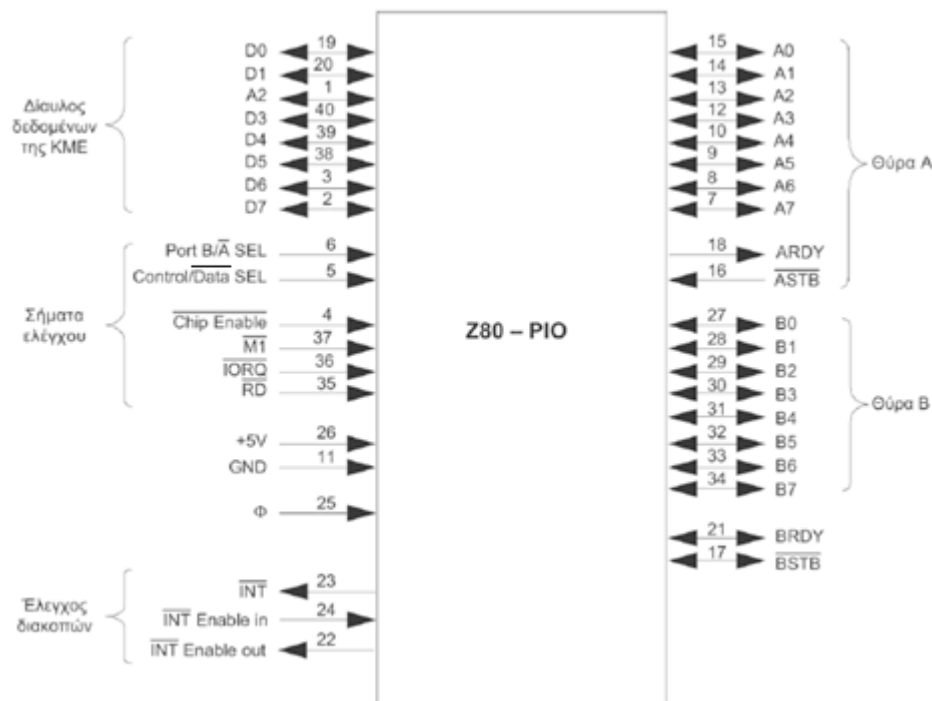
Σημείωση: ο όρος «μασκάρισμα» σημαίνει παρεμπόδιση ενός ακροδέκτη από το να προκαλέσει διακοπή.

Θέματα με το PIO

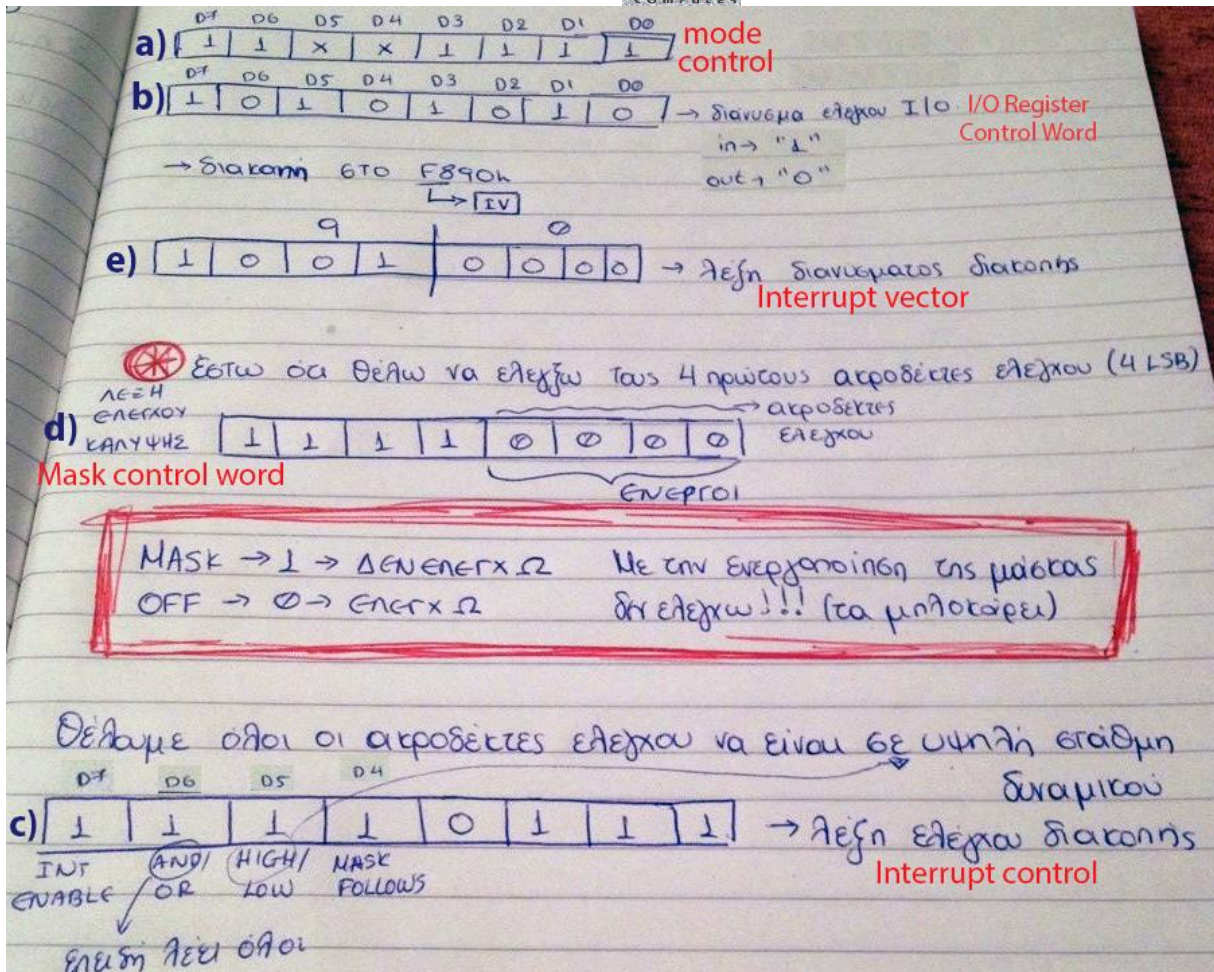
1) Να προγραμματιστεί κατάλληλα το PIO έτσι ώστε να λειτουργεί σε mode – 3, και οι ζυγοί ακροδέκτες της επιλεγμένης θύρας να λειτουργούν ως **έξοδοι**, ενώ οι περιττοί ακροδέκτες ως **είσοδοι**. Επιπλέον, να **επιτρέπονται** διακοπές και να προκαλείται διακοπή όταν **και οι τέσσερις λιγότερο σημαντικοί ακροδέκτες** της επιλεγμένης θύρας είναι **όλοι** σε υψηλό δυναμικό (κατάσταση «1»). Η διεύθυνση της ρουτίνας εξυπηρέτησης διακοπής είναι η F890H.

Λύση

Θα πρέπει **καταρχήν** να επιλεγεί το PIO, θέτοντας τον **ακροδέκτη 4 (~Chip Enable)** σε χαμηλό δυναμικό. Στη συνέχεια θα πρέπει να επιλεγεί η θύρα A ή B με τον ακροδέκτη 6 (**Port B/~A SEL**), και τέλος για τον προγραμματισμό του PIO, θα πρέπει ο **ακροδέκτης 5 (Control/~Data SEL)** να τεθεί σε υψηλό δυναμικό («1»), διότι θέτοντας «0» στον ακροδέκτη αυτό, πραγματοποιούμε μεταφορές δεδομένων μέσω αυτού, ενώ θέτοντας «1» στον ακροδέκτη αυτό, πραγματοποιούμε προγραμματισμό των καταχωρητών του PIO.



Στη συνέχεια ακολουθεί ο προγραμματισμός των καταχωρητών του PIO. Επειδή είμαστε σε κατάσταση λειτουργίας 3 (mode – 3) απαιτείται ο προγραμματισμός **όλων των καταχωρητών** του PIO. Σε κάποιο άλλο mode λειτουργίας θα χρειαζόταν να προγραμματίσουμε από 1 – 3 καταχωρητές (ή αλλιώς λέξεις ελέγχου). Στο mode – 3 και συγκεκριμένα στον προγραμματισμό της λέξης Interrupt control πάντα χρησιμοποιούμε μάσκα, για να **προσδιορίσουμε ποιοι ακροδέκτες επιτρέπεται να προκαλέσουν διακοπή και ποιοι όχι. Όσοι έχουν μάσκα, δεν μπορούν να προκαλέσουν διακοπή.**



Παρατήρηση 1: αν η εκφώνηση έλεγε να προκαλούνται διακοπές όταν **τουλάχιστον ένας από τους τέσσερις λιγότερο σημαντικούς ακροδέκτες** της επιλεγμένης θύρας είναι σε υψηλό δυναμικό (κατάσταση «1»), τότε στο bit D₆ του καταχωρητή Interrupt Control θα βάζαμε τιμή «0».

Παρατήρηση 2: αν η εκφώνηση έλεγε να προκαλούνται διακοπές όταν **τουλάχιστον ένας (ή και όλοι) από τους τέσσερις λιγότερο σημαντικούς ακροδέκτες** της επιλεγμένης θύρας **είναι σε χαμηλό δυναμικό (κατάσταση «0»)**, τότε στο bit D₅ του καταχωρητή Interrupt Control θα βάζαμε τιμή «0».

2) Για το PIO του Z80 να τεθεί η θύρα B σε κατάσταση λειτουργίας (mode) 0 και να **επιτρέψουμε διακοπές** που να εξυπηρετούνται από τη διεύθυνση F890h. Να εξηγήσετε τις ενέργειες που απαιτούνται για αυτό.

Λύση

a) Καταρχήν θα πρέπει να προγραμματίσουμε τη θύρα B του PIO και για το λόγο αυτό θα πρέπει πρώτα να την επιλέξουμε. Για να γίνει αυτό, θα πρέπει **καταρχήν** να επιλεγεί το PIO, θέτοντας τον **ακροδέκτη 4 (~Chip Enable)** σε χαμηλό δυναμικό.

b) Στη συνέχεια ο **ακροδέκτης 6 (Port B/A~ SEL)** θα πρέπει να τεθεί σε υψηλό δυναμικό («1») προκειμένου να επιλεγεί η θύρα B.



c) Επίσης, για τον προγραμματισμό του PIO, θα πρέπει ο **ακροδέκτης 5 (Control/Data~ SEL)** να τεθεί σε υψηλό δυναμικό («1»), διότι θέτοντας «0» στον ακροδέκτη αυτό, πραγματοποιούμε μεταφορές δεδομένων μέσω αυτού, ενώ θέτοντας «1» στον ακροδέκτη αυτό, πραγματοποιούμε προγραμματισμό των καταχωρητών του PIO.

d) Προγραμματίζουμε το **mode 0** μέσω του καταχωρητή **Mode Control (Λέξη Ελέγχου Κατάστασης)**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

0	0	X	X	1	1	1	1
---	---	---	---	---	---	---	---

e) Προγραμματίζουμε το **low byte** της διεύθυνσης **F890h** μέσω του καταχωρητή **Interrupt Vector (Λέξη δια-νύσματος διακοπής)**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

1	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---

f) Προγραμματίζουμε την ενεργοποίηση (επίτρεψη) διακοπών μέσω του καταχωρητή **Interrupt Control** (λέξη ελέγχου διακοπής). Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

1	X	X	X	0	1	1	1
---	---	---	---	---	---	---	---

Επειδή είμαστε σε mode – 0, τα bits D6, D5, D4 είναι αδιάφοροι όροι, αφού δεν χρησιμοποιούνται. Αυτό συμβαίνει επίσης και για τα mode 1 και 2.

3) Για το PIO του Z80 να τεθεί η **θύρα A σε κατάσταση λειτουργίας (mode) 1**. Να εξηγήσετε τις ενέργειες που απαιτούνται για αυτό.

Λύση

a) Καταρχήν θα πρέπει να προγραμματίσουμε τη θύρα A του PIO και για το λόγο αυτό θα πρέπει πρώτα να την επιλέξουμε. Για να γίνει αυτό, θα πρέπει **πρώτα** να επιλεγεί το PIO, θέτοντας τον **ακροδέκτη 4 (Chip Enable~)** σε χαμηλό δυναμικό.

b) Στη συνέχεια ο **ακροδέκτης 6 (Port B/A~ SEL)** θα πρέπει να τεθεί σε χαμηλό δυναμικό («0») προκειμένου να επιλεγεί η θύρα A.

c) Επίσης, για τον προγραμματισμό του PIO, θα πρέπει ο **ακροδέκτης 5 (Control/Data~ SEL)** να τεθεί σε υψηλό δυναμικό («1»), διότι θέτοντας «0» στον ακροδέκτη αυτό, πραγματοποιούμε μεταφορές δεδομένων μέσω αυτού.

d) Προγραμματίζουμε το mode 1 μέσω του καταχωρητή **Mode Control (Λέξη Ελέγχου Κατάστασης)**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

0	1	X	X	1	1	1	1
---	---	---	---	---	---	---	---

e) Επειδή **δεν** αναφέρεται κάτι σχετικό με διακοπές προγραμματίζουμε τον καταχωρητή **Interrupt Control** (Λέξη Ελέγχου διακοπής) ως εξής:

X	X	X	X	0	1	1	1
---	---	---	---	---	---	---	---



4) Έστω ότι ο καταχωρητής κάλυψης (μάσκας) ή λέξη ελέγχου κάλυψης έχει το εξής περιεχόμενο:

				D ₃		D ₁	D ₀
1	1	1	1	0	1	0	0

Θέλουμε να προκαλείται διακοπή όταν **όλες οι εισόδους είναι υψηλές**. Με δεδομένο ότι η λέξη ελέγχου εισόδων/εξόδων έχει το περιεχόμενο:

				D ₃		D ₁	D ₀
0	0	0	1	1	1	0	0

μπορεί να προκαλέσει διακοπή;

Λύση

Η συγκεκριμένη λέξη εισόδου/εξόδου, **δεν** μπορεί να προκαλέσει διακοπή, επειδή **δεν** είναι όλα τα bits σε υψηλό δυναμικό. Θα έπρεπε και τα τρία bits D₀, D₁, D₃ να ήταν «1». Θα μπορούσε να προκληθεί διακοπή, αν μια τουλάχιστον είσοδος ήταν «1» (αν το έλεγε η εκφώνηση).

Θέμα Ιανουαρίου 2020

Να προγραμματιστεί κατάλληλα το PIO, έτσι ώστε η θύρα B να λειτουργεί σε κατάσταση λειτουργίας 3 (mode – 3), όπου οι περιττού αριθμού ακροδέκτες (2λ + 1) θα είναι **έξοδοι** και οι άρτιου αριθμού ακροδέκτες θα είναι **είσοδοι**. Θα **επιτρέπονται** διακοπές στη διεύθυνση CAFE, όταν **δύο εισόδοι (n+1, n+3) θα βρίσκονται σε χαμηλή στάθμη δυναμικού**. α) Να γράψετε όλες τις απαιτούμενες λέξεις διανυσμάτων β) Να εξηγήσετε αναλυτικά την τιμή του κάθε ψηφίου για κάθε λέξη.

Λύση

Θα πρέπει **καταρχήν** να επιλεγεί το PIO, θέτοντας τον **ακροδέκτη 4 (~Chip Enable)** σε χαμηλό δυναμικό. Στη συνέχεια ο **ακροδέκτης 6 (Port B/~A SEL)** θα πρέπει να τεθεί σε υψηλό δυναμικό («1») προκειμένου να επιλεγεί η θύρα B. Επίσης, για τον προγραμματισμό του PIO, θα πρέπει ο **ακροδέκτης 5 (Control/~Data SEL)** να τεθεί σε υψηλό δυναμικό («1»), για προγραμματισμό των καταχωρητών του PIO. Ξεκινάμε με τον προγραμματισμό του **mode 3** μέσω του καταχωρητή **Mode Control (Λέξη Ελέγχου Κατάστασης)**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

1	1	X	X	1	1	1	1
---	---	---	---	---	---	---	---

Προγραμματίζουμε μόνο τα δύο αριστερότερα bits $M_1M_0 = 11 \rightarrow \text{mode} - 3$.

Στη συνέχεια προγραμματίζουμε ποιοι ακροδέκτες θα είναι εισόδοι και ποιοι θα είναι έξοδοι, μέσω της λέξης **Ελέγχου Εισόδων/Εξόδων (I/O Register Control Word)**, όπου εγγραφή «0» στο αντίστοιχο bit συμβολίζει έξοδο και εγγραφή «1» συμβολίζει είσοδο. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Έπειτα, προγραμματίζουμε το **low byte** της διεύθυνσης **CAFE** μέσω του καταχωρητή **Interrupt Vector (Λέξη διανύσματος διακοπής)**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι το byte FE:

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---



Στη συνέχεια προγραμματίζουμε την ενεργοποίηση (επίτρεψη) διακοπών μέσω της λέξης (καταχωρητή) **Interrupt Control (Λέξη Ελέγχου διακοπής)**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

1	1	0	1	0	1	1	1
---	---	---	---	---	---	---	---

Το αριστερότερο bit (D₇) του συγκεκριμένου καταχωρητή που είναι «1», δηλώνει ότι επιτρέπονται διακοπές, το επόμενο (D₆) που είναι «1», δηλώνει ότι οι διακοπές θα συμβούν, όταν δύο ή περισσότεροι ακροδέκτες **ταυτόχρονα** τίθενται σε κάποια στάθμη δυναμικού (υψηλή ή χαμηλή), το επόμενο (D₅) που είναι «0», δηλώνει ότι οι διακοπές θα συμβούν όταν οι δύο προηγούμενοι ακροδέκτες τεθούν **σε** χαμηλή στάθμη δυναμικού κα τέλος το bit D₄ που είναι ενεργό, δηλώνει ότι ακολουθεί μάσκα. Τα τέσσερα τελευταίας bits του εν' λόγω καταχωρητή είναι πάντα σταθερά, έχοντας τις τιμές 0111.

Στη συνέχεια προγραμματίζουμε μέσω της **λέξης Ελέγχου Κάλυψης (Mask Control Word)** ποιοι ακροδέκτες θα είναι μασκαρισμένοι (δηλ. θα παρεμποδιστούν προκειμένου να μην προκαλέσουν διακοπή) και ποιοι όχι. Επειδή από την εκφώνηση αναφέρεται ότι η διακοπή προκαλείται από τους ακροδέκτες n+1 και n+3, θα πρέπει αυτοί οι δύο **να μην μασκαριστούν**, προκειμένου να μπορέσουν να προκαλέσουν διακοπή. Για να γίνει αυτό πρέπει να τους βάλουμε τιμή «0». **Στο n θα θέσουμε την τιμή «3»**, προκειμένου να πάρουμε τους ακροδέκτες $n+1 = 3 + 1 = 4$ και $n+3 = 3 + 3 = 6$ που είναι άρτιοι αριθμοί, αφού από την εκφώνηση προκύπτει ότι οι άρτιου αριθμού ακροδέκτες θα είναι **είσοδοι**. Το περιεχόμενο του συγκεκριμένου καταχωρητή είναι:

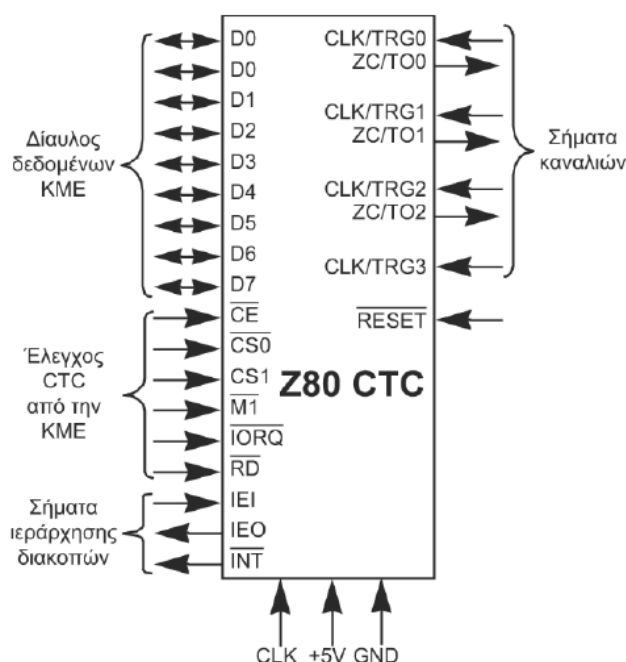
1	0	1	0	1	1	1	1
---	---	---	---	---	---	---	---

Εναλλακτικά στο **n μπορούμε να θέσουμε τιμή 1**, για να πάρουμε τους ακροδέκτες $n+1 = 1 + 1 = 2$ και $n+3 = 1 + 3 = 4$ που είναι και αυτοί άρτιοι αριθμοί.

1	1	0	1	0	1	1	1
---	---	---	---	---	---	---	---

15. CTC (Clock Timer Circuit)

Αποτελεί τη δεύτερη περιφερειακή συσκευή του Z80 και περιέχει 4 κανάλια εισόδου (CLK/TRG0 – CLK/TRG3) για λειτουργία μέτρησης (Counter) και 3 κανάλια εξόδου (ZC/TO0 – ZC/TO2) για λειτουργία timer. Κάθε κανάλι έχει μία μοναδική φυσική διεύθυνση μέσω της οποίας εντοπίζεται. Η διεύθυνση αυτή πρέπει να τελειώνει σε «0» για το κανάλι «0», σε «1» για το κανάλι «1» κ.ο.κ. Για την επιλογή του καναλιού που θα χρησιμοποιηθεί κάθε φορά, χρησιμοποιούνται οι ακροδέκτες **CS1** και **CS0**. Επιπλέον, το CTC περιέχει ένα 8 – bit data bus για την επικοινωνία του με την ΚΜΕ του Z80. Κάθε φορά που το CTC μετρήσει μια ακολουθία παλμών στην είσοδό του, στέλνει σήμα διακοπής **~INT** στην ΚΜΕ για να τη διακόψει. Επίσης, ο ακροδέκτης **~CE** χρησιμοποιείται για την επιλογή του CTC. Επίσης, το **Clock** αποτελεί το σήμα ρολογιού για συγχρονισμό των μετρητών και το σήμα **~RESET** χρησιμοποιείται για την αρχικοποίηση αυτού του περιφερειακού. Το σήμα **~IORQ** δηλώνει την επικοινωνία του CTC με περιφερειακή συσκευή. Για παράδειγμα, όταν το CTC θέλει να μετρήσει παλμούς στα κανάλια εισόδου του, που προέρχονται από μια περιφερειακή συσκευή, είναι απαραίτητη η εκ' των προτέρων επικοινωνία της περιφερειακής συσκευής με το CTC, μέσω του σήματος **~IORQ**.



Σε αντίθεση με το PIO που απαιτούσε να προγραμματιστούν έως και 5 καταχωρητές (Mode – 3), για το προγραμματισμό του CTC απαιτείται **μόνο ένας καταχωρητής (control register)**, τα περιεχόμενα του οποίου φαίνονται στη συνέχεια. Για να χρησιμοποιηθεί ο CTC ως μετρητής προς τα κάτω (down counter), εισάγουμε μια **χρονική σταθερά (time constant)** στον αντίστοιχο καταχωρητή και αυτή πρέπει να είναι μικρότερη ή ίση του FF. Για τη λειτουργία του CTC ως timer, κάτι τέτοιο δεν είναι απαραίτητο.

- ο Καταχωρητής Ελέγχου Καναλιού (Channel Control Register) πρέπει να προγραμματιστεί κατάλληλα. Τα πεδία του καταχωρητή αυτού είναι τα εξής:
- D0 (λιγότερο σημαντικό): Πρέπει να τεθεί 1 για να ερμηνευθεί η εγγραφή ως προσπέλαση στον καταχωρητή αυτό.
 - D1: Επιτρέπει (=1) ή απαγορεύει (=0) τη συνέχιση της μέτρησης από το συγκεκριμένο κανάλι.
 - D2: Αν το bit αυτό είναι 1 δηλώνει ότι η επόμενη εγγραφή θα είναι η χρονική σταθερά (time constant).
 - D3: Χρησιμοποιείται μόνο σε λειτουργία Χρονιστή (Timer mode). X
 - D4: Δηλώνει σε ποια ακμή της CLK/TRG γραμμής θα αρχίσει η μέτρηση.
 - D5: Χρησιμοποιείται μόνο σε λειτουργία Χρονιστή (Timer mode). X
 - D6: Αν είναι 1 το κανάλι θα λειτουργήσει σαν μετρητής αλλιώς ως χρονιστής.
 - D7: Αν είναι 0 απενεργοποιούνται οι διακοπές αλλιώς (με 1) ενεργοποιούνται.

Θέμα 1 με CTC

Ας υποθέσουμε ότι το κανάλι 1 θα χρησιμοποιηθεί για να μετράει «λάθη» υπό μορφή παλμών στη γραμμή CLK/TRG1. Δεν θα χρησιμοποιηθούν διακοπές και το περιφερειακό θα λειτουργήσει σε **polled mode**: δηλαδή ο Z80 θα διαβάσει την τιμή του μετρητή του καναλιού 1 και όταν το πλήθος των παλμών που έχει μετρηθεί είναι μικρότερο π.χ., του 46 θα συνεχίζει να ελέγχει το μετρητή μέσα από ένα βρόχο αλλιώς αν το πλήθος παλμών ξεπεράσει την τιμή 46, ο βρόχος θα τερματιστεί.

Λύση

Θα πρέπει να προγραμματίσουμε κατάλληλα το CTC, εγγράφοντας μια τιμή στον καταχωρητή ελέγχου του (control register). Πριν από αυτό, θα πρέπει να επιλέξουμε το ολοκληρωμένο (CTC) με το σήμα $\sim CE$ και στη συνέχεια να επιλέξουμε το κανάλι 1 (CLK/TRG1). Για να γίνει αυτό, θα πρέπει να χρησιμοποιήσουμε τους ακροδέκτες CS_1 και CS_0 , στους οποίους θα αναθέσουμε το ζεύγος τιμών 01, δηλ. $CS_1CS_0 = 01$. Θα κάνουμε την παραδοχή ότι το κανάλι «1» που έχει επιλεγεί, **αναγνωρίζεται στη διεύθυνση 41H** (κάθε διεύθυνση καναλιού επιλέγεται αυθαίρετα, όμως θα πρέπει να τελειώνει στον κατάλληλο δεκαεξαδικό αριθμό, π.χ. 0 $\rightarrow$ κανάλι 0, 1 $\rightarrow$ κανάλι 1 κ.ο.κ.). Για τον προγραμματισμό του CTC, σύμφωνα με την εκφώνηση, πρέπει να δώσουμε στον καταχωρητή ελέγχου του, το εξής περιεχόμενο:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	1	0	X	0	1	1	1

Στο bit D₇ θέτουμε «0» για να απενεργοποιήσουμε τις διακοπές. Στο bit D₆ θέτουμε «1» για να λειτουργήσει το CTC ως μετρητής (counter). Στα bits D₅ και D₃ θέτουμε «0» διότι έχουμε λειτουργία μετρητή. Στη λειτουργία timer θα βάζαμε «1» στα συγκεκριμένα bits. Στο bit D₄ θέτουμε «X» διότι δεν διευκρινίζει σε ποια ακμή (ανερχόμενη/κατερχόμενη) της CLK/TRG1 γραμμής θα αρχίσει η μέτρηση. Εναλλακτικά, θα μπορούσαμε να κάνουμε την υπόθεση ότι έχουμε ανερχόμενη/κατερχόμενη ακμή, και να θέσουμε στο bit αυτό τιμή 1/0 αντίστοιχα. Ως



χρονική σταθερά (D_2) λαμβάνουμε το **όριο (πλήθος) των παλμών** που μετράμε στην είσοδο. Στο bit D_1 θέτουμε «1» για να επιτρέπεται η συνέχιση της μέτρησης από το κανάλι αυτό. Τέλος στο bit D_0 θέτουμε πάντα «1». Με βάση αυτό το περιεχόμενο, ακολουθεί ο προγραμματισμός του CTC. Η εντολή **IN** εισάγει στον A (Accumulator) περιεχόμενο από περιφερειακή συσκευή και η εντολή **OUT** εξάγει από τον A (Accumulator) περιεχόμενο προς περιφερειακή συσκευή.

LD A, 010X0111b ; $A \leftarrow 010X0111b$ (προγραμματισμός του Control Register του CTC μέσω του Accumulator)
; δηλ. δεν κάνουμε κάποια θεώρηση για την ακμή του ρολογιού.

ή

LD A, 57h ; $A \leftarrow 01010111b$ (αν θεωρήσουμε ότι έχουμε rising edge, δηλ. μέτρηση στη θετική ακμή του ρολογιού)

OUT (41 h), A ; $(41H) \leftarrow A$, πρώτα στέλνουμε στο κανάλι 1 (CLK/TRG1) τη λέξη ελέγχου για τον προγραμματισμό του CTC

LD A, 2Eh ; $A \leftarrow 2Eh = 46_{10} = 00101110$ (πλήθος παλμών που μετρούνται στην είσοδο). Για να χρησιμοποιηθεί ο CTC ως μετρητής προς τα κάτω (down counter), εισάγουμε μια χρονική σταθερά (time constant) ; στον αντίστοιχο καταχωρητή και αυτή πρέπει να είναι μικρότερη ή ίση του FF

OUT (41 h), A ; $(41H) \leftarrow A$, μετά στέλνουμε στο κανάλι 1 (CLK/TRG1) το όριο παλμών, με τη μορφή χρονικής σταθεράς

LOOP:

IN A, (41 h) ; $A \leftarrow (41 h)$, δηλ. εισάγουμε στον A (Accumulator) τον παλμό που μετράμε κάθε φορά.
; Κάθε φορά που μετράται ένας παλμός, η τιμή του A μειώνεται αυτόματα κατά 1.

CP 00h ; Σύγκριση του περιεχομένου του A (46_{10}) με την τιμή 0, δηλ. εκτέλεση της πράξης $A - 00$. Αυτό
; γίνεται για να ελέγχουμε αν έχουν ολοκληρωθεί οι παλμοί που μετράμε στην είσοδο.

JR NZ, LOOP ; για όσο διάστημα ο A **δεν** έχει μηδενιστεί (NZ = No Zero, Z = 0), επιστροφή στην ετικέτα LOOP
; για συνέχιση της μέτρησης της ακολουθίας παλμών στο κανάλι 1. Όταν ο A μηδενιστεί (Z = 1),
; τότε ολοκληρώνεται ο βρόχος.

HLT

Θέμα 2 με CTC

3. Στο κύκλωμα CTC – Clock Timer Counter, Z80, το κανάλι $i=3$ χρησιμοποιείται για να μετράει λάθη υπό τη μορφή παλμών στη γραμμή CLK/TRG(i), χωρίς να χρησιμοποιούνται διακοπές (polled mode). Το όριο του πλήθους των σφαλμάτων είναι 26 (Decimal). Να γραφεί και να εξηγηθεί ο κώδικας υλοποίησης της λειτουργίας. (2.00 Μονάδες)

Λύση



Θα πρέπει να προγραμματίσουμε κατάλληλα το CTC, εγγράφοντας μια τιμή στον καταχωρητή ελέγχου του (control register). Πριν από αυτό, θα πρέπει να επιλέξουμε το ολοκληρωμένο (CTC) με το σήμα $\sim CE$ και στη συνέχεια να επιλέξουμε το κανάλι 3 (CLK/TRG3). Για να γίνει αυτό, θα πρέπει να χρησιμοποιήσουμε τους ακροδέκτες CS_1 και CS_0 , στους οποίους θα αναθέσουμε το ζεύγος τιμών 11, δηλ. $CS_1CS_0 = 11$. Θα κάνουμε την παραδοχή ότι το κανάλι «3» που έχει επιλεγεί, **αναγνωρίζεται για παράδειγμα στη διεύθυνση 43H** (κάθε διεύθυνση καναλιού επιλέγεται αυθαίρετα, όμως θα πρέπει να τελειώνει στον κατάλληλο δεκαεξαδικό αριθμό, π.χ. 0 $\rightarrow$ κανάλι 0, 1 $\rightarrow$ κανάλι 1 κ.ο.κ.). Για τον προγραμματισμό του CTC, σύμφωνα με την εκφώνηση, πρέπει να δώσουμε στον καταχωρητή ελέγχου του, το εξής περιεχόμενο:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	1	0	X	0	1	1	1

Ως χρονική σταθερά (D₂) λαμβάνουμε το **όριο (πλήθος) των παλμών** που μετράμε στην είσοδο. Με βάση αυτό το περιεχόμενο, ακολουθεί ο προγραμματισμός του CTC:

LD A, 010X0111b ; A $\leftarrow$ 010X0111b (προγραμματισμός του Control Register του CTC μέσω του Accumulator)

ή

LD A, 57h ; A $\leftarrow$ 01010111b (αν θεωρήσουμε ότι έχουμε rising edge, δηλ. θετική πυροδότηση παλμού)

OUT (43 h), A ; (43H) $\leftarrow$ A, πρώτα στέλνουμε στο κανάλι 3 (CLK/TRG3) τη λέξη ελέγχου για τον προγραμματισμό του CTC

LD A, 1Ah ; A $\leftarrow$ 1Ah = 26₁₀ = 00011010 (πλήθος παλμών που μετρούνται στην είσοδο). Για να χρησιμοποιηθεί ο CTC ως μετρητής προς τα κάτω (down counter), εισάγουμε μια χρονική σταθερά (time constant) ; στον αντίστοιχο καταχωρητή και αυτή πρέπει να είναι μικρότερη ή ίση του FF

OUT (43 h), A ; (43H) $\leftarrow$ A, μετά στέλνουμε στο κανάλι 3 (CLK/TRG3) το όριο παλμών, με τη μορφή χρονικής σταθεράς

LOOP:

IN A, (43 h) ; A $\leftarrow$ (43 h), δηλ. εισάγουμε στον A (Accumulator) τον παλμό που μετράμε κάθε φορά.
; Κάθε φορά που εισάγεται ένας παλμός, η τιμή του A μειώνεται αυτόματα κατά 1.

CP 00h ; Σύγκριση του περιεχομένου του A (26₁₀) με την τιμή 0, δηλ. εκτέλεση της πράξης A – 00. Αυτό γίνεται για να ελέγχουμε αν έχουν ολοκληρωθεί οι παλμοί που μετράμε στην είσοδο.

JR NZ, LOOP ; για όσο διάστημα ο A **δεν** έχει μηδενιστεί (NZ = No Zero), επιστροφή στην ετικέτα LOOP
; για συνέχιση της μέτρησης της ακολουθίας παλμών στο κανάλι 3. Όταν ο A μηδενιστεί (Z = 1), τότε ολοκληρώνεται ο βρόχος.

HLT

Θέμα 3 με CTC

4. Να αναπτυχθεί ρουτίνα που να μετράει αντίστροφα στο κανάλι 2, υπό μορφή παλμών, στη γραμμή CLK/TRG2, κατ' επιλογή με ή χωρίς διακοπές, για πλήθος έως και την τιμή 5 (20%).
✓ Το ερώτημα να απαντηθεί μόνο στην 5^η σελίδα του φυλλαδίου απαντήσεων.

Λύση

Θα πρέπει να προγραμματίσουμε κατάλληλα το CTC, εγγράφοντας μια τιμή στον καταχωρητή ελέγχου του (control register). Πριν από αυτό, θα πρέπει να επιλέξουμε το ολοκληρωμένο (CTC) με το σήμα $\sim CE$ και στη συνέχεια να επιλέξουμε το κανάλι 2 (CLK/TRG1). Για να γίνει αυτό, θα πρέπει να χρησιμοποιήσουμε τους ακροδέκτες CS_1 και CS_0 , στους οποίους θα αναθέσουμε το ζεύγος τιμών 10, δηλ. $CS_1CS_0 = 10$. Θα κάνουμε την παραδοχή ότι το κανάλι «2» που έχει επιλεγεί, **αναγνωρίζεται στη διεύθυνση 42H** (κάθε διεύθυνση καναλιού επιλέγεται αυθαίρετα, όμως θα πρέπει να τελειώνει στον κατάλληλο δεκαεξαδικό αριθμό, π.χ. 0 $\rightarrow$ κανάλι 0, 1 $\rightarrow$ κανάλι 1 κ.ο.κ.). Θα θεωρήσουμε ότι έχουμε διακοπές. Για τον προγραμματισμό του CTC, σύμφωνα με την εκφώνηση, πρέπει να δώσουμε στον καταχωρητή ελέγχου του, το εξής περιεχόμενο:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	1	0	X	0	1	1	1

Στο bit D₇ θέτουμε «1» για να ενεργοποιήσουμε τις διακοπές, αν υποθέσουμε ότι θα χρησιμοποιήσουμε σήματα διακοπής. Στο bit D₆ θέτουμε «1» για να λειτουργήσει το CTC ως μετρητής (counter). Στα bits D₅ και D₃ θέτουμε «0» διότι έχουμε λειτουργία μετρητή. Στο bit D₄ θέτουμε «X» διότι δεν διευκρινίζει σε ποια ακμή (ανερχόμενη/κατερχόμενη) της CLK/TRG1 γραμμής θα αρχίσει η μέτρηση. Εναλλακτικά, θα μπορούσαμε να κάνουμε την υπόθεση ότι έχουμε ανερχόμενη/κατερχόμενη ακμή, και να θέσουμε στο bit αυτό τιμή 1/0 αντίστοιχα. Ως χρονική σταθερά (D₂) λαμβάνουμε το **όριο (πλήθος) των παλμών** που μετράμε στην είσοδο. Στο bit D₁ θέτουμε «1» για να επιτρέπεται η συνέχιση της μέτρησης από το κανάλι αυτό. Τέλος στο bit **D₀ θέτουμε πάντα «1»**. Με βάση αυτό το περιεχόμενο, ακολουθεί ο προγραμματισμός του CTC. Επειδή ζητείται η συγγραφή ρουτίνας, ονομάζουμε την ρουτίνα αυτή ως Subrtn και την καλούμε από το κύριο πρόγραμμα με μια εντολή jr. Πριν την κλήση της υπορουτίνας, αποθηκεύουμε στο σωρό τις τιμές των καταχωρητών του Z80 με τις εντολές EX και EXX.

Κύριο πρόγραμμα

..... ; εντολές κύριου προγράμματος

EX AF, AF' ; πριν την κλήση της υπορουτίνας, αποθηκεύεται το περιεχόμενο του A στη στοίβα

EXX ; επίσης, αποθηκεύονται τα περιεχόμενα των υπόλοιπων καταχωρητών στη στοίβα

jr subrtn ; κλήση υπορουτίνας subrtn

HLT

Υπορουτίνα

Subrtn:



LD A, 010X0111b ; A $\leftarrow$ 110X0111b (προγραμματισμός του Control Register του CTC μέσω του Accumulator)

ή

LD A, D7h ; A $\leftarrow$ 11010111b (αν θεωρήσουμε ότι έχουμε rising edge, δηλ. θετική πυροδότηση παλμού)

OUT (42 h), A ; (42H) $\leftarrow$ A, πρώτα στέλνουμε στο κανάλι 2 (CLK/TRG2) τη λέξη ελέγχου για τον προγραμματισμό του CTC

LD A, 05h ; A $\leftarrow$ 05h = 5_{10} = 00000101 (πλήθος παλμών που μετρούνται στην είσοδο). Για να χρησιμοποιηθεί ο CTC ως μετρητής προς τα κάτω (down counter), εισάγουμε μια χρονική σταθερά (time constant) στον αντίστοιχο καταχωρητή και αυτή πρέπει να είναι μικρότερη ή ίση του FF

OUT (42 h), A ; (42 h) $\leftarrow$ A, μετά στέλνουμε στο κανάλι 2 (CLK/TRG2) το όριο παλμών, με τη μορφή χρονικής σταθεράς

LOOP:

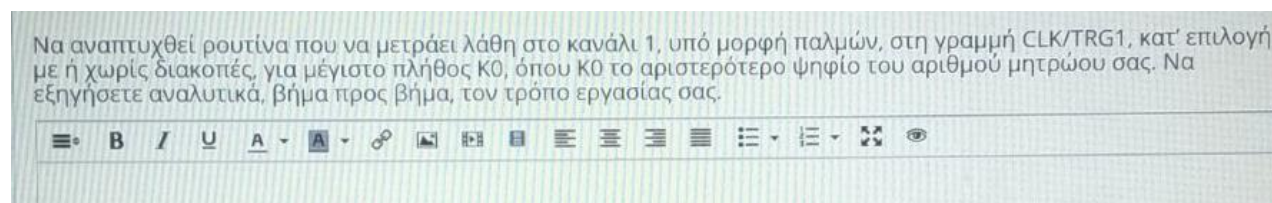
IN A, (42 h) ; A $\leftarrow$ (42 h), δηλ. εισάγουμε στον A (Accumulator) τον παλμό που μετράμε κάθε φορά.
; Κάθε φορά που εισάγεται ένας παλμός, η τιμή του A μειώνεται αυτόματα κατά 1.

CP 00h ; Σύγκριση του περιεχομένου του A (5_{10}) με την τιμή 0, δηλ. εκτέλεση της πράξης A – 00. Αυτό γίνεται για να ελέγχουμε αν έχουν ολοκληρωθεί οι παλμοί που μετράμε στην είσοδο.

JR NZ, LOOP ; για όσο διάστημα ο A **δεν** έχει μηδενιστεί (NZ = No Zero), επιστροφή στην ετικέτα LOOP
; για συνέχιση της μέτρησης της ακολουθίας παλμών στο κανάλι «2». Όταν ο A μηδενιστεί (Z = 1), τότε ολοκληρώνεται ο βρόχος.

RET ; επιστροφή από την υπορουτίνα

Θέμα Σεπτέμβριου 2020



Λύση

Υποθέτουμε ότι έχουμε ένα AM με αριστερότερο ψηφίο ίσο με 2. Άρα το **K0 = 2**. Θα πρέπει να προγραμματίσουμε κατάλληλα το CTC, εγγράφοντας μια τιμή **στον καταχωρητή ελέγχου του (control register)**. Το CTC όταν χρησιμοποιείται ως μετρητής (counter) έχει 4 κανάλια εισόδου, από CLK/TRG0 – CLK/TRG3 και όταν χρησιμοποιείται ως χρονιστής (timer) έχει 3 κανάλια εξόδου, από ZC/TO0 – ZC/TO2. **Επειδή, θα χρησιμοποιήσουμε το CTC ως μετρητή**, θα χρησιμοποιούμε τα **κανάλια εισόδου** και από αυτά θα επιλέξουμε κάθε φορά ένα κανάλι, ανάλογα με την εκφώνηση. Τα κανάλια έχουν μια φυσική διεύθυνση, μέσω της οποίας εντοπίζονται. Θα πρέπει αρχικά να επιλέξουμε το ολοκληρωμένο (CTC) με το σήμα $\sim$ CE και στη συνέχεια θα επιλέξουμε



το κανάλι 1 (CLK/TRG1). Για να γίνει αυτό, θα πρέπει να χρησιμοποιήσουμε τους ακροδέκτες επιλογής καναλιού, CS_1 και CS_0 , στους οποίους θα αναθέσουμε το ζεύγος τιμών 01, δηλ. $CS_1CS_0 = 01$. Θα κάνουμε την παραδοχή ότι το κανάλι «1» που έχει επιλεγεί, αναγνωρίζεται στη διεύθυνση 41H (κάθε διεύθυνση καναλιού επιλέγεται αυθαίρετα, όμως θα πρέπει να τελειώνει στον κατάλληλο δεκαεξαδικό αριθμό, π.χ. 0 → κανάλι 0, 1 → κανάλι 1 κ.ο.κ.). Στη συνέχεια, προχωράμε στον προγραμματισμό του control register του συγκεκριμένου κυκλώματος, βάσει της ακόλουθης εικόνας:

7	6	5 (timer)	4	3 (timer)	2	1	0
Interrupts: 1-Enable 0-Disable	Mode: 1-Counter 0-Timer	Prescaler: 1-256 0-16	Trigger: 1-Rising Edge 0-Falling Edge	1-Start Timer at next CLK pulse 0-Start Timer immediately	1-Time Constant Follows 0-No TC Follows	1-Reset 0-No Reset	1-Control 0-Vector

Θα θεωρήσουμε ότι έχουμε διακοπές. Για τον προγραμματισμό του CTC, σύμφωνα με την εκφώνηση, πρέπει να δώσουμε στον καταχωρητή ελέγχου του, το εξής περιεχόμενο:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
1	1	0	X	0	1	1	1

Στο bit D₇ θέτουμε «1» για να ενεργοποιήσουμε τις διακοπές, αν υποθέσουμε ότι θα χρησιμοποιήσουμε σήματα διακοπής. Στο bit D₆ θέτουμε «1» για να λειτουργήσει το CTC ως μετρητής (counter). Στα bits D₅ και D₃ θέτουμε «0», διότι έχουμε λειτουργία μετρητή. Εναλλακτικά, μπορούμε να θέσουμε και την τιμή «x» στα συγκεκριμένα bits. Στο bit D₄ θέτουμε «X» διότι δεν διευκρινίζεται σε ποια ακμή (ανερχόμενη/κατερχόμενη) της CLK/TRG1 γραμμής θα αρχίσει η μέτρηση. Εναλλακτικά, θα μπορούσαμε να κάνουμε την υπόθεση ότι έχουμε ανερχόμενη/κατερχόμενη ακμή, και να θέσουμε στο bit αυτό τιμή 1/0 αντίστοιχα. Ως χρονική σταθερά (Time Constant Follows) στο bit D₂ λαμβάνουμε το όριο (πλήθος) των παλμών που μετράμε στην είσοδο. Στο bit D₁ θέτουμε «1» για να επιτρέπεται η συνέχιση της μέτρησης από το κανάλι αυτό, διότι θέλουμε να μετράμε συνεχώς παλμούς στην είσοδο. Τέλος στο bit D₀ θέτουμε πάντα «1». Με βάση αυτό το περιεχόμενο, ακολουθεί ο προγραμματισμός του CTC. Επειδή ζητείται η συγγραφή ρουτίνας, ονομάζουμε την ρουτίνα αυτή π.χ. ως **Subrtn** και την καλούμε από το κύριο πρόγραμμα με μια εντολή jr. Πριν την κλήση της υπορουτίνας, αποθηκεύουμε στο σωρό του Z80 τις τιμές των καταχωρητών του, με τις εντολές EX και EXX.

Κύριο πρόγραμμα

..... ; εντολές κύριου προγράμματος

EX AF, AF' ; πριν την κλήση της υπορουτίνας, αποθηκεύεται το περιεχόμενο του A στη στοίβα

EXX ; επίσης, αποθηκεύονται τα περιεχόμενα των υπόλοιπων καταχωρητών στη στοίβα

jr subrtn ; κλήση υπορουτίνας με όνομα subrtn

HLT ; τέλος κύριου προγράμματος

Υπορουτίνα

Subrtn:



LD A, 010X0111b ; A $\leftarrow$ 010X0111b (προγραμματισμός του **Control Register** του CTC μέσω του Accumulator, όπου ; η ένδειξη b δηλώνει δυαδική τιμή) **ή**

LD A, 57h ; A $\leftarrow$ 01010111b (αν θεωρήσουμε ότι έχουμε **rising edge**, δηλ. **θετική πυροδότηση παλμού**)

OUT (41 h), A ; (41H) $\leftarrow$ A, πρώτα στέλνουμε στο κανάλι 1 (CLK/TRG1) τη λέξη ελέγχου για τον προγραμματισμό του CTC

LD A, 02h ; A $\leftarrow$ 02h = 2_{10} = 00000010 (**πλήθος παλμών που μετρούνται στην είσοδο**). Για να χρησιμοποιηθεί ο CTC ως μετρητής προς τα κάτω (down counter), εισάγουμε μια χρονική σταθερά (time constant) ; στον αντίστοιχο καταχωρητή και αυτή **πρέπει να είναι μικρότερη ή ίση του FF**. Αυτή η χρονική σταθερά είναι ουσιαστικά **το πλήθος των παλμών που μετράμε**.

OUT (41 h), A ; (41 h) $\leftarrow$ A, μετά στέλνουμε στο κανάλι 1 (CLK/TRG1) **το όριο παλμών, με τη μορφή χρονικής σταθεράς**

LOOP:

IN A, (41 h) ; A $\leftarrow$ (41 h), δηλ. εισάγουμε στον A (Accumulator) τον παλμό που μετράμε κάθε φορά στο ; κανάλι CLK/TRG1 που έχει διεύθυνση 41h. Κάθε φορά που **εισάγεται ένας παλμός, η τιμή του ; A μειώνεται ; αυτόματα κατά 1**.

CP 00h ; Σύγκριση του περιεχομένου του A (2_{10}) με την τιμή 0, δηλ. εκτέλεση της πράξης A – 00. Αυτό ; γίνεται για να ελέγχουμε αν έχουν ολοκληρωθεί οι παλμοί που μετράμε στην είσοδο.

JR NZ, LOOP; για όσο διάστημα ο A **δεν** έχει μηδενιστεί (NZ = No Zero, δηλ. Z = 0), επιστροφή στην ετικέτα LOOP για ; συνέχιση της μέτρησης της ακολουθίας παλμών από το κανάλι 1. Όταν ο A μηδενιστεί (δηλ. το ; Z = 1), ; τότε η συνθήκη γίνεται ψευδής και ολοκληρώνεται ο βρόχος.

RET ; επιστροφή από την υπορουτίνα

Θέμα με λειτουργία CTC ως timer

Να προγραμματίσετε το CTC ώστε το κανάλι 1 να ξεκινήσει να λειτουργεί **αμέσως** ως **χρονιστής** με αρχική τιμή την A0 και να φτάνει στο «0» μετά από 2560 παλμούς ρολογιού. Δεν απαιτείται η χρήση διακοπών.

Λύση

Θα πρέπει να επιλέξουμε αρχικά το ολοκληρωμένο (CTC) με το σήμα $\sim$ CE και στη συνέχεια να επιλέξουμε το κανάλι 1 (δηλ. το ZC/TO1) ως έξοδο. Για να γίνει αυτό, θα πρέπει να χρησιμοποιήσουμε τους ακροδέκτες επιλογής καναλιού CS_1 και CS_0 , στους οποίους θα αναθέσουμε το ζεύγος τιμών 01, δηλ. **$CS_1CS_0 = 01$** . **Θα κά-**
νουμε την παραδοχή ότι το κανάλι «1» που έχει επιλεγεί, αναγνωρίζεται στη διεύθυνση 41H Επειδή έχουμε προγραμματισμό **CTC ως timer**, ο προγραμματισμός του control register του καταχωρητή αυτού θα είναι:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	0	0	X	0	1	0	1



Στο bit D₇ θέτουμε «0» για να **απενεργοποιήσουμε τις διακοπές**. Στο bit D₆ θέτουμε «0» για να λειτουργήσει το CTC ως χρονιστής (timer). Στο bit D₅ θέτουμε «0», διότι έχουμε λειτουργία χρονιστή και θέλουμε να φτάνει στο «0» μετά από 2560 παλμούς ρολογιού. Για να συμβεί αυτό, **πολλαπλασιάζουμε την αρχική τιμή A0 με τον κατάλληλο Prescaler (διαιρέτης συχνότητας) που είναι είτε 256, είτε 16**. Στην προκειμένη περίπτωση χρειάζεται το 16, διότι $A0 \times 16 = 10100000 \times 16 = (128 + 32) \times 16 = 160 \times 16 = 2560$. Στο bit D₄ θέτουμε «X» διότι δεν διευκρινίζεται σε ποια ακμή (ανερχόμενη/κατερχόμενη) της ZC/TO1 γραμμής θα γίνεται ο χρονισμός. Στο bit D₃ θέτουμε «0», για να ξεκινήσει ο μετρητής να μετρά **αμέσως** (Start Timer immediately). **Ως χρονική σταθερά (Time Constant Follows)** βάζουμε το A0, οπότε στο bit D₂, θέτουμε «1». Στο bit D₁ θέτουμε «0» γιατί ξεκινάμε με μια αρχική τιμή $A0 \neq 0$ (No Reset). Τέλος στο bit D₀ θέτουμε πάντα «1». Με βάση αυτό το περιεχόμενο, ακολουθεί ο προγραμματισμός του. **Δεν χρειάζεται στην περίπτωση αυτή επανάληψη**, όπως γίνεται στην περίπτωση του Counter.

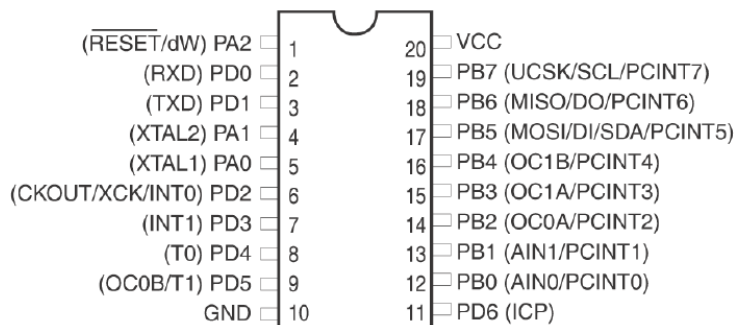
Συμπερασματικά, αναφέρεται ότι ο προγραμματισμός του CTC είτε σε λειτουργία counter, είτε σε λειτουργία timer, γίνεται πάντα σε **δύο** στάδια: Πρώτα, προγραμματίζεται ο control register του CTC και στέλνεται το περιεχόμενό του στη διεύθυνση του καναλιού που χρησιμοποιούμε και στη συνέχεια ακολουθεί η χρονική σταθερά (που είτε είναι το όριο των παλμών που μετράμε, είτε είναι η αρχική τιμή από την οποία θα ξεκινήσει ο timer) που στέλνεται και πάλι στη διεύθυνση του καναλιού που χρησιμοποιούμε. Η εντολή OUT εξάγει το περιεχόμενο ενός καταχωρητή σε μια περιφερειακή συσκευή. Επομένως, ο προγραμματισμός του CTC είναι ο εξής:

LD A, 000x0101b	;A ← 000x0101b
OUT (41h), A	;(41h) ← A.
LD A, A0h	; A ← A0h (χρονική σταθερά)
OUT (41h), A	;(41h) ← A (εξαγωγή της χρονικής σταθεράς στο κανάλι 1)

16. Μικροελεγκτής ATiny2313

Μια **πρώτη διαφορά** ενός μικροεπεξεργαστή και ενός μικροελεγκτή είναι ότι ο τελευταίος αποτελεί ένα σύστημα που περιλαμβάνει στο ίδιο ολοκληρωμένο κύκλωμα: την ΚΜΕ, τις μνήμες των κατηγοριών RAM και ROM και τα απαραίτητα περιφερειακά κυκλώματα. Πρόκειται για **πλήρες υπολογιστικό σύστημα**. Μια **δεύτερη διαφορά** μεταξύ μικροεπεξεργαστή και μικροελεγκτή είναι η αρχιτεκτονική μνήμης που χρησιμοποιούν, δηλ. Von Neumann έναντι Harvard. Αυτός ο μικροελεγκτής περιέχει ενσωματωμένα περιφερειακά κυκλώματα και οι ακροδέκτες του διαθέτουν πολύπλεξη λειτουργιών. Αυτό σημαίνει ότι κάθε ακροδέκτης μπορεί να εκτελέσει **ταυτόχρονα περισσότερες από μία λειτουργίες**, όπως φαίνεται στην επόμενη εικόνα.

PDIP/SOIC



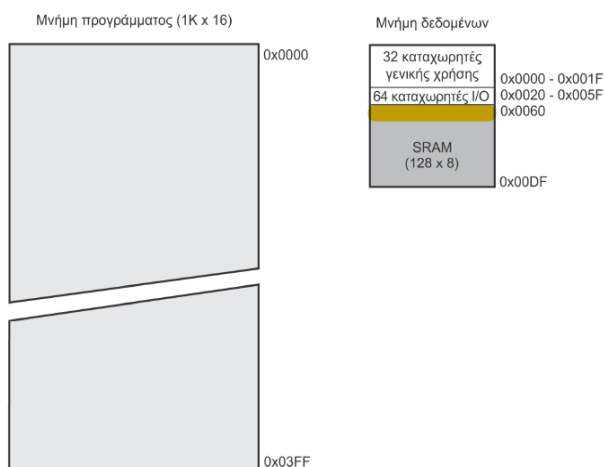
Z80 (αρχιτεκτονική Von Neumann) $\leftrightarrow$ ενιαίος χώρος διευθύνσεων εντολών και δεδομένων

ATiny 2313 (αρχιτεκτονική Harvard) $\leftrightarrow$ ξεχωριστός χώρος διευθύνσεων εντολών και δεδομένων

Ο μικροελεγκτής Atiny 2313 ακολουθεί το μοντέλο **Harvard για την αρχιτεκτονική μνήμης του**, που σημαίνει ότι σε άλλη μνήμη βρίσκονται οι εντολές και σε άλλη τα δεδομένα. **Μάλιστα αυτές οι δύο μνήμες διαφέρουν όχι μόνο ως προς τη χωρητικότητά τους, αλλά επίσης και ως προς το μέγεθος της κάθε θέσης μνήμης (εσωτερική οργάνωση).**

Μνήμη Προγράμματος και Δεδομένων

Αρχιτεκτονική Von Neumann



0000h-3FFFh EPROM
4000h-DFFFh RAM
E000h-EFFFh Περιφερειακά
F000h-FFFFh BOOT ROM

- Ενιαίος χώρος διευθύνσεων.
- Μικροεπεξεργαστής: 16-bit δίαυλο διευθύνσεων μπορεί να δει μέχρι και 65536 θέσεις μνήμης με το ίδιο μήκος λέξης.
- Τμήματα του χώρου διευθύνσεων για RAM, ROM, ή και περιφερειακά.

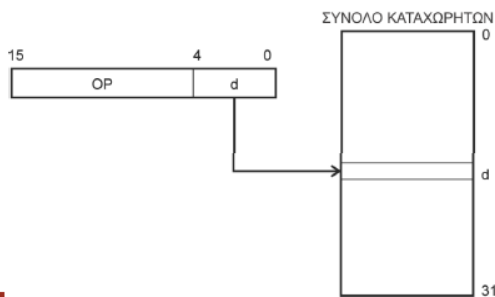
Οι καταχωρητές X, Y, Z του συγκεκριμένου μικροελεγκτή είναι των 16 bits, διότι καθένας αποτελείται από δύο καταχωρητές R_i των 8 bits. Συγκεκριμένα, ο καταχωρητής X περιέχει τους καταχωρητές R_{26} και R_{27} , ο καταχωρητής Y περιέχει τους καταχωρητές R_{28} και R_{29} και ο καταχωρητής Z περιέχει τους καταχωρητές R_{30} και R_{31} .

Τρόποι διευθυνσιοδότησης ATiny 2323

Απευθείας διευθυνσιοδότηση (Direct)

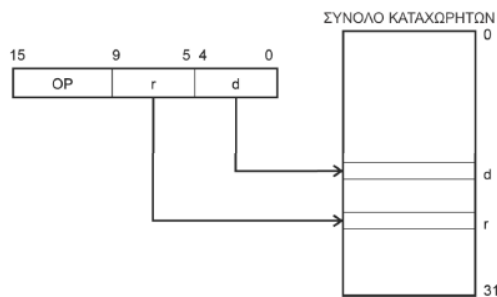
Απευθείας (ή άμεση) με έναν καταχωρητή

Απευθείας με δύο καταχωρητές



INC R5

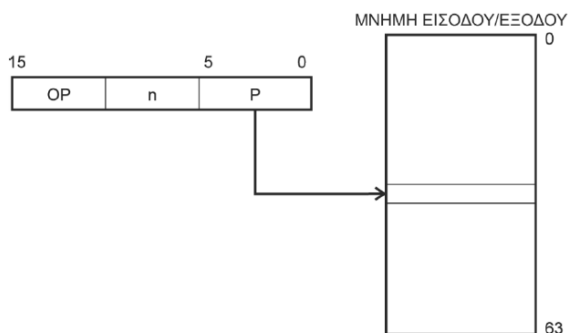
;R5 $\leftarrow$ R5 + 1 (με ένα έντελο)



EOR R4, R5

;R4 $\leftarrow$ R4 $\oplus$ R5 (με δύο έντελα)

Απευθείας διευθυνσιοδότηση σε καταχωρητή περιφερειακού (Direct I/O)

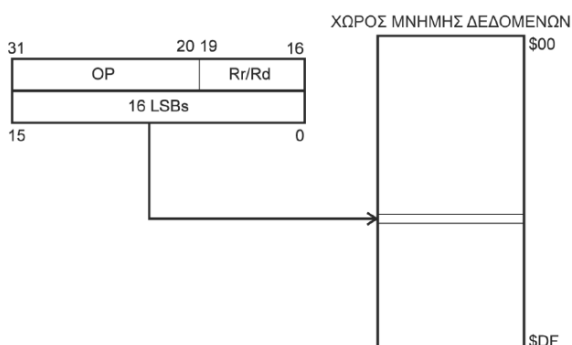


Έξοδος σε καταχωρητή περιφερειακού

OUT \$33, R4

;R4 $\leftarrow$ R4

Απευθείας διευθυνσιοδότηση δεδομένων (Data Direct)

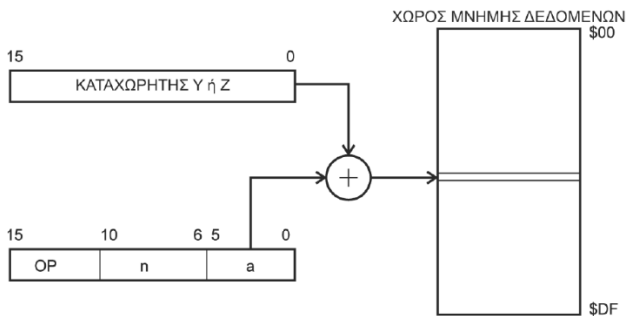


Απευθείας σε τιμή δεδομένων (αριθμός)

ANDI R8, 15

;R8 $\leftarrow$ R8 AND 15

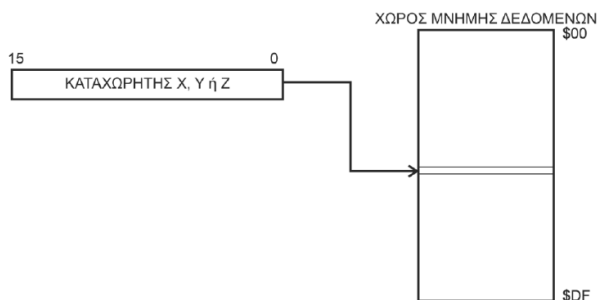
Έμμεση διευθυνσιοδότηση δεδομένων με εκτόπιση (Data Indirect with Displacement)



Έμμεση δεδομένων με μετατόπιση (εκτόπιση)

ST Y+30, R4 ; Y+30 $\leftarrow$ R4

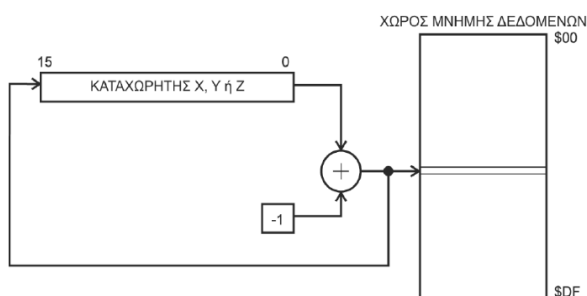
Έμμεση διευθυνσιοδότηση δεδομένων (Data Indirect)



Έμμεση δεδομένων χωρίς μετατόπιση

ST Y, R4 ; Y $\leftarrow$ R4

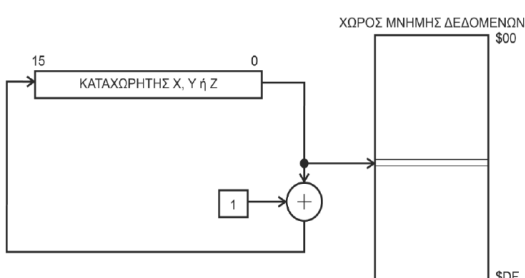
Έμμεση διευθυνσιοδότηση δεδομένων με μείωση του δείκτη (indirect with pre-decrement)



Έμμεση δεδομένων με προηγούμενη μείωση του δείκτη κατά 1

ST -Y, R4 ; Y $\leftarrow$ Y - 1
; Y $\leftarrow$ R4

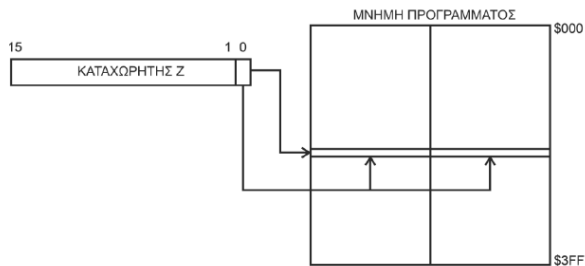
Έμμεση διευθυνσιοδότηση δεδομένων με αύξηση του δείκτη (indirect with post-increment)



Έμμεση δεδομένων με επακόλουθη αύξηση του δείκτη κατά 1

ST X+, R4 ; X $\leftarrow$ R4
; X $\leftarrow$ X+1

Σταθερή διευθυνσιοδότηση για προσπέλαση μνήμης προγράμματος (Constant using LPM)

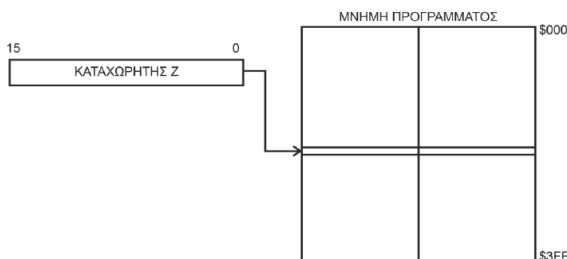


LPM = Load Program Memory.

Στην εντολή αυτή το δεξιότερο bit (LSB) επιλέγει ανάλογα με την τιμή του (0 ή 1) τη λειτουργία της εντολής.

- Ο μοναδικός τρόπος διευθυνσιοδότησης που επιτρέπει την ανάγνωση των περιεχομένων της μνήμης του προγράμματος,
- Χρησιμοποιείται με την εντολή **LPM**, για τη φόρτωση ενός byte,

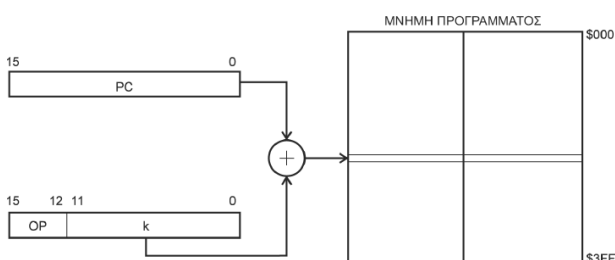
Έμμεση διευθυνσιοδότηση προγράμματος (Program Indirect)



IJMP, ICALL (στο άλμα που πραγματοποιείται, **δεν** συμμετέχει ο PC)

- Ο δείκτης Z καθορίζει τη διεύθυνση της μνήμης προγράμματος που θα φορτωθεί ο μετρητής προγράμματος (Program Counter),

Σχετική διευθυνσιοδότηση με μετρητή προγράμματος (Relative Program)



RJMP, RCALL (στο άλμα που πραγματοποιείται, **συμμετέχει** ο PC, δηλ. το άλμα γίνεται βάσει της τιμής του PC)

Άσκηση τρόπων διευθυνσιοδότησης

Ποια **λειτουργία** επιτελούν οι κάτωθι εντολές και να αναγνωρίσετε **τον τρόπο διευθυνσιοδότησης** των παρακάτω εντολών:

- ADD R5, R9** ; $R5 \leftarrow R5 + R9$. Άμεση ή απευθείας διευθυνσιοδότηση με 2 καταχωρητές
- ST Z+, R3** ; $Z \leftarrow R3$; $Z \leftarrow Z+1$. Έμμεση δεδομένων με επακόλουθη αύξηση του δείκτη κατά 1.
- ST -Z, R10** ; $Z \leftarrow Z - 1$, $Z \leftarrow R10$. Έμμεση δεδομένων με προηγούμενη μείωση του δείκτη κατά 1.
- ROL R5** ; Αριστερή περιστροφή μέσω κρατουμένου στον R5 (Rotate Left through Carry), άμεση.
- BSET C** ; Θέτει το C = 1 που υπάρχει στον καταχωρητή κατάστασης SREG του Atiny 2313, άμεση



RJMP 20 ;Άλλα σχετικό ως προς το δείκτη προγράμματος. Σχετική διευθυνσιοδότηση με μετρητή προγράμματος
LDD R2, Z+3 ; $R2 \leftarrow Z + 3$. Φορτώνει καταχωρητή με περιεχόμενο θ.μ. Έμμεση δεδομένων με εκτόπιση.
SBIC 30h, 1 ;Κάνει skip την επόμενη εντολή, αν το bit “1” του καταχωρητή του περιφερειακού με διεύθυνση 30h είναι 1, άμεση σε καταχωρητή περιφερειακού.

Χρήση δείκτη στοίβας ATiny 2313 – Παραδείγματα χρήσης του δείκτη στοίβας

Κανόνες: Η αρχική τιμή της στοίβας πρέπει να τεθεί στη διεύθυνση πάνω από 60h ή και 60h. Δεν πρέπει να πέσει κάτω από την τιμή αυτή.

- Κάθε φορά που τοποθετείται στη στοίβα ένα byte (με εντολή push), η τιμή του SP μειώνεται κατά ένα.
- Κάθε φορά που καλείται μια υπορουτίνα (με εντολή call), η διεύθυνση επιστροφής της (2 bytes) τοποθετείται στη στοίβα (push) και η τιμή του SP μειώνεται κατά δύο.
- Κάθε φορά που ανακαλείται από τη στοίβα (pop) ένα byte, η τιμή του SP αυξάνεται κατά ένα
- Κάθε φορά που έχουμε επιστροφή από μια υπορουτίνα (με εντολή ret), η διεύθυνση επιστροφής (2 bytes) αφαιρείται από τη στοίβα (pop) και η τιμή του SP αυξάνεται κατά δύο.

Παράδειγμα 1: Αρχικοποιείται ο δείκτης στοίβας (SP) στην τιμή 70h. Επιτρέπεται η εκτέλεση 9 υπορουτινών;

Λύση

Η θέση μνήμης της στοίβας (που βρίσκεται στη μνήμη δεδομένων) είναι μεγέθους 8 bits = 1 byte. Κάθε φορά που τοποθετείται η διεύθυνση μιας υπορουτίνας (16 bit) στη στοίβα, η τιμή του SP μειώνεται κατά 2. Για 9 συνολικά υπορουτίνες χρειαζόμαστε $2 \times 9 = 18$ θ.μ. = 12h. Αν αφαιρέσουμε από την τρέχουσα τιμή του SP, την τιμή 12h, θα έχουμε πράξη: $70h - 12h = 5Eh < 60h$. Επομένως, δεν επιτρέπεται η εκτέλεση 9 υπορουτινών. Εναλλακτικά η αφαίρεση στο δυαδικό.

Παράδειγμα 2: Θέλουμε να εκτελέσουμε 8 υπορουτίνες και το διπλάσιο αριθμό για αποθήκευση μεταβλητών. Σε ποια θέση πρέπει να αρχικοποιηθεί ο SP για να είναι εφικτή αυτή η αποθήκευση;

Λύση

Επειδή κάθε μεταβλητή πιάνει μια θέση στη στοίβα και κάθε υπορουτίνα πιάνει δύο θέσεις, συνολικά χρειαζόμαστε 8×2 bytes (θ.μ.) + 16×1 byte = 32 bytes (θ.μ.) = $100000_2 = 20h$. Με δεδομένο ότι η κατώτατη τιμή του SP είναι το 60h, θα πρέπει να τον αρχικοποιήσουμε στην τιμή $60h + 20h = 80h$. Αν ζητηθεί και χαρτογράφηση μνήμης (δηλ. σε ποιες διευθύνσεις μνήμης θα τοποθετηθούν οι μεταβλητές και σε ποιες διευθύνσεις μνήμης θα τοποθετηθούν οι διευθύνσεις επιστροφής των υπορουτινών, τότε στην περιοχή διευθύνσεων $60h - 70h$ τοποθετούνται οι 16 μεταβλητές ($16_{10} = 10h$) στη συνέχεια από $70h - 80h$ τοποθετούνται οι διευθύνσεις επιστροφής των υπορουτινών.

Παράδειγμα 3: Έστω ότι σε ένα σύστημα μικροελεγκτή ATtiny2313, η στοίβα αρχικοποιείται στην τιμή 93h. Να βρεθεί το μέγιστο πλήθος των ένθετων υπορουτινών που μπορεί να κληθεί. Πως καταλήξατε σε αυτό το συμπέρασμα;



Λύση

Η στοίβα τίθεται στην τιμή 93h. Με δεδομένο ότι, η μικρότερη τιμή του SP είναι το 60 h, εκτελούμε την αφαίρεση $93h - 60h = 33h$. Με δεδομένο ότι λαμβάνουμε υπόψη και την αρχική διεύθυνση, ο συνολικός αριθμός των θέσεων μνήμης της στοίβας είναι $34h = 00110100 = 52_{10}$. **Με δεδομένο κάθε διεύθυνση υπορουτίνας καταλαμβάνει 2 θέσεις στη στοίβα**, το μέγιστο πλήθος των ένθετων υπορουτινών που μπορεί να κληθεί είναι 26.

Παρατήρηση: σε περίπτωση που η στοίβα αρχικοποιείται στην τιμή 73h, το μέγιστο πλήθος των ένθετων υπορουτινών που μπορεί να κληθεί είναι $73h - 60h = 13h$. Με δεδομένο ότι λαμβάνουμε υπόψη και την αρχική διεύθυνση, ο συνολικός αριθμός των θέσεων μνήμης της στοίβας είναι $14h = 00010100 = 20_{10}$. **Με δεδομένο κάθε διεύθυνση υπορουτίνας καταλαμβάνει 2 θέσεις στη στοίβα**, το μέγιστο πλήθος των ένθετων υπορουτινών που μπορεί να κληθεί είναι 10.

Πηγές διακοπών ATiny 2313

Για το μικροελεγκτή Atiny 2313 να εξηγηθεί η πηγή επανεκκίνησης του reset που καθυστερεί σε σχέση με την τροφοδοσία, σχηματικό διάγραμμα και εξήγηση.

Λύση

Διαφάνειες 8^{ου} σελ από 28 – 33.

Θέμα Ιουνίου 2020

Θεωρείστε ένα μικροπολογιστικό σύστημα, με χρήση του Atiny2313. Εάν τα τέσσερα δεξιότερα ψηφία του αριθμού μητρώου σας: K3K2K1K0 (ψηφία στο δεκαδικό σύστημα αρίθμησης), να βρείτε την τιμή του δείκτη στοίβας μετά το τέλος της εκτέλεσης του προγράμματος: α) LDI R1K0, K3K1h β) POP R2K0 γ) RCALL Exams δ) RET ε) PUSH R1K2 στ) RCALL Exams ζ) RET

Η τιμή του δείκτη στοίβας είναι

Λύση

Έστω το $AM = 4311 = K3K2K1K0$, επομένως $K0 = 1, K1 = 1, K2 = 3, K3 = 4$. **Θα υποθέσουμε ότι $SP = 70h$.** Η πρώτη εντολή του κώδικα που ακολουθεί, δεν έχει κάποια πρακτική σημασία αναφορικά με την τιμή του δείκτη στοίβας που ψάχνουμε. **Οι εντολές που επηρεάζουν το δείκτη στοίβας είναι οι εντολές, RCALL – RET (αυξομειώνουν την τιμή του SP κατά ± 2), PUSH – POP (αυξομειώνουν την τιμή του SP κατά ± 1).** Επίσης, **οι καταχωρητές που χρησιμοποιούνται στο πρόγραμμα είναι όλοι του ενός byte.**

LDI R11, 41h	;R11 $\leftarrow$ 41h (LDI = Load Immediate = απευθείας φόρτωση καταχωρητή με μια συγκεκριμένη τιμή).
POP R21	;Ανάκτηση από τη στοίβα του περιεχομένου του καταχωρητή R21 (SPL $\leftarrow$ SPL + 1 = 71h).
RCALL Exams ;	;Κλήση της υπορουτίνας με όνομα Exams. Επομένως, αποθηκεύεται στη στοίβα η διεύθυνση
	;επιστροφής από την υπορουτίνα Exams (Άρα SP $\leftarrow$ SP - 2 = 71h - 2 = 6Fh)
RET	;επιστροφή από υπορουτίνα, οπότε ανακτάται από τη στοίβα η διεύθυνση επιστροφής για την
	;υπορουτίνα (SPL $\leftarrow$ SPL + 2 = 6Fh + 2 = 71h).
PUSH R13	;αποθηκεύεται στη στοίβα η τιμή του καταχωρητή R13 (SPL $\leftarrow$ SPL - 1 = 71h - 1 = 70h).
RCALL Exams ;	;Αποθηκεύεται στη στοίβα η διεύθυνση επιστροφής από την υπορουτίνα Exams (SPL $\leftarrow$ SPL - 2
	= 70h - 2 = 6Eh)
RET	;επιστροφή από υπορουτίνα, οπότε ανακτάται από τη στοίβα η διεύθυνση επιστροφής από την
	;υπορουτίνα (SPL $\leftarrow$ SPL + 2 = 6Eh + 2 = 70h).

Θέμα Σεπτεμβρίου 2020

Θεωρείστε ένα μικρουπολογιστικό σύστημα, με χρήση του Atiny2313. Εάν τα τέσσερα αριστερότερα ψηφία του αριθμού μητρώου σας: K3K2K1K0 (ψηφία στο δεκαδικό σύστημα αρίθμησης), να βρείτε την τιμή του δείκτη στοίβας μετά το τέλος της εκτέλεσης του προγράμματος: α) LDI R2K2, K1K0h β) POP R1K3 γ) RCALL Success δ) RET ε) PUSH R3K2 στ) RCALL Success ζ) RET η) STS SPL, K3K2#. Να εξηγήσετε αναλυτικά, βήμα προς βήμα, τον τρόπο εργασίας σας.

Λύση

Έστω το AM = 4311 = K3K2K1K0, επομένως K0 = 1, K1 = 1, K2 = 3, K3 = 4. Θα υποθέσουμε ότι **SPL = 70h**

LDI R23, 41h ;R23 ← 41h. Και πάλι αυτή η εντολή δεν επηρεάζει τον SP.
 POP R14 ;Ανάκτηση από τη στοίβα του καταχωρητή R14 (SPL ← SPL + 1 = 71h)
 RCALL Success; ;Αποθηκεύεται στη στοίβα η διεύθυνση επιστροφής από την υπορουτίνα Success (SPL ← SPL - 2 = 71h - 2 = 6Fh)
 RET ;επιστροφή από υπορουτίνα, οπότε ανακτάται από τη στοίβα η διεύθυνση επιστροφής της υπορουτίνας (SPL ← SPL + 2 = 6Fh + 2 = 71h).
 PUSH R33 ;αποθηκεύεται στη στοίβα η τιμή του καταχωρητή R33 (SP ← SP - 1 = 71h - 1 = 70h).
 RCALL Success; ;Αποθηκεύεται στη στοίβα η διεύθυνση επιστροφής από την υπορουτίνα Success (SP ← SP - 2 = 70h - 2 = 6Eh).
 RET ;επιστροφή από υπορουτίνα, οπότε ανακτάται από τη στοίβα η διεύθυνση επιστροφής από την υπορουτίνα (SPL ← SPL + 2 = 6Eh + 2 = 70h).
 STS SPL, 43# ;αποθηκεύεται η δεκαδική τιμή 43 στο SPL (Stack Pointer Low). Αυτό σημαίνει ότι η τελική του SP **θα είναι η τιμή 43 στο δεκαδικό (η προηγούμενη τιμή του SPL διαγράφεται)**. Δηλαδή, ανεξάρτητα από το πλήθος των εντολών που προηγούνται, η τελική τιμή του SPL προσδιορίζεται ουσιαστικά από την εντολή αυτή. Θα πρέπει να αναφερθεί ότι το 43₁₀ = 2Bh < 60 h. Επομένως, έχουμε εσφαλμένη αποθήκευση σε I/O καταχωρητές.

Παρατήρηση: Ο Stack Pointer στον ATtiny 2313 αποτελείται από δύο επιμέρους καταχωρητές: SPL και τον SPH, καθένας από τους οποίους είναι των 8 bits.

Bit	15	14	13	12	11	10	9	8	
	-	-	-	-	-	-	-	-	SPH
	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	SPL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R	R	R	R	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	
	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	

Θέμα Ιανουαρίου 2020

Για την οικογένεια μικροελεγκτών ATiny, να εξηγήσετε πως μεταβάλλεται η τιμή του δείκτη στοίβας με τη διαδοχική εκτέλεση των παρακάτω εντολών:

5. Για την οικογένεια μικροελεγκτών ATiny, να εξηγήσετε πως μεταβάλλεται η τιμή του δείκτη στοίβας με την διαδοχική εκτέλεση των παρακάτω εντολών (20%):
 α) STS SPL, R10
 β) RCALL OperationB
 γ) RET
 δ) RCALL OperationA
 ✓ Το ερώτημα να απαντηθεί μόνο στην 6^η σελίδα του φυλλαδίου απαντήσεων.

Λύση

Θέμα 5

Έστω ότι υλοποιείτε την αρχιτεκτονική:

```
LDI R20, 62H
STS SPL, R20
```

αυτοδημιουργείται η τιμή του R20 στο Stack Pointer Low

RCALL OperationB: $SPL \leftarrow SPL - 2 = 60H$, δηλ. μειώνεται η τιμή μισών 2 και αποθηκεύεται στη σελίδα η διεύθυνση επιστροφής από την καλούμενη υπορουτίνα

RET : $SPL \leftarrow SPL + 2 = 62H$. Αναστρέφεται από τη σελίδα η διεύθυνση επιστροφής από υπορουτίνα

RCALL OperationA: $SPL \leftarrow SPL - 2 = 60H$. Δεν επιτρέπεται να πάσει ο καταχωρητής σελίδας να πάσει κάτω από τη διεύθυνση 60H

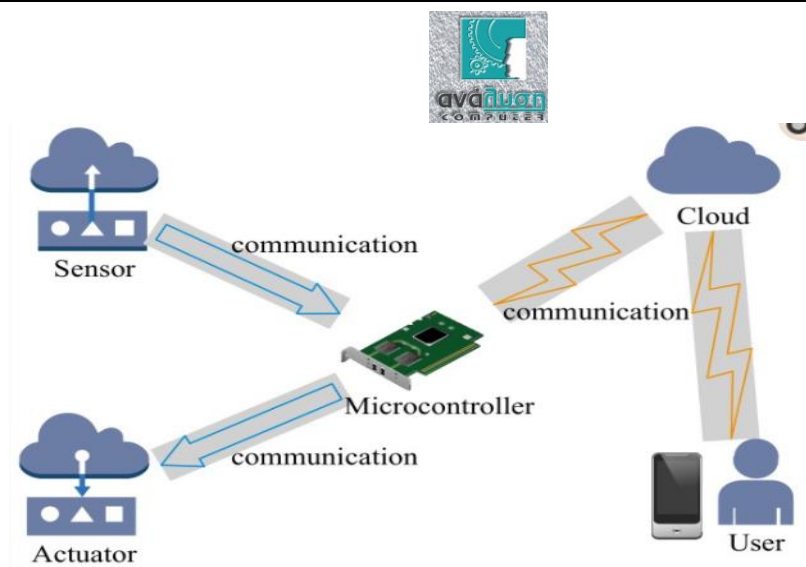
Θέμα Ιανουαρίου 2020

Θεωρείστε για μια δομή έξυπνου σπιτιού (smart home), τη διάταξη ενός συστήματος ασφαλείας συναγερμού). α) Να σχεδιάσετε την αρχιτεκτονική της εφαρμογής ενός μικροελεγκτή, για τη λειτουργία της διάταξης αυτής (10%), β) να εξηγήσετε τη λειτουργία και τα κριτήρια της αρχιτεκτονικής που σχεδιάσατε (10%).

Η απάντηση να απαντηθεί μόνο στη 2^η σελίδα του φυλλαδίου απαντήσεων.

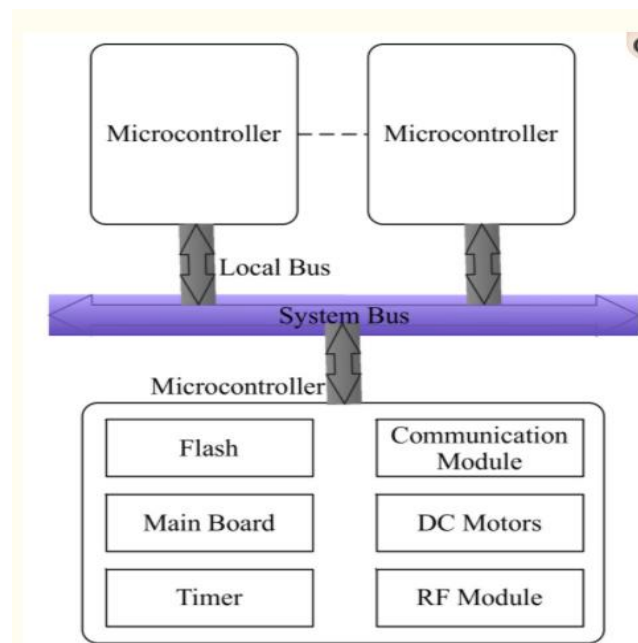
Λύση

(α) Μια επισκόπηση ενός συστήματος IoT φαίνεται στην Εικόνα που ακολουθεί, η οποία περιλαμβάνει τα ακόλουθα βασικά στοιχεία: **(1) αισθητήρες (sensor)**: χρησιμοποιούνται για την παρακολούθηση του περιβάλλοντος έτσι ώστε να μπορούν να δημιουργηθούν δεδομένα ανίχνευσης. Αυτά τα δεδομένα θα μπορούσαν να επεξεργαστούν τοπικά ή να μεταφορτωθούν στο κέντρο cloud για ασφαλή αποθήκευση. **(2) επικοινωνία (communication)**: οι συσκευές θα μπορούσαν να διασυνδεθούν μεταξύ τους και να έχουν τη δυνατότητα διασύνδεσης με το Διαδίκτυο. Πρωτόκολλα επικοινωνίας όπως Zigbee, Bluetooth και WiFi θα μπορούσαν να χρησιμοποιηθούν σε αυτό το στοιχείο. **(3) μικροελεγκτές (microcontroller)**: κάνουν όλες τις συσκευές να λειτουργούν μαζί ως σύστημα παροχής έξυπνων υπηρεσιών. Ένας μικροελεγκτής αποτελείται από στοιχεία υλικού και λογισμικού. Τα δεδομένα ανίχνευσης αποστέλλονται στον μικροελεγκτή για επεξεργασία και αυτός παράγει σήματα για ενεργοποιητές. Με αυτόν τον τρόπο, ολόκληρο το σύστημα θα μπορούσε να ελεγχθεί στην κατάσταση - στόχο. **(4) ενεργοποιητές (actuator)**: χρησιμοποιούνται για χειρισμό του περιβάλλοντος. Οι ενεργοποιητές παίρνουν ένα ηλεκτρικό σήμα ως είσοδο και μετατρέπουν το σήμα σε μηχανική φυσική κίνηση. **(5) διεπαφές χρήστη (user)**: το σύστημα IoT θα πρέπει να παρέχει έξυπνες υπηρεσίες στις έξυπνες συσκευές (smartphones) των χρηστών και οι αλληλεπιδράσεις μεταξύ τους θα πρέπει να γίνονται με τον κατάλληλο τρόπο.



Εικόνα 7: Η επισκόπηση ενός συστήματος IoT

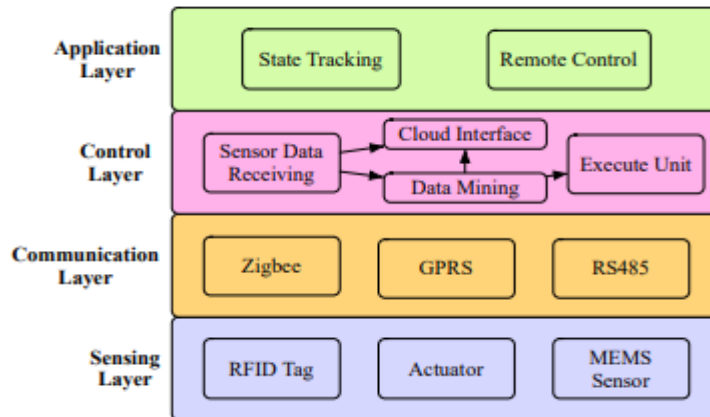
(β) Γενικά, ένας μικροελεγκτής περιλαμβάνει τα ακόλουθα στοιχεία: **μητρική πλακέτα, flash memory, χρονοδιακόπτη, μονάδα επικοινωνίας, κινητήρες συνεχούς ρεύματος (DC) και μονάδες ραδιοσυχνοτήτων (RF)**. Η **μητρική πλακέτα (main Board)** χρησιμοποιείται για τη σύνδεση και τον συντονισμό όλων των εξαρτημάτων. Τα δεδομένα ανίχνευσης και διαμόρφωσης θα μπορούσαν να αποθηκευτούν από την μνήμη **Flash**. Το στοιχείο χρονοδιακόπτη (**Timer**) θα μπορούσε να παρέχει τον πραγματικό χρόνο του συστήματος. Με τη μονάδα επικοινωνίας, ο έξυπνος μικροελεγκτής θα μπορούσε να επικοινωνεί με τα απομακρυσμένα συστήματα, όπως τα κέντρα **cloud**. Προκειμένου να λάβει τα δεδομένα από το περιβάλλον και να αλληλεπιδράσει με αυτό, έχουν σχεδιαστεί δύο εξαρτήματα που ονομάζονται **DC Motors** και **μονάδες ραδιοσυχνοτήτων (RF Module)**. Επιπλέον, ένας μικροελεγκτής θα πρέπει να συνεργάζεται με άλλους μικροελεγκτές προκειμένου να επιτελεί τις σύνθετες εργασίες. Αυτό ονομάζεται **κατανεμημένη αρχιτεκτονική**. Σε μια κατανεμημένη αρχιτεκτονική, οι μικροελεγκτές συνδέονται με **δίαυλο συστήματος (System bus)**. Κάθε μικροελεγκτής συνδέεται στο δίαυλο συστήματος μέσω **τοπικού διαύλου (Local bus)**. Η αρχιτεκτονική ενός έξυπνου εμφανίζεται στην επόμενη εικόνα.



Εικόνα 8: Αρχιτεκτονική ενός έξυπνου μικροελεγκτή

Ο απώτερος στόχος ενός έξυπνου μικροελεγκτή είναι να παρέχει έξυπνες υπηρεσίες στους χρήστες. Επομένως, εκτός από την περίπλοκη σχεδίαση των εξαρτημάτων υλικού, θα πρέπει επίσης να αποσαφηνιστούν οι υπηρεσίες που θα πα-

ρέχει. Το πλαίσιο λογισμικού για την εφαρμογή του προτεινόμενου μικροελεγκτή φαίνεται στην επόμενη Εικόνα. Οι λειτουργίες του λογισμικού μικροελεγκτή χωρίζονται σε τέσσερα επίπεδα, συμπεριλαμβανομένου του επιπέδου ανίχνευσης (sensing layer), του επιπέδου επικοινωνίας (control layer), του επιπέδου ελέγχου (communication layer) και του επιπέδου εφαρμογής (application layer).



Εικόνα 9: Το πλαίσιο λογισμικού μικροελεγκτή

Στο **επίπεδο ανίχνευσης (sensing layer)**, ο μικροελεγκτής αλληλεπιδρά με συσκευές τελικού σημείου όπως αισθητήρες και ενεργοποιητές. Αυτό το επίπεδο περιλαμβάνει την επιλογή τύπων αισθητήρων σύμφωνα με τους διαφορετικούς τύπους εργασιών, όπως ετικέτες RFID, αισθητήρες MEMS. Τα δεδομένα του αισθητήρα θα μπορούσαν να θεωρηθούν ως η είσοδος του μικροελεγκτή. Ο ενεργοποιητής λαμβάνει το σήμα εξόδου του μικροελεγκτή και εκτελεί τις αντίστοιχες ενέργειες.

Το **επίπεδο επικοινωνίας (communication layer)** περιλαμβάνει τα πρωτόκολλα επικοινωνίας, τα οποία ο μικροελεγκτής χρησιμοποιούσε για να αλληλεπιδράσει με άλλα στοιχεία, όπως αισθητήρες, ενεργοποιητές και τα κέντρα cloud. Γενικά, το πρωτόκολλο Zigbee χρησιμοποιείται για την επικοινωνία με τους κόμβους Zigbee. Το πρωτόκολλο GPRS χρησιμοποιείται για την επικοινωνία με τα απομακρυσμένα κέντρα cloud. Το πρωτόκολλο RS485 χρησιμοποιείται για τοπική επικοινωνία με άλλους μικροελεγκτές.

Το **επίπεδο ελέγχου (control layer)** είναι το βασικό συστατικό ενός μικροελεγκτή. Σε αυτό το επίπεδο, ο μικροελεγκτής θα πρέπει να αποθηκεύει και να αναλύει τα ληφθέντα δεδομένα. Επομένως, το επίπεδο ελέγχου περιλαμβάνει τις ακόλουθες λειτουργίες: λήψη δεδομένων αισθητήρα, εξόρυξη δεδομένων, διεπαφή κέντρου cloud και μονάδα εκτέλεσης. Η λειτουργικότητα λήψης δεδομένων αισθητήρα ανταποκρίνεται στη συλλογή και αποθήκευση των δεδομένων αισθητήρα. Η λειτουργία εξόρυξης δεδομένων χρησιμοποιείται για την ανάλυση των δεδομένων με μεθόδους μηχανικής μάθησης. Η διεπαφή του κέντρου cloud θα μπορούσε να στείλει τα αρχικά δεδομένα ή να αναλύσει τα αποτελέσματα στο απομακρυσμένο κέντρο σύννεφων. Η μονάδα εκτέλεσης ανακτά τα αποτελέσματα εξόρυξης δεδομένων και καθορίζει τις εντολές εκτέλεσης εξόδου.

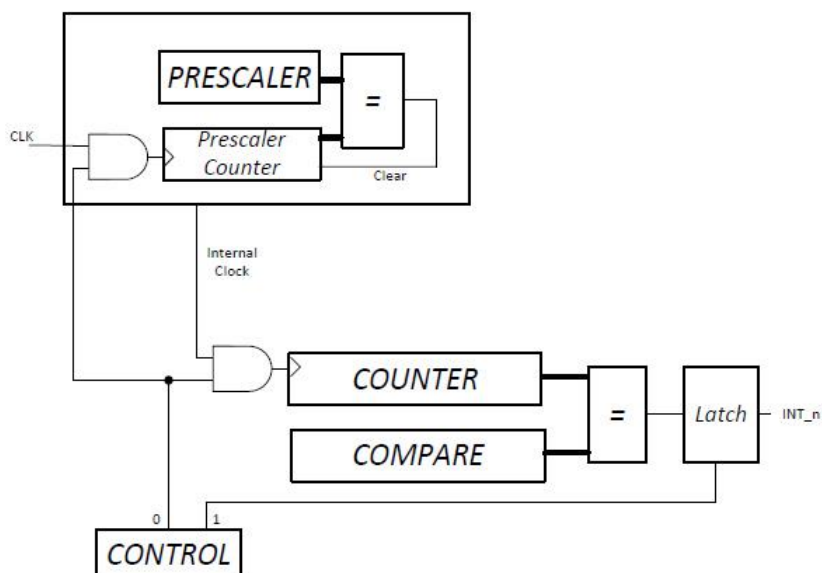
Στο **επίπεδο εφαρμογής (application layer)**, υπάρχουν δύο βασικές λειτουργίες. Η μία είναι η παρακολούθηση καταστάσεων (state tracking) και το άλλο είναι ο απομακρυσμένος έλεγχος (remote control). Η παρακολούθηση καταστάσεων θα μπορούσε να παρακολουθεί την κατάσταση των αισθητήρων, των ενεργοποιητών και του ίδιου του μικροελεγκτή. Ο απομακρυσμένος έλεγχος παρέχει έναν μηχανισμό για τον έλεγχο από απόσταση των αισθητήρων ή των ενεργοποιητών.

17. TIMER 0

TIMER0	
CE_n	
OE_n	
WE_n	INT_n
A[2:0]	
D[7:0]	
CLK	

Οι ακροδέκτες A[2:0] του address bus χρησιμοποιούνται για τη διευθυνσιοδότηση των καταχωρητών του Timer 0. **Προϋπόθεση** για να λειτουργεί ο Timer0 είναι αφενός το σήμα RESET_n να είναι απενεργοποιημένο (δηλαδή «1») και αφετέρου το σήμα CE_n να είναι ενεργοποιημένο (δηλαδή «0»). Οι άλλοι δύο ακροδέκτες OE_n και WE_n ενεργοποιούνται ανάλογα με την περίπτωση.

Λειτουργία: Ο Timer0 μετρά κάποια τιμή που του έχουμε αναθέσει στον καταχωρητή Compare και όταν τη μετρήσει, ενεργοποιείται το σήμα INT_n, το οποίο και διακόπτει τον επεξεργαστή. Στον Timer0 εισέρχεται ένα εξωτερικό ρολόι, το οποίο δίνει τη συχνότητα λειτουργίας του (CLK). Επειδή, όμως ορισμένα κυκλώματά του χρειάζονται χαμηλότερη συχνότητα ρολογιού, υπάρχει ένα εσωτερικό κύκλωμα που ονομάζεται **Prescaler** (διαιρέτης συχνότητας), ο οποίος παίρνει ως είσοδο το (εξωτερικό) ρολόι και το υποδιαιρεί σε μικρότερες συχνότητες. Ο καταχωρητής **Counter** του Timer0 μετρά με βάση το εσωτερικό (υποδιαιρεμένο) ρολόι μέχρι να φτάσει την τιμή που περιέχει ο καταχωρητής **Compare** του Timer0. **Τότε ενεργοποιούνται οι διακοπές με τον ακροδέκτη INT_n** και επιπλέον μηδενίζεται ο Counter, για να αρχίσει να μετρά από την αρχή. Οι καταχωρητές Counter, Compare, Prescaler χρησιμοποιούνται ως ζεύγη 8 – bit καταχωρητών με τις ενδείξεις L και H στο τέλος των ονομάτων τους, π.χ. PREH, PREL κ.ο.κ. Ισχύει ο τύπος: $\frac{\text{συχνότητα εξωτερικού ρολογιού}}{\text{prescaler}} = \text{συχνότητα εσωτερικού ρολογιού}$ ή $\frac{F(\text{CLK})}{\text{prescaler}} = F(\text{internal clock})$.



Ασκήσεις με τον Timer

- a) Στην παρακάτω άσκηση δίνονται οι συχνότητες του εξωτερικού ρολογιού και του εσωτερικού ρολογιού. Ζητείται η τιμή του διαιρέτη συχνότητας (Prescaler).



F(CLK) = 4 MHz
F(internal Clock) = 1 KHz
PRESCALER = ?

Ισχύει ότι $\frac{F(CLK)}{prescaler} = F(\text{internal clock}) \Rightarrow \frac{4 \text{ MHz}}{x} = 1 \text{ KHz} \Rightarrow x = \frac{4 \text{ MHz}}{1 \text{ KHz}} = \frac{4 \times 10^6 \text{ Hz}}{1 \times 10^3 \text{ Hz}} = 4000$. Η τιμή του

Prescaler είναι πάντα αριθμός δεκαδικός χωρίς μονάδες μέτρησης. Αυτήν την τιμή πρέπει να τη δώσουμε ως περιεχόμενο (σε δεκαεξαδική μορφή) στον καταχωρητή Prescaler, ο οποίος αποτελείται από δύο επιμέρους καταχωρητές: PREL και PREH. Το $4000_{10} = \text{FA0h} = 0\text{FA0h}$, οπότε στον PREL = A0h, και στον PREH = 0Fh.

b) Στην παρακάτω άσκηση δίνεται η συχνότητα του εξωτερικού ρολογιού και η τιμή του διαιρέτη συχνότητας (Prescaler). Ζητείται η τιμή του εσωτερικού ρολογιού.

F(CLK) = 4 MHz
PRESCALER = 40 (decimal)
F(internal Clock) = ?

Ισχύει ότι $\frac{F(CLK)}{prescaler} = F(\text{internal clock}) \Rightarrow F(\text{internal clock}) = \frac{4 \text{ MHz}}{40} = \frac{4 \times 10^6 \text{ Hz}}{40} = 10^5 \text{ Hz} =$

100 KHz.

c) Στην παρακάτω άσκηση δίνεται η συχνότητα του εξωτερικού ρολογιού και η συχνότητα των διακοπών F(interrupts). Ζητείται η τιμή με την οποία πρέπει να διαιρεθεί η συχνότητα του εσωτερικού ρολογιού F(internal clock), η οποία ουσιαστικά θα τεθεί ως περιεχόμενο στον καταχωρητή Compare.

F(CLK) = 4 MHz
F(Interrupts) = 1 Hz
????

Στην περίπτωση αυτή έχουμε γνωστή τη συχνότητα του εξωτερικού ρολογιού F(CLK) = 4 MHz και τη συχνότητα των διακοπών F(interrupts). Θα πρέπει να θεωρηθεί γνωστή η συχνότητα του Prescaler, και έστω

ότι είναι 4000_{10} . Τότε η συχνότητα του εσωτερικού ρολογιού (internal clock) $= \frac{4 \text{ MHz}}{4000} = \frac{4 \times 10^6 \text{ Hz}}{4 \times 10^3} = 1000 \text{ Hz} = 1 \text{ KHz} = F(\text{internal clock})$.

Θέλουμε η συχνότητα των διακοπών F(interrupts) = 1 Hz. Για το λόγο αυτό θα πρέπει το $\frac{F(\text{internal clock})}{x} = \frac{1000 \text{ Hz}}{x} =$

$1 \text{ Hz} \Rightarrow x = 1000$. Αυτή είναι η τιμή που θα πρέπει να φορτώσουμε στον καταχωρητή Compare και αυτό γιατί κάθε φορά που το περιεχόμενο του καταχωρητή Counter γίνεται ίσο με το περιεχόμενο του καταχωρητή Compare, το σύστημα του TIMER0 προκαλεί διακοπή στη γραμμή INT_n. Με άλλα λόγια ο καταχωρητής Compare περιέχει την τιμή που θέλουμε να μετρήσει ο μετρητής. Επειδή οι τιμές των περιεχομένων των καταχωρητών είναι σε μορφή Hex, το $1000 = 3\text{E8h} = 03\text{E8h}$ και το high byte της τιμής αυτής (03) θα πάει στον CMPH, δηλαδή CMPH = 03h, ενώ το low byte της τιμής αυτής (E8) θα πάει στον καταχωρητή CMPL, δηλαδή CMPL = E8h.

Θέμα Φεβρουαρίου 2018

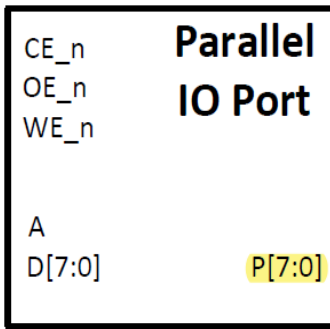
Θεωρείστε μια τυπική διάταξη timer/counter στην οποία η είσοδος F(Clk) είναι ίση με 8 MHz. Εάν το εσωτερικό ρολόι ισούται με την τιμή 4 KHz, ζητείται να βρεθεί η τιμή του PRESCALER. Να αιτιολογήσετε την απάντησή σας.

Λύση

Ισχύει ότι $\frac{\text{εξωτερικό ρολόι}}{prescaler} = \text{εσωτερικό ρολόι ή αλλιώς } \frac{F(Clk)}{prescaler} = F(\text{internal clock}) \Rightarrow \frac{8 \text{ MHz}}{prescaler} = 4 \text{ KHz} \Rightarrow \text{Prescaler} = \frac{8 \text{ MHz}}{4 \text{ KHz}}$

$= \frac{8 \times 10^6 \text{ Hz}}{4 \times 10^3 \text{ Hz}} = 2000_{10}$. Αυτήν την τιμή πρέπει να τη θέσουμε ως περιεχόμενο στους καταχωρητές PREH και PREL και για το λόγο αυτό θα πρέπει να τη μετατρέψουμε σε δεκαεξαδική. Το $2000_{10} = 07\text{D0h}$, οπότε PREH = 07h και στον PREL = D0h.

18. Port



Πρόκειται για παράλληλη θύρα εισόδου/εξόδου 8 ακροδεκτών P[7: 0]. Ο προγραμματισμός της παράλληλης θύρας E/E γίνεται μέσω του καταχωρητή DIRECTION, ο οποίος μπορεί να προγραμματίσει το PORT να λειτουργεί είτε ως είσοδος (DATA_IN), είτε ως έξοδος (DATA_OUT). Πιο συγκεκριμένα, αν θέσουμε κάποιο bit στον καταχωρητή DIRECTION ίσο με 1/0, σημαίνει ότι ο αντίστοιχος ακροδέκτης του PORT λειτουργεί ως έξοδος/είσοδος αντίστοιχα. Οι ακροδέκτες του PORT είναι όλοι αρνητικής λογικής. Σε αντίθεση με τον TIMER0, υπάρχει μόνο ένα bit από το address bus, που είναι το A και αυτό γιατί για τιμή A = A[0] = 0 επιλέγεται ο καταχωρητής DATA (ο οποίος ονομάζεται DATA_OUT ή DATA_IN) και A = A[0] = 1 επιλέγεται ο καταχωρητής DIRECTION. Στις ασκήσεις ο καταχωρητής κατεύθυνσης DIRECTION διατυπώνεται ως καταχωρητής κατάστασης και ο καταχωρητής DATA_OUT διατυπώνεται ως δίαυλος εξόδου.

Ασκήσεις με το Port

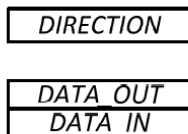
a) Ζητάμε τον προγραμματισμό του PORT έτσι ώστε:

P[7:4] : Inputs
P[3:0] : Outputs

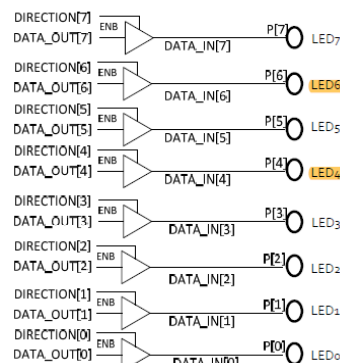
Θα πρέπει να προγραμματίσουμε κατάλληλα τον καταχωρητή DIRECTION του PORT.

Πιο συγκεκριμένα, πρέπει να τον προγραμματίσουμε με την τιμή 00001111 = 0Fh.

b) Ζητάμε τον προγραμματισμό του PORT έτσι ώστε:



P[7:0] : Outputs
Να ανάψουν τα Leds 4,6 μόνο
DATA_OUT = ;

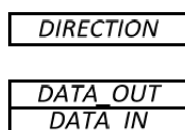


Για να προγραμματιστούν όλοι οι ακροδέκτες του PORT ως έξοδοι, θα πρέπει στον καταχωρητή DIRECTION να θέσουμε ως περιεχόμενο την τιμή 11111111 = FFh. Επειδή, όλοι οι ακροδέκτες του PORT έχουν προγραμματιστεί ως έξοδοι (έχουμε συνδέσει σε αυτούς leds), θα πρέπει να χρησιμοποιηθεί μόνο ο καταχωρητής DATA_OUT και να προγραμματιστεί με την τιμή 50h = 01010000, διότι έτσι θέτουμε «1»

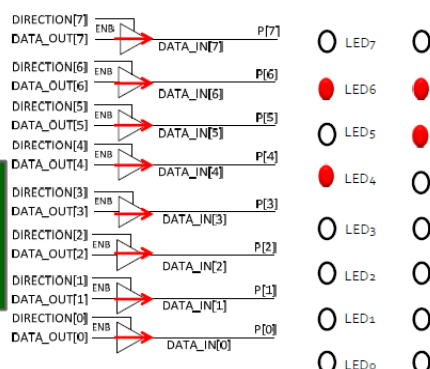
στους ακροδέκτες 4, 6 και ανάβουν τα leds που έχουν συνδεθεί στους συγκεκριμένους ακροδέκτες.

Παράδειγμα 3

DIRECTION = FF (hex)
DATA_OUT = 50 (hex)



P[7:0] : 50 (hex)
(Τα Leds 4,6 είναι ήδη αναμμένα)
Θέλουμε να ανάψουμε το 5 &
να σβήσουμε το 4.
Το 6 να μείνει αναμμένο.



c)

Κανόνας: όταν θέλουμε να αλλάξουμε το περιεχόμενο του καταχωρητή DATA, θα πρέπει να το εισάγουμε στον συσσωρευτή (A), να κάνουμε σε αυτόν τις αλλαγές που θέλουμε και στη συνέχεια να εξάγουμε το περιεχόμενο του A πίσω στο Port.



IN A, (DATA_IN)

OR 20h

AND EFh

OUT (DATA_OUT), A

; A ← 50h = 01010000

; A ← A OR 00100000 = 01010000 OR 00100000 = 01110000 = 70h,

; για να ανάψει το LED 5.

; A ← A AND 11101111 = 01110000 AND 11101111 = 01100000 = 60h

; για να σβήσει το LED 4.

; DATA_OUT ← 60

Κανόνες: όταν θέλουμε να ανάψει κάποιο/α Led(s) εκτελούμε την πράξη OR, θέτοντας «1» στη θέση (-εις) που είναι το (-α) Led(s) που θέλουμε να ανάψει (-ουν). Αντίστοιχα, όταν θέλουμε να σβήσει κάποιο/α Led(s), εκτελούμε την πράξη AND, θέτοντας «0» στη θέση που είναι το (-α) Led(s) που θέλουμε να σβήσει (-ουν).

Θέμα Σεπτεμβρίου 2019

Σε μια παράλληλη θύρα εισόδου/εξόδου ο καταχωρητής κατάστασης έχει την τιμή «B5» και οδηγεί μια συστοιχία από 8 leds, με τιμή διαύλου εξόδου «A7». Να γράψτε τις αντίστοιχες εντολές του προγράμματος για να **διατηρήσετε** μόνο την κατάσταση λειτουργίας όλων των leds στις θέσεις με **περιττό** αριθμό. Να αναλυθεί ο τρόπος εργασίας σας.

Λύση

Ο καταχωρητής κατάστασης είναι ο DIRECTION = B5 = **10110101**. Αυτό σημαίνει ότι οι ακροδέκτες «0», «2», «4», «5», «7» έχουν προγραμματιστεί ως **έξοδοι** και οι υπόλοιποι ως **είσοδοι**. Συμπεραίνουμε ότι οι ακροδέκτες του PORT με **περιττό** αριθμό που λειτουργούν ως **έξοδοι**, είναι οι ακροδέκτες «5», «7». Η τιμή διαύλου εξόδου αναφέρεται στον καταχωρητή DATA_OUT = A7 = 10100111.

IN A, (DATA_IN)

; A ← A7h

AND A0h

; A ← A AND A0h = A AND 10100000 (διατηρούμε τις τιμές των leds που υπάρχουν στους ακροδέκτες με περιττό αριθμό «5», «7» και σβήνουμε όλα τα υπόλοιπα.)

OUT (DATA_OUT), A

; DATA_OUT ← A0

Θέμα Φεβρουαρίου 2019

Σε μια παράλληλη θύρα εισόδου/εξόδου ο καταχωρητής κατάστασης έχει την τιμή «A4» και οδηγεί μια συστοιχία από 8 leds, με τιμή διαύλου εξόδου «CB». Να γράψτε τις αντίστοιχες εντολές του προγράμματος για να **διατηρήσετε** μόνο την κατάσταση λειτουργίας όλων των leds στις θέσεις με **περιττό** αριθμό. Να αναλυθεί ο τρόπος εργασίας σας.

Λύση

Ο καταχωρητής κατάστασης είναι ο DIRECTION = A4 = **10100100**. Αυτό σημαίνει ότι οι ακροδέκτες «2», «5», «7» έχουν προγραμματιστεί ως **έξοδοι** και οι υπόλοιποι ως **είσοδοι**. Συμπεραίνουμε ότι οι ακροδέκτες του PORT με **περιττό** αριθμό που λειτουργούν ως **έξοδοι**, είναι οι ακροδέκτες «5», «7». Η τιμή διαύλου εξόδου αναφέρεται στον καταχωρητή DATA_OUT = CB = 11001011. Από εδώ συμπεραίνουμε ότι το led5 είναι σβηστό και το led7 είναι ανοικτό.

IN A, (DATA_IN)

; A ← CBh

AND A0h

; A ← A AND A0h = A AND 10100000 (διατηρούμε τις τιμές των leds που υπάρχουν στους ακροδέκτες με περιττό αριθμό «5», «7» και σβήνουμε όλα τα υπόλοιπα.)

OUT (DATA_OUT), A

; DATA_OUT ← A



Θέμα Φεβρουαρίου 2016

Σε μια παράλληλη θύρα εισόδου/εξόδου ο καταχωρητής κατάστασης έχει την τιμή «FF» και οδηγεί μια συστοιχία από 8 leds, με τιμή διαύλου εξόδου «AA». Να γράψτε τις αντίστοιχες εντολές του προγράμματος για να **αντιστρέψτε** την κατάσταση λειτουργίας όλων των leds στις θέσεις με **άρτιο** αριθμό. Να αναλυθεί ο τρόπος εργασίας σας.

Λύση

Ο καταχωρητής κατάστασης είναι ο $DIRECTION = FF = 11111111$. Αυτό σημαίνει ότι όλοι οι ακροδέκτες έχουν προγραμματιστεί ως **έξοδοι**. Η τιμή διαύλου εξόδου αναφέρεται στον καταχωρητή $DATA_OUT = AA = 10101010$. Με την τιμή αυτή καταλαβαίνουμε ότι τα leds που είναι ανοικτά είναι αυτά των θέσεων «1», «3», «5» και «7», ενώ τα leds των άρτιων θέσεων είναι σβηστά.

IN A, (DATA_IN) ; $A \leftarrow AAh$,

OR 55h ; $A \leftarrow A \text{ OR } 55h = A \text{ OR } 01010101$ (ανάβουμε τα leds των άρτιων θέσεων, δηλ. αντιστρέφουμε την κατάσταση λειτουργίας τους).

OUT (DATA_OUT), A ; $DATA_OUT \leftarrow A$

Θέμα Σεπτεμβρίου 2017

Σε μια παράλληλη θύρα εισόδου/εξόδου ο καταχωρητής κατάστασης έχει την τιμή «CA» και οδηγεί μια συστοιχία από 8 leds, με τιμή διαύλου εξόδου «FE». Να γράψτε τις αντίστοιχες εντολές του προγράμματος για να **αντιστρέψτε** την κατάσταση λειτουργίας όλων των leds στις θέσεις με **περιττό** αριθμό. Να αναλυθεί ο τρόπος εργασίας σας.

Λύση

Ο καταχωρητής κατάστασης είναι ο $DIRECTION = CA = 11001010$. Αυτό σημαίνει ότι ακροδέκτες «1», «3», «6», «7» έχουν προγραμματιστεί ως **έξοδοι**. Η τιμή διαύλου εξόδου αναφέρεται στον καταχωρητή $DATA_OUT = FE = 11111110$. Με την τιμή αυτή καταλαβαίνουμε ότι τα leds που είναι ανοικτά είναι αυτά των θέσεων «1», «3», «6» και «7». Τα leds στις θέσεις με περιττό αριθμό, των οποίων θέλουμε να **αντιστρέψουμε** την κατάσταση λειτουργίας τους, είναι αυτά των θέσεων «1», «3» και «7».

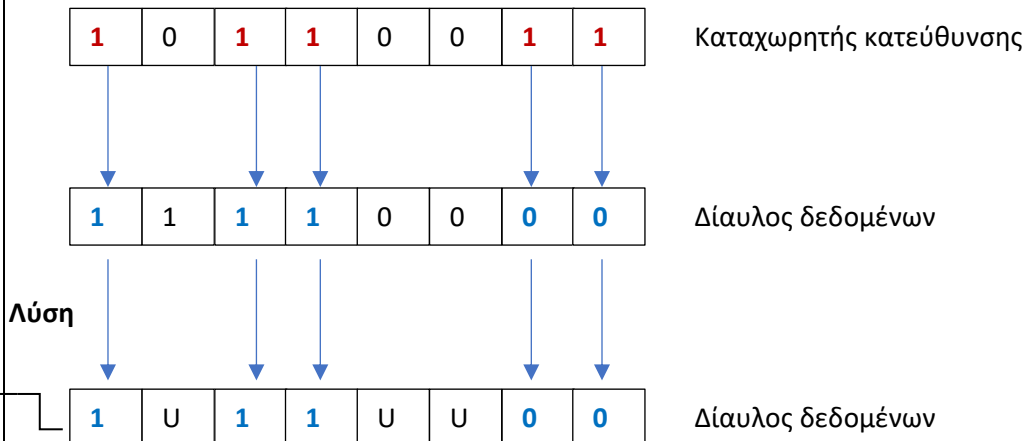
IN A, (DATA_IN) ; $A \leftarrow FEh$,

AND 75h ; $A \leftarrow A \text{ AND } 75h = A \text{ AND } 01110101$.

OUT (DATA_OUT), A ; $DATA_OUT \leftarrow A$

Ασκήσεις με το Port του Z80

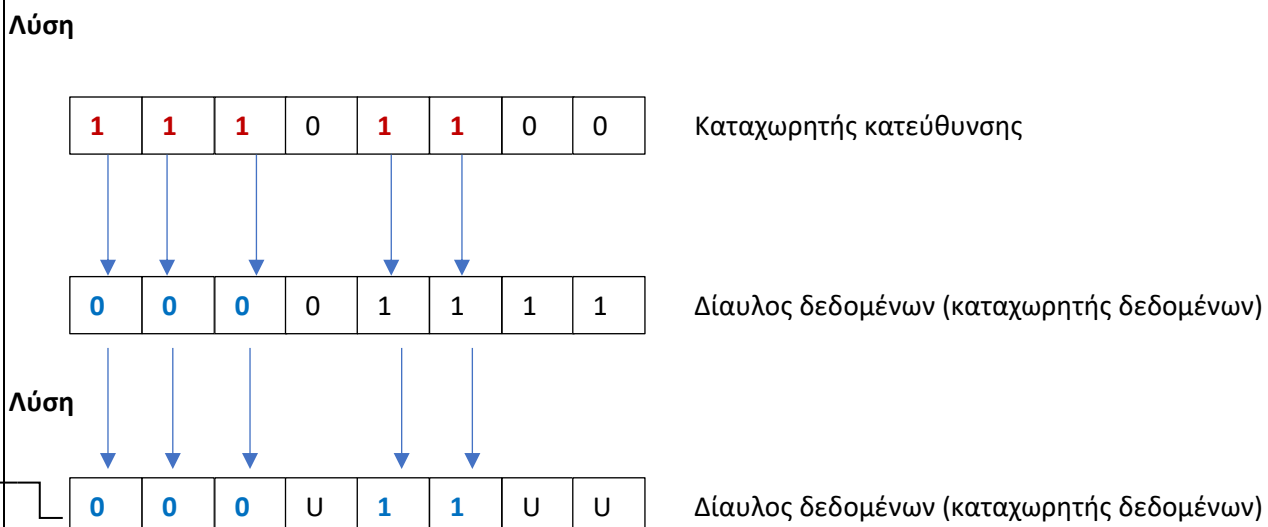
Έστω ότι έχουμε μια θύρα 8 ακροδεκτών εισόδου/εξόδου και επιλέγουμε μια τυχαία τιμή για τον καταχωρητή κατεύθυνσης. Ισχύει η εξής σύμβαση: **0: είσοδος και 1: έξοδος**. Ο καταχωρητής κατεύθυνσης περιέχει μια τυχαία τιμή. Δίνονται τα αρχικά περιεχόμενα του καταχωρητή κατεύθυνσης και του διαύλου δεδομένων. Μετά από μια χρονική στιγμή (1 παλμό) ποιο θα είναι το νέο περιεχόμενο του διαύλου δεδομένων;



Παρατηρούμε ότι όταν στον καταχωρητή κατεύθυνσης υπάρχει «1» (δηλαδή έχουμε εξόδους), τα αντίστοιχα δυαδικά ψηφία του διαύλου δεδομένων **διατηρούν** την τιμή τους μετά από έναν παλμό ρολογιού. Σε διαφορετική περίπτωση, τα αντίστοιχα bits του διαύλου δεδομένων γίνονται απροσδιόριστα (U). Ο συμβολισμός U = Undefined = απροσδιόριστη κατάσταση.

Θέμα Σεπτεμβρίου 2019

Σε μια θύρα E/E των 8 ακροδεκτών, ο καταχωρητής κατεύθυνσης οδηγείται με τη δυαδική τιμή 11101100. Στη συνέχεια αποθηκεύεται στον καταχωρητή δεδομένων η τιμή 00001111. Ποια τιμή θα έχει ο καταχωρητής δεδομένων όταν ξαναδιαβαστεί.



19. Ασκήσεις με βήματα εντολών επεξεργαστή 8080A

Έστω ότι μας δίνεται η εντολή του επεξεργαστή 8080A: **ADD A, 3000h**, που αθροίζει το περιεχόμενο του συσσωρευτή (A) με το περιεχόμενο της θέσης μνήμης με διεύθυνση 3000h και καταχωρεί το αποτέλεσμα στο συσσωρευτή (A). Έχουμε ως δεδομένα τα εξής:

- Συσσωρευτής A.
- Η εντολή είναι αποθηκευμένη στη θέση μνήμης με διεύθυνση 8000h.
- Ο κωδικός της (opcode) είναι μεγέθους 1 byte και η διεύθυνση 3000h καταλαμβάνει άλλα δύο bytes, τα οποία είναι αποθηκευμένα στις διευθύνσεις 8001h και 8002h.

- Η αρχιτεκτονική που χρησιμοποιείται είναι η little - endian.
- Τα περιεχόμενα της θέσης μνήμης με διεύθυνση 3000h είναι ο αριθμός F0h.

Ζητούμενο: να περιγράψετε τα βήματα εκτέλεσης της εντολής.

Λύση

1° βήμα: πρέπει να γίνει οδήγηση του διαύλου διευθύνσεων (address bus) με την τιμή του μετρητή προγράμματος (PC = Program Counter) 8000h, προκειμένου να γίνει η προσκόμιση του κωδικού λειτουργίας (opcode) της εντολής ADD από τη συγκεκριμένη διεύθυνση.

2° βήμα: πρέπει να αυξηθεί η τιμή του PC κατά «1», για να δείξει στην επόμενη διεύθυνση 8001h.

3° βήμα: με δεδομένο ότι έχουμε πάρει τη διεύθυνση 8000h, η μνήμη τοποθετεί στο δίαυλο δεδομένων (data bus) τον κωδικό λειτουργίας της εντολής ADD.

4° βήμα: αυτός ο κωδικός λειτουργίας αποθηκεύεται από τη μονάδα ανάκλησης εντολής, στον καταχωρητή εντολών (IR = Instruction Register), που βρίσκεται στην ΚΜΕ. Αυτό το βήμα είναι ουσιαστικά η προσκόμιση της εντολής (fetch).

5° βήμα: η μονάδα αποκωδικοποίησης της ΚΜΕ θα διακρίνει ότι πρόκειται για τη συγκεκριμένη εντολή (ADD A, 3000h) και θα οδηγήσει τα κατάλληλα σήματα ελέγχου στη μονάδα εκτέλεσης εντολών. Αυτό το βήμα είναι ουσιαστικά η αποκωδικοποίηση της εντολής (decode).

6° βήμα: η μονάδα εκτέλεσης εντολών αποφασίζει για την προσκόμιση άλλων δύο bytes.

7° βήμα: ο PC οδηγεί το δίαυλο διευθύνσεων στη θέση 8001h και έπειτα αυξάνεται κατά «1».

8° βήμα: η μνήμη οδηγεί το δίαυλο δεδομένων (data bus) με τα περιεχόμενα της θέσης 8001h, δηλαδή το 00.

9° βήμα: ο δίαυλος διευθύνσεων οδηγείται στη θέση 8002h από τον PC και έπειτα ο PC αυξάνεται κατά «1».

10° βήμα: η μνήμη οδηγεί το δίαυλο δεδομένων (data bus) με τα περιεχόμενα της θέσης 8002h, δηλαδή το 30.

11° βήμα: ο δίαυλος διευθύνσεων, αφού έχει πάρει την τιμή 00 και την τιμή 30, οδηγείται στη διεύθυνση 3000h, που ουσιαστικά είναι η διεύθυνση που έχει σχηματιστεί στις δύο προηγούμενες προσπελάσεις.

12° βήμα: η μνήμη οδηγεί το δίαυλο δεδομένων με την τιμή (περιεχόμενο) της διεύθυνσης 3000h, που είναι το F0h.

13° βήμα: η ALU παίρνει τα δεδομένα από τον ACC (A) και από τον καταχωρητή γενικού σκοπού (FO) και εκτελεί την πράξη της πρόσθεσης και αποθηκεύει το αποτέλεσμα στον ACC.

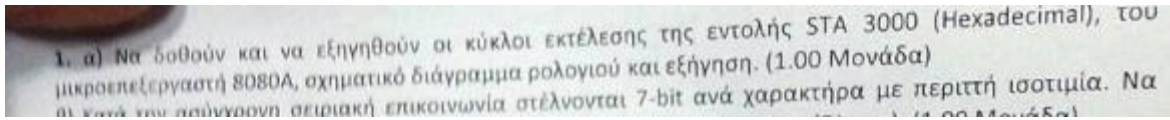
14° βήμα: η τιμή του PC (8003h) οδηγείται στο δίαυλο διευθύνσεων, για την προσκόμιση της επόμενης εντολής.

Σημείωση: παρόμοιες εντολές που μπορούν να ζητηθούν για τα βήματα εκτέλεσής τους είναι οι εξής:

ADD A, #30h; $A \leftarrow A + 30h$ (αυτός ο αριθμός δεν χρειάζεται να προσκομιστεί, θεωρούμε ότι υπάρχει στην ΚΜΕ).
INC A; $A \leftarrow A + 1$, ADD A, B; $A \leftarrow A + B$

8000h	Opcode ADD
8001h	00
8002h	30

Θέμα Ιανουαρίου 2016



Λύση

Θα κάνουμε τις παραδοχές:

- Συσσωρευτής A.
- Η εντολή είναι αποθηκευμένη στη θέση μνήμης με διεύθυνση 8000h.
- Ο κωδικός της (opcode) είναι μεγέθους 1 byte και η διεύθυνση 3000h καταλαμβάνει άλλα δύο bytes, τα οποία είναι αποθηκευμένα στις διευθύνσεις 8001h και 8002h.
- Η αρχιτεκτονική που χρησιμοποιείται είναι η little-endian.

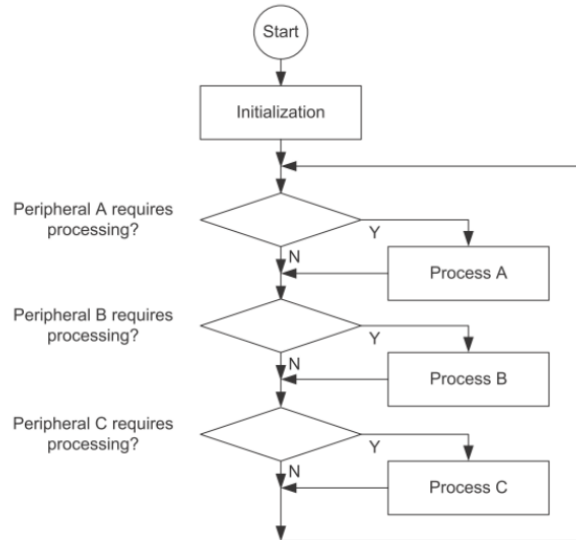
Τα βήματα που απαιτούνται για την εκτέλεση της εντολής είναι τα εξής:

- 1° βήμα:** πρέπει να γίνει οδήγηση του διαύλου διευθύνσεων (address bus) με την τιμή του μετρητή προγράμματος (PC = Program Counter) 8000h, προκειμένου να γίνει η προσκόμιση του κωδικού λειτουργίας (opcode) της εντολής STA από τη συγκεκριμένη διεύθυνση.
- 2° βήμα:** πρέπει να αυξηθεί η τιμή του PC κατά «1», για να δείξει στην επόμενη διεύθυνση 8001h.
- 3° βήμα:** με δεδομένο ότι έχουμε πάρει τη διεύθυνση 8000h, η μνήμη τοποθετεί στο δίαυλο δεδομένων (data bus) τον κωδικό λειτουργίας της εντολής STA.
- 4° βήμα:** αυτός ο κωδικός λειτουργίας αποθηκεύεται από τη μονάδα ανάκλησης εντολής, στον καταχωρητή εντολών (IR = Instruction Register), που βρίσκεται στην ΚΜΕ.
- 5° βήμα:** η μονάδα αποκωδικοποίησης της ΚΜΕ θα διακρίνει ότι πρόκειται για τη συγκεκριμένη εντολή (STA 3000h) και θα οδηγήσει τα κατάλληλα σήματα ελέγχου στη μονάδα εκτέλεσης εντολών.
- 6° βήμα:** η μονάδα εκτέλεσης εντολών αποφασίζει για την προσκόμιση άλλων δύο bytes.
- 7° βήμα:** ο PC οδηγεί το δίαυλο διευθύνσεων στη θέση 8001h και έπειτα αυξάνεται κατά «1».
- 8° βήμα:** η μνήμη οδηγεί το δίαυλο δεδομένων (data bus) με τα περιεχόμενα της θέσης 8001h, δηλαδή το 00.
- 9° βήμα:** ο δίαυλος διευθύνσεων οδηγείται στη θέση 8002h από τον PC, και έπειτα ο PC αυξάνεται κατά «1».
- 10° βήμα:** η μνήμη οδηγεί το δίαυλο δεδομένων (data bus) με τα περιεχόμενα της θέσης 8002h, δηλαδή το 30.
- 11° βήμα:** ο δίαυλος διευθύνσεων, αφού έχει πάρει την τιμή 00 και την τιμή 30, οδηγείται στη διεύθυνση 3000h, που ουσιαστικά είναι η διεύθυνση που έχει σχηματιστεί στις δύο προηγούμενες προσπελάσεις.
- 12° βήμα:** ο δίαυλος δεδομένων μεταφέρει το περιεχόμενο του ACC στη διεύθυνση 3000h και το αποθηκεύει.
- 13° βήμα:** η τιμή του PC (8003h) οδηγείται στο δίαυλο διευθύνσεων, για την προσκόμιση της επόμενης εντολής.

20. Τρόποι εξυπηρέτησης περιφερειακών συσκευών

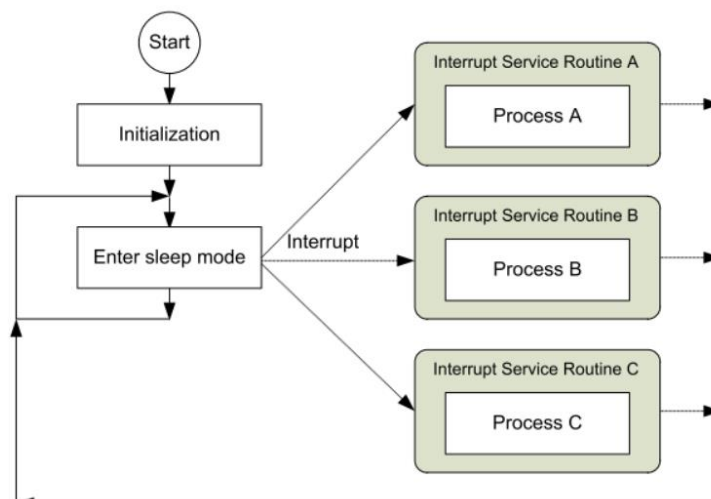
Στο παρακάτω σχήμα φαίνεται η εξυπηρέτηση περιφερειακών με τη βοήθεια του χρονοπρογραμματισμένου ελέγχου (polling). Αυτός ο τρόπος εξυπηρέτησης περιφερειακών συσκευών είναι χρονοβόρος, διότι υπάρχει μεγάλη καθυστέρηση, αφού απαιτείται ο έλεγχος όλων των περιφερειακών μονάδων και η εξυπηρέτηση μόνο μιας.

Processing with Polling



Στο παρακάτω σχήμα φαίνεται η εξυπηρέτηση περιφερειακών με τη βοήθεια διακοπών (interrupts). Ο επεξεργαστής βρίσκεται σε κατάσταση sleep mode και όποια μονάδα (περιφερειακό) θέλει εξυπηρέτηση, στέλνει σήμα διακοπής (INT) στον επεξεργαστή και τον διακόπτει. Στη συνέχεια εκτελείται μια Ρουτίνα Εξυπηρέτησης Διακοπών (ISR = Interrupt Service Routine) για να εξυπηρετήσει τη διακοπή που προκλήθηκε. Όταν ολοκληρωθεί μια ISR, ο έλεγχος του προγράμματος επιστρέφεται στον επεξεργαστή. Αυτός ο τρόπος εξυπηρέτησης περιφερειακών συσκευών είναι καλύτερος από τον προηγούμενο, διότι ο επεξεργαστής δεν χρειάζεται να ελέγχει όλες τις περιφερειακές συσκευές, όπως πριν.

Processing with Interrupts

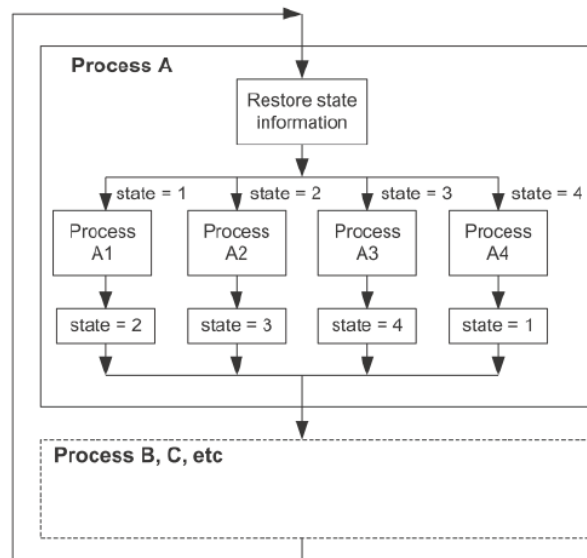


Είναι δυνατό να συνδυαστούν το polling με τα interrupts.

21. Τρόποι εκτέλεσης διεργασιών

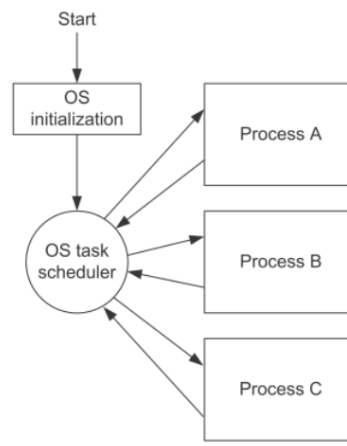
Στην εργασία με συστήματα μεγάλου όγκου συχνά συμβαίνει η επεξεργασία δεδομένων να γίνεται επιζήμια για την υγεία του συστήματος. Εάν επιτρέπουμε μόνο μία διαδικασία να λειτουργεί στα δεδομένα μας, αντιμετωπίζουμε πολλές προκλήσεις: Η διαδικασία μας μπορεί να καθυστερήσει. Για αυτούς τους λόγους, **θέλουμε να σχεδιάσουμε ένα σύστημα που επιτρέπει σε N αριθμό διεργασιών να λειτουργούν σε ένα μόνο σύνολο δεδομένων**. Κάνουμε κατανομή του φόρτου εργασίας σε N επεξεργαστές. Στο παρακάτω σχήμα φαίνεται η υποδιαίρεση μιας εκτελούμενης διεργασίας (Process A) σε επιμέρους διεργασίες (Process A1 – Process A4), **οι οποίες εκτελούνται παράλληλα, κάθε μια από διαφορετικό επεξεργαστή**. Για παράδειγμα, η διεργασία A υποδιαιρείται σε 4 επιμέρους διεργασίες και κάθε μια όταν ολοκληρωθεί, θέτει μια μεταβλητή κατάστασης (state) ίση με μια τιμή, που δείχνει τη σειρά ολοκλήρωσης τους και όταν έχουν ολοκληρωθεί **όλες** οι επιμέρους διεργασίες, ο επεξεργαστής συνεχίζει με την επόμενη διεργασία (B, C) κ.λ.π.

Process Partitioning



RTOS (Real Time Operating System)

|| RTOS



Ένα λειτουργικό σύστημα πραγματικού χρόνου, κοινώς γνωστό ως RTOS, είναι ένα στοιχείο λογισμικού που εναλλάσσεται γρήγορα μεταξύ των διεργασιών, δίνοντας την εντύπωση ότι πολλαπλά προγράμματα εκτελούνται ταυτόχρονα σε

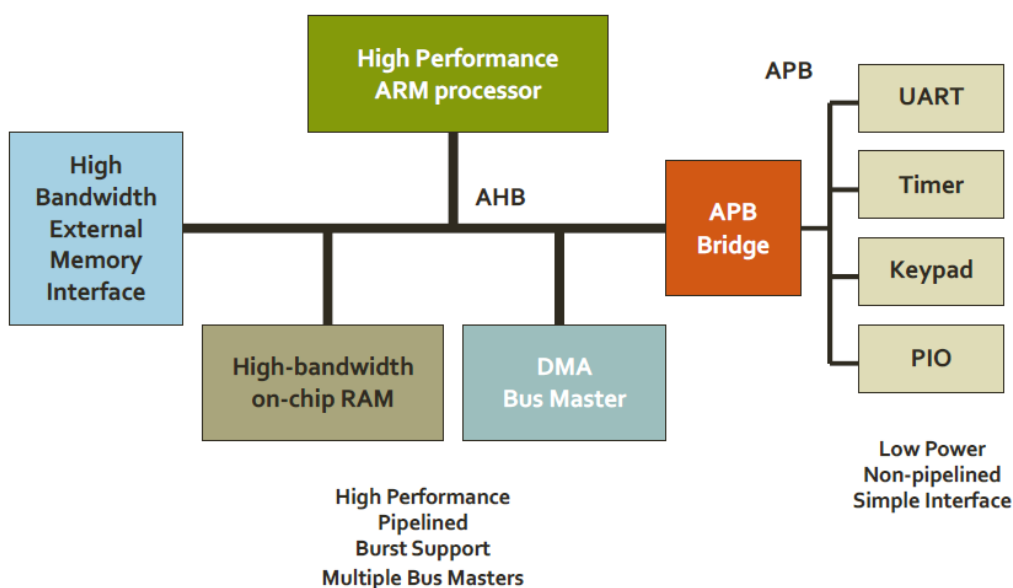


έναν μόνο πυρήνα επεξεργασίας. Στην πραγματικότητα, **ο πυρήνας επεξεργασίας μπορεί να εκτελέσει μόνο ένα πρόγραμμα κάθε φορά και αυτό που κάνει το RTOS αλλάζει γρήγορα** μεταξύ μεμονωμένων νημάτων προγραμματισμού (ή Tasks) για να δώσει την εντύπωση ότι πολλά προγράμματα εκτελούνται ταυτόχρονα

Συγκρίνοντας τους δύο προηγούμενους τρόπους επεξεργασίας, αναφέρουμε ότι ο **πρώτος είναι πιο γρήγορος**, αφού κάθε **επιμέρους διεργασία εκτελείται από διαφορετικό επεξεργαστή**, αλλά έχει **μεγαλύτερο κόστος υλικού (λόγω της ύπαρξης περισσότερων πυρήνων)** σε σχέση με το δεύτερο.

22. Επεξεργαστής Cortex – M0

Το παρακάτω διάγραμμα απεικονίζει ένα μπλοκ διάγραμμα ενός συστήματος SoC. Ο επεξεργαστής ARM υψηλής απόδοσης είναι ο πυρήνας του συστήματος. Τα άλλα στοιχεία είναι α) High-bandwidth on-chip RAM, b) DMA bus master c) το High-bandwidth Memory Interface συνδέονται με τον επεξεργαστή, μέσω διαύλου που είναι ο AHB. Τα άλλα περιφερειακά χαμηλού εύρους ζώνης όπως το UART, το Timer, το πληκτρολόγιο και το PIO συνδέονται στο δίαυλο συστήματος μέσω της γέφυρας, χρησιμοποιώντας την αρτηρία APB. Σε αυτό το σενάριο, η γέφυρα που χρησιμοποιείται για να διασυνδέσει τα δύο διαφορετικά πρωτόκολλα μεταξύ τους (δηλ. το AHB και το APB), λειτουργεί ως Slave από την μεριά του επεξεργαστή και ως Master από τη μεριά των εξωτερικών περιφερειακών χαμηλού εύρους ζώνης.



Εικόνα 10: Σύστημα AMB

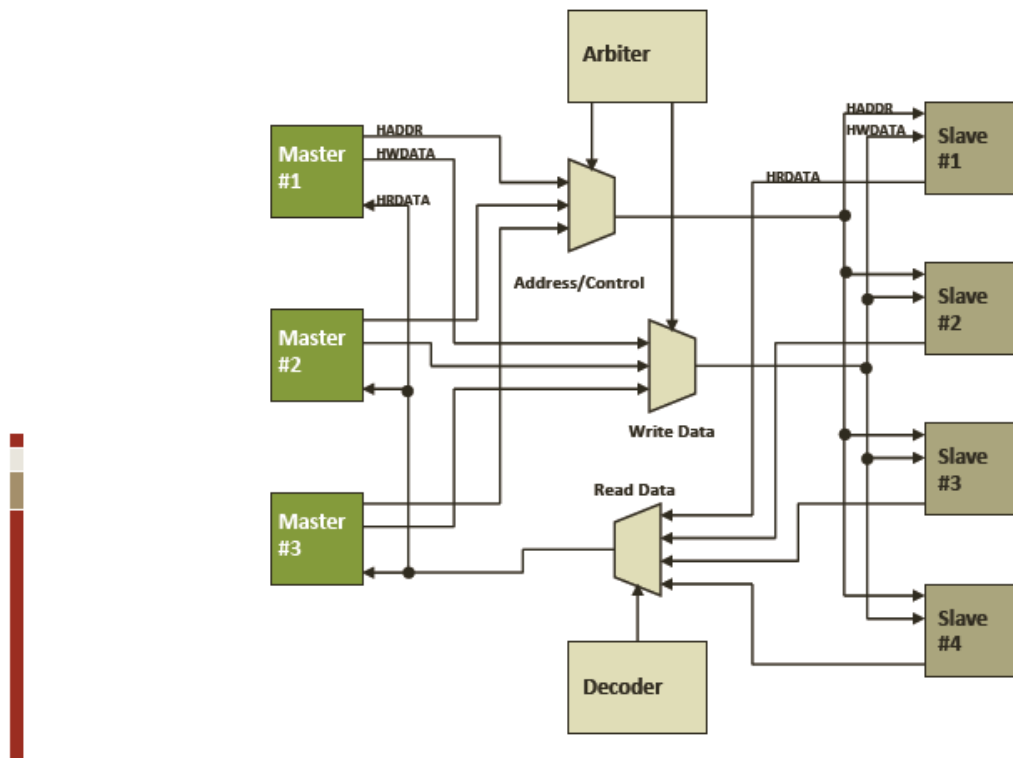
Η **ARM Advanced Microcontroller Bus Architecture (AMBA)** είναι μια ανοιχτή προδιαγραφή, on-chip διασύνδεσης για τη σύνδεση και τη διαχείριση λειτουργικών μονάδων σε ένα SoC (System on Chip). Διευκολύνει την ανάπτυξη σχεδίων πολλαπλών επεξεργαστών με μεγάλο αριθμό ελεγκτών και εξαρτημάτων με αρχιτεκτονική διαύλου. Σήμερα, το AMBA χρησιμοποιείται ευρέως σε μια σειρά επεξεργαστών ειδικού σκοπού (ASIC) και SoC, συμπεριλαμβανομένων επεξεργαστών εφαρμογών που χρησιμοποιούνται σε σύγχρονες φορητές συσκευές, όπως smartphone. Είναι μια διεπαφή χαμηλού κόστους και βελτιστοποιείται για ελάχιστη κατανάλωση ενέργειας και μειωμένη πολυπλοκότητα διεπαφής. Σε αντίθεση με το AHB, είναι ένα πρωτόκολλο Non-Pipelined, που χρησιμοποιείται για τη σύνδεση περιφερειακών χαμηλού εύρους ζώνης. **Κυρίως, χρησιμοποιείται για τη σύνδεση του εξωτερικού περιφερειακού με το SoC.**

Το **πρωτόκολλο APB** έχει σχεδιαστεί για προσπελάσεις ελέγχου χαμηλού εύρους ζώνης, για παράδειγμα σε περιφερειακές συσκευές. Έχει χαμηλή πολυπλοκότητα και χαμηλό εύρος bit (32 bit)

Το **πρωτόκολλο AHB** είναι ένα πρωτόκολλο διαύλου που χρησιμοποιείται για να συνδέσει τον επεξεργαστή με γρήγορες συσκευές, όπως η μνήμη RAM

Μια απλή συναλλαγή στο AHB αποτελείται από μια φάση διεύθυνσης και μια επόμενη φάση δεδομένων (χωρίς καταστάσεις αναμονής: μόνο δύο κύκλοι διαύλου). Η πρόσβαση στη συσκευή στόχου ελέγχεται μέσω MUX (non-tristate), επιτρέποντας έτσι την πρόσβαση του λεωφορείου σε έναν κύριο bus-bus κάθε φορά.

Δομή AHB



Σε μεγάλα συστήματα, κάποιοι πόροι (resources) είναι διαμοιραζόμενοι από πολλά υποσυστήματα. Για παράδειγμα, μερικοί επεξεργαστές μπορεί να μοιράζονται το ίδιο block μνήμης ενώ πολλές περιφερειακές συσκευές μπορεί να είναι συνδεδεμένες με το ίδιο bus. Ο arbiter (εκτελεί διαιτησία αρτηρίας) είναι το κύκλωμα που επιλύει κάθε διένεξη (conflict) και συντονίζει την πρόσβαση στον εκάστοτε διαμοιραζόμενο πόρο (shared resource).